

učebnice jazyka

```
private final Board board;
private final Base64Converter converter;
private String errorMessage;
private boolean gameOver;

/**
 * Constructor with full converter
 * @param width A width of the board
 * @param height A height of the board
 * @param content An optional content taken from the method
 */
public Board(int width, int height, String content) {
    this(width, height, content, new Base64Converter());
}
```

a tvorba webových aplikací
pro samouky



Kniha byla zakoupena na serveru Palmknihy.cz.

Kupující: Andrea Stratilová

ID 2126862036

Upozorňujeme, že kniha je určena pouze pro potřeby kupujícího. Kniha jako celek ani žádná její část nesmí být volně šířena na internetu, ani jinak dále zveřejňována. V případě dalšího šíření neoprávněně zasáhnete do autorského práva s důsledky dle platného autorského zákona a trestního zákoníku. Neoprávněným šířením knihy poškodíte rozvoj elektronických knih v ČR. Tak nám, prosím, pomozte v rozvoji e-knih a chovejte se ke knize, k nakladatelům, k autorům a také k nám fér.

Učebnice jazyka Java a tvorba webových aplikací pro samouky
Pavel Ponec
<https://jbook.ponec.net/cs3>

Korektura prvního vydání: Google Gemini 1.5 Pro
Odborná korektura druhého vydání: Ing. Ondřej Guth, Ph.D.
Jazyková korektura druhého vydání: Ing. Vladimíra Krejčíková
Ilustrace obálky: Marie Brogowski
Vnitřní ilustrace: autor knihy (pokud není uvedeno jinak)

Vydavatel: Pavel Ponec, Brno
3. přepracované vydání (1. a 2. vydání vyšlo pod názvem Učebnice jazyka Java
na webových příkladech pro úplné začátečníky)
2026, leden
ISBN 978-80-909828-0-2



FAKULTA
INFORMAČNÍCH
TECHNologiÍ
ČVUT V PRAZE

PROPOJÍME TĚ SE SVĚTEM IT

Kvalitní vzdělání ve všech oblastech
informatiky od informačních systémů
a webových technologií přes
počítačovou grafiku až po hardware.



PROČ JÍT NA FIT?

- ▶ 100% pokrytí všech oblastí IT
- ▶ více jak 200 firem pro praxi při studiu
- ▶ 100% zaměstnanost na trhu práce
- ▶ 19 špičkově vybavených laboratoří
- ▶ 12 výzkumných skupin



BAKALÁŘSKÝ STUDIJNÍ PROGRAM

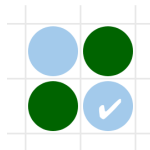
- ▶ titul bakalář (Bc.)
- ▶ standardní doba studia 3 roky
- ▶ 10 specializací

NEPOTŘEBUJEŠ PŘEDCHOZÍ ZNALOST IT. VŠE TĚ NAUČÍME.



Poděkování

Rád bych poděkoval své laskavé dceři za mnoho užitečných podnětů a připomínek, bez kterých by tato kniha neměla současnou podobu. Poděkování patří také všem mým dlouhodobým kolegům v týmu za celou řadu užitečných rad, nápadů a konzultací. Kniha byla vytvořena pomocí open-source aplikací včetně ukázkových příkladů, vektorových obrázků a nástrojů pro sestavení výsledku – to vše na svobodném operačním systému Linux. Tímto bych rád vyjádřil své uznání a poděkování také široké komunitě lidí, kteří se starají o vývoj, údržbu a distribuci open-source projektů. V neposlední řadě děkuji také doc. RNDr. Josefu Kolářovi, CSc., a Ing. Ondřeji Guthovi, Ph.D., z Katedry teoretické informatiky FIT ČVUT za pomoc při druhém vydání této knihy.



Obsah

Poděkování	1
1 Předmluva ke druhému vydání	7
2 Úvodní informace	8
2.1 Ochranné známky	8
2.2 Vyloučení odpovědnosti	8
2.3 Licence ukázkových příkladů	9
2.4 Typografie	9
2.5 Různé	9
3 Úvod a motivace	10
3.1 Proč vznikla tato kniha?	10
3.2 Co budeme potřebovat?	11
3.3 Proč se učit programovat?	11
3.4 Proč se učit právě jazyk Java?	12
3.5 Proč webové aplikace?	13
3.6 Pár slov o jazyku Java	13
4 Základní pojmy	15
4.1 Data	15
4.2 Algoritmus	15
4.3 Paměť	18
4.3.1 Ukládání čísel	18
4.3.2 Ukládání znaků	19
4.3.3 Ukládání logických hodnot	19
4.3.4 Jak počítač rozumí informacím v paměti	19
4.4 Proměnná	20
4.5 Primitivní datové typy a přetypování	20
4.6 Java výrazy a příkazy	22
4.7 Stromová struktura dat	24
4.8 Model reálného světa	25
4.9 Fyzikální objekt	25
4.10 Programový objekt	26
4.11 Třída objektů	26
4.11.1 Diagram modelu tříd	27
4.11.2 Název třídy	28
4.11.3 Atributy	28
4.11.4 Metody	28

4.11.5 Konstruktor	29
4.11.6 Třída ve zdrojovém kódu	29
5 Od třídy k reálnému kódu	31
5.1 Užití objektu	31
5.1.1 Texty v jazyce Java	31
5.1.2 Vytvoření instance a užití třídy	31
5.1.3 Autoboxing	33
5.1.4 Neplatné objekty null	34
5.1.5 Vztahy mezi třídami	35
5.2 Další poznámky ke třídám	38
5.2.1 Viditelnost metod, atributů a tříd	38
5.2.2 Dědičnost	39
5.2.3 Výjimky	42
5.2.4 Balíčky	45
5.2.5 Komentáře kódu a JavaDoc	46
5.2.6 Interface	48
5.2.7 Generické datové typy	49
5.2.8 Inline třídy a lambda výrazy	50
5.2.9 Třída typu Record	51
5.2.10 Knihovny	52
5.3 Datový typ pole	52
5.3.1 Pole vytvořené metodou třídy	53
5.4 Vybrané třídy standardní knihovny	54
5.4.1 Třída String	54
5.4.2 Rozhraní List	55
Třída ArrayList	56
List s typem prvku	57
5.4.3 Rozhraní Map	57
5.4.4 Rozhraní Stream	58
5.4.5 Třída BigDecimal	59
5.4.6 Shrnutí kapitoly o třídách	61
6 Webové technologie a pojmy	62
6.1 XML – rozšiřitelný značkovací jazyk	63
6.2 HTML – hypertextový značkovací jazyk	65
6.3 CSS – Kaskádové styly	66
6.4 URL – Adresa internetové stránky	69
6.5 JSON – úsporný datový formát	70

6.6 Shrnutí	71
7 Řešené příklady	72
7.1 Instalace	73
7.1.1 Hardware a operační systém	73
7.1.2 Apache Maven	74
7.1.3 Znakový terminál	75
7.1.4 Získání příkladů	75
7.1.5 Instalace vývojového prostředí	75
7.1.6 Spouštění příkladů	76
7.1.7 Co dělat při potížích?	76
7.1.8 Virtuální počítač	77
7.1.9 Virtualizační kontejner	78
7.1.10 Vývojové editory	79
7.2 Servlety	79
7.2.1 Hello, World!	80
První servlet	81
Výsledek servletu	84
7.2.2 Datový model HTML stránky	84
Připojení knihovny do projektu	87
7.2.3 Programové výrazy	88
7.3 Tvorba tabulek	90
7.3.1 Jednoduchá tabulka	90
Příkaz for	91
Inkrementace a dekrementace	92
Řešení úkolu	93
Alternativní procházení pole	94
Podmíněný příkaz if-else	95
Tabulka náhodných čísel	95
7.3.2 Přehled vozidel	97
7.4 Formuláře	99
7.4.1 Formulář s jedním vstupním elementem	99
7.4.2 Formulář s více vstupními elementy	101
Regulární výrazy	102
Formulář osobních údajů	103
Ternární operátor	106
Hodnocení platnosti textů	107
7.4.3 Počítání slov	108

7.5 Zábava	110
7.5.1 Vlastní rodič servletu	110
7.5.2 Bodové kreslení	111
Pomocná třída Base64Converter	113
Parsování textu	113
Class – třída pro popis tříd	114
Logování událostí	114
Třída Optional	115
7.5.3 Binární podoba textu	115
Řešení	116
7.5.4 Piškvorky s defenzivní strategií hry	118
Diagram tříd	119
Jak to funguje?	119
Enumerátor	121
Příkaz switch	123
Popis tříd diagramu a jejich metod	124
Zpracování dotazu	125
7.6 Interaktivní aplikace s AJAX	126
7.6.1 Co je to AJAX	127
7.6.2 Testování regulárních výrazů	128
7.6.3 Piškvorky s rychlejší odezvou	131
7.7 Herní strategie od umělé inteligence	134
7.7.1 Jak formulovat požadavek na herní strategii pro AI	134
7.7.2 Závěrečné kroky a autorská práva	135
7.7.3 Automatizované testy	135
7.8 Verzování zdrojového kódu	136
8 Úvod do databází a práce s daty	139
8.1 Základní pojmy relačních databází	139
8.2 Relace a entity	140
8.3 Jazyk SQL aneb jak mluvit s databází	141
8.3.1 Tvorba databázové tabulky	141
8.3.2 Vložení řádku	142
8.3.3 Čtení řádku	143
8.3.4 Změna záznamu	144
8.3.5 Mazání řádku	144
8.3.6 Databázové transakce	145
8.4 Programové rozhraní Javy	145

8.4.1 Základní rozhraní JDBC	145
8.4.2 Pokročilé rozhraní JPA	146
8.4.3 Jak usnadnit práci s JDBC	147
8.5 Hledání hotelů v jazyce Java	147
8.6 Diagram tříd	148
8.7 Životní cyklus databázového spojení	149
8.8 Čtení databáze v Java objektech	151
8.9 Vkládání Java objektů do databáze	153
8.10 Generická abstraktní třída	154
9 Jak psát dobrý kód	156
Reference a zdroje pro další studium	158
Rejstřík slov	161

1 Předmluva ke druhému vydání

Kniha, kterou právě čtete, je úvodem do světa tvorby aplikací. Nečekejte však suchý popis programovacího jazyka. Čtenář se na jednoduchých příkladech naučí, jak při tvorbě programů přemýšlet. Pokud vás v této souvislosti straší slovo „algoritmicizace“, kniha vás nezklame. Neobsahuje partie z teoretické informatiky, namísto toho je vše vysvětleno formou příkladů pochopitelných i pro naprosté nováčky ve světě tvorby software.

Tato publikace však návodem, jak při programování přemýšlet, nekončí. Čtenář se dozví, jak používat jazyk Java, jeho pravidla, a v neposlední řadě knihovny, které s tvorbou software pomohou. Tento jazyk je již dlouho osvědčenou volbou pro výuku základů programování, jeho znalost zároveň uplatníte i v praxi. Ani jazykem Java záběr knihy nekončí. V knize se dozvíte o základních technologiích webu a naučíte se tvořit webové aplikace.

Text zabíhá i mimo téma základů nutných pro vytvoření fungujícího programu. Dozvíte se, jak zdrojový kód organizovat, aby byl přehledný a pochopitelný, nebo proč a jak zdrojový kód obohatit o dokumentaci. Jistě oceníte popis aplikací, které s programováním pomohou, například editoru – vývojového prostředí. Seznámíte se i s dlouhodobě osvědčenou a stále používanou technologií, díky které je možné své softwarové projekty efektivně spravovat nebo sdílet s dalšími vývojáři. Kromě příkladů částí programu v textu je součástí knihy i hotová aplikace spolu s návodem na její zprovoznění. Vedle možnosti prohlédnutí všech detailů je tak možné se učit pomocí zkoušení si vlastních úprav.

Přeji příjemné vykročení do oblasti programování při čtení této knihy.

Ing. Ondřej Guth, Ph.D. v roce 2022

2 Úvodní informace

V této kapitole jsou uvedeny některé důležité informace, které však nemají přímý vliv na porozumění technickému výkladu.

2.1 Ochranné známky

Oracle, **Java** a **VirtualBox** jsou registrované ochranné známky společnosti Oracle Corporation anebo jejích poboček. Autor knihy nemá žádnou vazbu ke společnosti Oracle. **Apache Maven**, **Apache Tomcat**, **Apache NetBeans** a **NetBeans** jsou ochranné známky neziskové organizace Apache Software Foundation (ASF) [25]. **Ubuntu**, **Xubuntu** a **Canonical** jsou registrované ochranné známky společnosti Canonical Ltd. **Microsoft** a **Windows** jsou registrované ochranné známky společnosti Microsoft Corporation v USA anebo jiných zemích. **Docker** je registrovaná ochranná známka společnosti Docker, Inc., v USA anebo jiných zemích. Docker, Inc., a také další strany mohou mít práva k ochranné známce. **Eclipse** a **Jetty** jsou registrované ochranné známky společnosti Eclipse Foundation, Inc., ve Spojených státech a případně v dalších jiných zemích. **ECMAScript** je ochranná známka skriptovacího jazyka specifikovaná standardem ECMA international.

2.2 Vyloučení odpovědnosti

Mějme na paměti, prosím, že vzorové ukázky programového kódu byly vytvořeny za účelem výuky. Naznačují koncept, ale obsahují některá zjednodušení, například ošetřování vyhozených výjimek je z důvodu stručnosti omezené, a proto ukázky nejsou určeny přímo k produkčnímu použití.

I když autor věnoval značné úsilí odstraňování chyb a překlepů v textu, nelze vyloučit jejich přehlédnutí. Opravy, které byly zaregistrovány po vydání knihy, najdeme na její domovské stránce [1], kde lze nahlásit případné nesrovnalosti ve výkladu či jiné připomínky. Děkuji za všechny konstruktivní podněty.

2.3 Licence ukázkových příkladů

Po zakoupení knihy si prosím uschovejte doklad o nákupu, protože vás opravňuje používat ukázkové příklady k soukromým i komerčním účelům. Distribuce zdrojového kódu k této knize je možná pouze při zachování původního komentáře v hlavičce. Třetí strana je oprávněna použít zdrojový kód pouze pro své vlastní studium, jinak je třeba získat vlastní licenci zakoupením knihy nebo e-booku. Licence tak umožňuje pedagogům použití ukázkového projektu při výuce. V případě nejasností se obraťte na vydavatele knihy, nebo přímo na autora, kontakty najdete na internetové stránce knihy [1].

Zvažte, prosím, že výroba knihy není jedinou nákladovou položkou, další výdaje souvisí s propagací a distribucí. Děkuji tímto za podporu i pochopení.

2.4 Typografie

Slova psaná *kurzívou* budou označovat názvy produktů a nové technické termíny. Kvůli jejich vzrůstajícímu počtu však tyto termíny nebudou vyznačovány v celém rozsahu knihy, ale jen ve vybraných částech pro zdůraznění jejich specifického významu. Slova psaná *neproporcionálním* písmem (znaky s pevnou šířkou) značí programové výrazy a příkazy, klávesy nebo názvy souborů. V ukázkách zdrojového kódu jsou klíčová slova jazyka *Java* vyznačena modře (●), třídy objektů zeleně (●) a literály červeně (●).

2.5 Různé

Jména osob, uvedená v programových ukázkách v této knize, jsou smyšlená za účelem názorného výkladu a jejich případná podobnost s reálnými osobami je čistě náhodná.

3 Úvod a motivace

Write once, run anywhere.

— Sun Microsystems, americká společnost

Od roku 1991 usilovala firma *Sun Microsystems* o vytvoření jednoduchého, robustního, objektově orientovaného programovacího jazyka, jehož aplikace by byly nezávislé na platformě zařízení. Hlavní myšlenku reprezentoval slogan v záhlaví této kapitoly. Vedoucím projektu a původním autorem byl kanadský softwarový programátor James Gosling. Roku 1996 vydala firma první oficiální verzi jazyka pod názvem *Java* a o jedenáct let později (roku 2007) uvolnila většinu zdrojových kódů pod licencí **open-source**. Počátkem roku 2010 proběhla akvizice firmy *Sun Microsystems* společností *Oracle*, která se tak stala vlastníkem oficiální implementace Java SE platformy.

3.1 Proč vznikla tato kniha?

Když jsem s jazykem *Java* začínal, měl jsem pocit, že učebnice by se dala napsat i zábavnější formou. Po letech praxe jsem zveřejnil článek o tvorbě webových stránek v Javě pomocí objektového modelu HTML elementů. Napadlo mě, že právě to by mohl být skvělý základ pro interaktivní příklady při výuce. Když jsem pak v knihkupectví otevřel jednu z nových učebnic, znovu jsem narazil na formální popis, černobílé obrázky a ukázky postavené na zastaralé knihovně Swing. Proto jsem se rozhodl vykročit na nejistou dráhu autora inovativní učebnice programování. Výsledek máte právě před sebou.

Abych příklady co nejvíce přiblížil reálným aplikacím, ponechal jsem názvy programových konstrukcí (jako jsou *třídy*, metody či komentáře) v angličtině. Pro pochopení principů webových aplikací si vysvětlíme základy jazyka *HTML* (z anglického *Hypertext Markup Language*) a zmíníme i další technologie. Kniha má také přesah do oblasti návrhu a životního cyklu informačních systémů. Vzhledem k širšímu záběru témat se nebudeme zabývat všemi aspekty jazyka do nejmenších detailů. Místo toho se zaměříme raději na praktické využití, pochopení souvislostí a využití užitečných vlastností. Jsem si vědom, že ve snaze o čtivost textu jsem se občas dopustil určitého zjednodušení. Kniha tedy neposkytuje úplnou specifikaci jazyka, ale nabízí spíše nadhled potřebný pro další samostatné studium. Výklad se záměrně vyhýbá konstrukcím, které jsou sice platné, ale v praxi považované za nevhodné či zastaralé. Důležitým doplňujícím zdrojem informací je dokumentace standardní knihovny zvaná *JavaDoc* [5]. Užitečné rady a řešení problémů lze najít také na serveru *Stack Overflow* [15]. Pokud se podaří probudit ve čtenáři zájem o vývoj aplikací, splní jeden ze svých hlavních cílů.

3.2 Co budeme potřebovat?

Kniha předpokládá, že čtenář je běžným uživatelem internetu, zvládne instalaci počítačových programů (například hry nebo internetového prohlížeče), orientuje se v základních pojmech, jako jsou například *pevný disk* či *soubor* na disku, a rozumí organizaci uložených *souborů* na disku. První kapitola se zaměřuje na motivaci výuky programovacího jazyka *Java* a zmiňuje také výhody webových aplikací.

Pro lepší pochopení obsahu knihy doporučuji zkoušet si jednotlivé příklady na počítači a následně zkoumat vliv vlastních úprav zdrojového kódu na chování spuštěné aplikace. V případě potíží je vždy možné aktuální projekt nahradit původní verzí jejím rozbalením z archivu.

Co tedy budeme potřebovat pro spuštění ukázkových příkladů?

1. Počítač (či notebook) s volnou kapacitou RAM **minimálně** 1 GB a 1 GB **volného** místa na pevném disku s operačním systémem *Windows*, *Linux*, *MacOS*, případně i jiným, na který lze nainstalovat *Java Development Kit*.
2. Připojení k internetu nejen pro instalaci potřebných programů, ale i k automatickému stažení nezbytných knihoven během spuštění přiložených příkladů.
3. Prostředí *Java Development Kit*, verze 25, pro sestavení a spuštění příkladů.

Další podrobnosti lze najít v kapitole [O instalaci](#).

3.3 Proč se učit programovat?

Pokud se vám dostala tato kniha do ruky, zřejmě už jste nějakou motivaci našli. Pracovní trh se potýká s nedostatkem odborníků, a to nejen v oboru informačních technologií (dále jen IT). V mnoha oborech může být řešením nasazení automatizace či umělé inteligence, nic z toho se však neobejde bez vývojářů softwaru (SW), což ještě více posílí poptávku po IT odbornících. Běžným obchodníkům pomůže někdy i jednoduchá prezentace zboží (či služeb) na internetu s možností sestavení objednávky a následné platby.

Je dobré vědět, že zadání projektu i výsledek práce SW vývojáře se sdílí relativně snadno přes internet, a tak pokud někdo má dostatečné jazykové znalosti, a získá zároveň důvěru zákazníka, může dodávat svoji práci třeba na druhý konec světa z pohodlí svého domova.

3.4 Proč se učit právě jazyk Java?

Říká se s nadsázkou: „Kolik právníků, tolik názorů.“ U programovacích jazyků je to podobné: diskuze o výhodách různých programovacích jazyků bývají velmi emotivní, přitom každý argument může mít svůj díl pravdy. V této nepřehledné situaci může být pro začátečníka obtížné vybrat správný směr. Proč tedy *Java*?

1. Programovací jazyk *Java* patří na trhu práce dlouhodobě k nejžádanějším. Je tedy pravděpodobné, že čas strávený studiem se podaří brzy zhodnotit [6].
2. Hotové aplikace lze spouštět (při dodržení určitých pravidel) na různých operačních systémech (například *Windows*, *Linux*, *MacOS*), pokud je na nich k dispozici běhové prostředí zvané *Java Runtime Environment* (dále jen *JRE*). Takové schopnosti programovacího jazyka se označují výrazem multiplatformní. Standardní *Java* obsahuje také sofistikovanou knihovnu na tvorbu plnohodnotných desktopových aplikací pro podporované platformy.
3. Nástroje nezbytné pro vývoj a provoz standardních komerčních aplikací lze získat zdarma, včetně pokročilých editorů zaměřených na vývoj aplikací (dále jen IDE z anglického *Integrated Development Environment*), podporujících i sestavení projektu a jeho spuštění. Referenční implementace *Javy* s otevřenou licencí má název *OpenJDK* [4]. Při instalaci *Javy* však nejsme vázání na jediného dodavatele, její implementaci nabízí několik dalších společností.
4. Vývojáři *Javy* mají k dispozici rozsáhlou nabídku různých **knihoven** s otevřeným zdrojovým kódem (anglicky *open-source*), které pokrývají široké spektrum nejrůznějších témat, což šetří vývojářům drahý čas i zbytečnou práci. Jedním tématem se někdy zabývá hned několik knihoven, které se liší svým programovým rozhraním či implementací.
5. Kolem jazyka se vytvořila silná komunita, která přispívá ke včasnému odhalení případných chyb či komplikací.
6. Vývojáři jazyka *Java* dosud věnovali značnou pozornost zpětné kompatibilitě jazyka při vydávání jeho nových verzí. V praxi to znamená, že uživatel má velkou naději, že spustí starší *Java* projekt i na poslední verzi prostředí (anglicky *run-time*). Vývojáři zase mohou používat (kompilovat) starý zdrojový kód bez úprav v poslední verzi překladače.
7. Jazyk nabízí silnou typovou kontrolu, která urychluje odhalení některých chyb zápisu zdrojového kódu, a tím šetří čas potřebný na testování aplikace.
8. *Java* se řadí k programovacím jazykům, které za běhu aplikace uvolňují automaticky paměť po nepotřebných objektech, což urychluje vývoj a eliminuje některé problémy s vyčerpáním paměti. Uvolňování paměti má na starost algoritmus označovaný anglickým termínem *Garbage collector*. V určitých situacích (například během řízení časově kritických procesů) však může krátké

časové prodlení spojené s uvolněním paměti představovat handicap, který lze ovšem eliminovat výběrem vhodné strategie algoritmu *Garbage collector*.

9. Programovací jazyk *Java* je k dispozici vývojářům relativně dlouhou dobu a dnes už je standardem pro výuku objektového programování (OOP). Znalost principů objektového programování jazyka *Java* může usnadnit pochopení objektového přístupu také v jiných jazycích.

3.5 Proč webové aplikace?

Webové aplikace pro internetové prohlížeče se těší stále větší oblibě koncových uživatelů i provozovatelů a důvodů je hned několik.

1. Internetové připojení je dnes už snadno dostupné pro většinu uživatelů osobních počítačů i mobilních zařízení.
2. Nové internetové technologie umožňují vývojářům stále zlepšovat ergonomii webového uživatelského rozhraní, které se pomalu blíží komfortu desktopových aplikací.
3. Webové aplikace jsou na operačním systému prakticky nezávislé, klienti tedy mohou používat libovolné zařízení, které poskytuje internetový prohlížeč.
4. Distribuce nových verzí SW produktů k zákazníkovi má minimální náklady.
5. Pro útočníka je obtížné webovou aplikaci odcizit, nebo prolomit její licenční omezení.

3.6 Pár slov o jazyku Java

Java je objektový programovací jazyk se silnou typovou kontrolou. Objektově orientované programování (dále *OOP*) je specifický způsob zápisu zdrojového kódu do textových souborů popisujících *třídy*. Ty mají svůj *název*, *datovou složku* a skupinu *algoritmů*, jež jsou obsaženy v pojmenovaných útvarech zvaných *metody*. Vlastnostmi *OOP* se budeme zabývat později, předtím si však ještě ujasníme některé základní pojmy z informatiky.

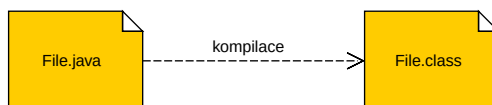
Zdrojový kód jazyka *Java* se zapisuje do textových souborů (s příponou *java*) podle pravidel, kterým se říká *specifikace jazyka* [3]. Jak se programovací jazyk vyvíjí, vznikají nové verze specifikace, které korespondují s určitým *Java* prostředím (instalovaným na počítači). Staré programy zpravidla fungují i v novějších prostředích *Javy*, této vlastnosti se říká *zpětná kompatibilita*. Příklady ke knize byly odladěny ve verzi *Java* 25, pravděpodobně je však bude možné používat (bez dalších úprav) i na verzích vyšších. Verze 25 *LTS* má dlouhodobou podporu.

Zdrojové soubory se ukládají do adresářové struktury *projektu* (ve *Windows* se souborovým *adresářům* říká *složka*). Stromová struktura projektu však není úplně libovolná, ale má svá pravidla – některá vycházejí ze specifikace jazyka. Více si řekneme v kapitole [O sestavení aplikace](#).

Při instalaci *Javy* se setkáváme se dvěma pojmy:

1. *Java Runtime Environment* (JRE) je minimalistické běhové prostředí, které potřebuje koncový uživatel programu ke spuštění aplikace vytvořené v jazyce *Java*. Běhové prostředí některé společnosti ke stažení nenabízejí, vývojáři si ho však mohou vytvořit sami. Podrobnější informace lze najít třeba na odkazu [\[33\]](#).
2. *Java Development Kit* (JDK) je nástroj pro vývojáře, který poskytuje *kompilátor* zdrojových kódů a některé další služby užitečné pro vývoj a ladění aplikace. Prostor *JRE* je automaticky obsaženo v *JDK*. Pro vývoj aplikací budeme potřebovat instalovaný produkt *JDK*.

Kompilátor v jazyce *Java* je program, jenž prověří správnou syntaxi (soubor pravidel) zápisu zdrojového kódu a převede ho do binární podoby (zvané *bytecode*), se kterou umí pracovat prostředí *JRE*. Kompilované soubory mají příponu `class`.



Obrázek 1. Znázornění kompilace souboru

Standardní kompilátor *JDK* se tím liší od některých jiných kompilátorů (i jiných jazyků), které převádějí zdrojový kód rovnou do binárního formátu, jenž obsahuje přímé instrukce procesoru počítače. Proces spuštění projektu si představíme v kapitole [O sestavení aplikace](#).



Nezaměňujme programovací jazyk *Java* se skriptovacím jazykem *JavaScript*, protože se jedná o dva **různé** jazyky. Zdrojový kód *JavaScriptu* se provádí (interpretuje) bez předchozí kompilace, což neumožňuje zachytit včas překlepy a typové nekonzistence. Interpretace skriptu bývá pomalejší. Výhodou *JavaScriptu* je jeho podpora v internetových prohlížečích.

4 Základní pojmy

Pokud chcete problém vyřešit, musíte jej nejdříve pojmenovat, a řešit příčinu, ne příznaky.

— Otto Wichterle, český vědec a vynálezce

Většina problémů, se kterými se (nejen) začínající programátoři setkávají, má své řešení někde na internetu, stačí ho najít. Klíčem k nalezení řešení bývá správná formulace vyhledávacího dotazu, ale ten se bez správných pojmů sestavuje jen obtížně.

4.1 Data

V oblasti výpočetní techniky jsou *data* libovolné informace vhodné k počítačovému zpracování. Nejmenší jednotkou dat je *bit*, který může nabývat pouze dvou stavů: (**ano/ne**) nebo (**pravda/nepravda**) nebo třeba (**přítomen/nepřítomen**), interpretace mohou být různé. Protože stav lze vyjádřit také pouhou číslicí (**1/0**, anglicky *digit*), nazývají se různá zařízení (stroje, přístroje), která interně pracují s čísly, občas výrazem *digitální zařízení* (číslicové stroje, přístroje). Skládáním *bitů* do větších celků vznikají datové útvary, jejichž hodnota opět závisí na interpretaci. Pro vyjádření většího objemu dat se běžně používá jednotka *bajt* (anglicky *byte*) obsahující 8 *bitů*. U těchto jednotek můžeme použít předpony soustavy SI (*Mezinárodní systém jednotek*) podle vzoru:

- *kilobajt* – označuje tisíc *bajtů*, zkratka je *kB*,
- *megabajt* – označuje milion *bajtů*, zkratka je *MB*,
- *gigabajt* – označuje miliardu *bajtů*, zkratka je *GB*.

Předpony velkých čísel využijeme například při **kontrole** volného místa na pevném disku.

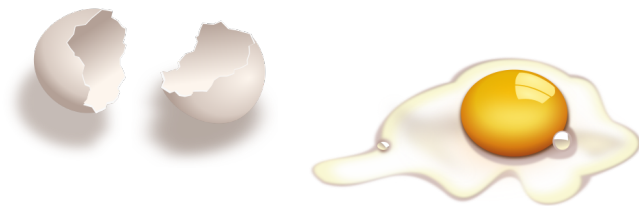
4.2 Algoritmus

Výklad pojmu *algoritmus* začneme jednoduchou definicí:

Algoritmus je návod či postup, kterým lze vyřešit daný typ úlohy. Přepis algoritmu do programovacího jazyka se nazývá programování.

— inspirováno zdrojem [10]

Jistou podobnost k *algoritmu* lze najít třeba v kuchařském receptu na přípravu pokrmu. Vezměme si například přípravu míchaných vajec pro dvě osoby.



Obrázek 2. Ilustrativní obrázek, upravený zdroj [9]

Postup přípravy lze rozepsat do následujících kroků:

1. Připravíme si suroviny: 4 vejce, šunku, olej a sůl.
2. Na pánvi ohřejeme olej na 160 °C.
3. Přidáme šunku nakrájenou na kostičky.
4. Přidáme vejce.
5. Mícháme po dobu 3 minut.
6. Pokrm podáváme na talíři teplý a osolený.

Tento recept jistě není zcela přesný, ale zkušená kuchařka ví, že vejce je třeba zbavit nejdříve skořápky a sporák je nutné nakonec vypnout; stroj to však neví. Z příkladu je také zřejmé, že pro srozumitelný popis přípravy pokrmu se neobejdeme bez jisté míry abstrakce. Hned první krok by se dal rozepsat samostatným algoritmem:

1. Vyhledáme v lednici potraviny na přípravu míchaných vajec.
2. Pokud některá z potravin chybí (kontrola obsahu lednice), pošleme člena domácnosti na nákup chybějících surovin.
3. Požadované suroviny odneseme na kuchyňskou linku.

Také **proces nakupování** by se dal popsat samostatným algoritmem a tímto způsobem bychom mohli pokračovat směrem ke stále podrobnějším detailům. Pojem *kroky algoritmu* lze chápat jako *příkazy*, které v jazyce Java realizujeme voláním metod. Každý příkaz přebírá odpovědnost za nějakou dílčí část algoritmu. Na našem příkladu vidíme, že součástí algoritmu mohou být také *příkazy* zpracované za určité podmínky (pokud nějaká potravina chybí, *běž nakoupit*).

Pokud pošleme člena rodiny na nákup, s dalším postupem přípravy pokračujeme až po jeho návratu. V běžném životě však zpravidla nečekáme, až se někdo vrátí, ale

třeba nachystáme jídelnu. Čtyřjádrový procesor si můžeme představit jako rodinu se čtyřmi členy a zpracování algoritmů několika jádry procesoru, které umožňuje *paralelní zpracování*. Toto téma však přesahuje plánovaný rámec knihy, a tak budeme dál uvažovat o všech algoritmech pouze v režii jediného jádra procesoru. Dopad na praktický život by to mělo ten, že nákup chybějících potravin zajistí kuchař sám.

Někdy nastane situace, že kuchař není schopen připravit požadovaný počet porcí najednou — třeba kvůli omezenému objemu kuchyňského nádobí. V tom případě můžeme postup vaření (*algoritmus*) opakovat tak dlouho, dokud počet porcí neodpovídá očekávání (*podmínka*). Procesu opakování se říká *iterace*. Protože však nakupování potravin, během každé iterace, by neúměrně zpomalilo čas přípravy, společný nákup lze provést jen jednou. Procesu urychlení pak můžeme říkat *optimalizace* algoritmu.



Programové *příkazy* se běžně provádějí shora dolů (nebo zleva doprava, pokud je více *příkazů* na jednom řádku). Každý *příkaz* se obvykle spustí až po dokončení předchozího. Pro tuto chvíli pomejme příkazy jazyka pro *opakované* či *souběžné* zpracování na více procesorech.

Jak už jsme naznačili před chvílí, algoritmy se v jazyce *Java* zapisují (v textovém formátu) do útvarů zvaných *metody*. Každá metoda má svůj **název**, může vyžadovat seznam **parametrů** a může vracet návratovou hodnotu. Parametry umožňují *předávání dat* do metody z místa, kde je metoda volána svým jménem. Nákup potravin by vedl k návrhu metody s těmito vlastnostmi:

1. název metody: „běž nakoupit“
2. parametr metody: „seznam potravin“
3. návratový objekt: „nákupní taška se zakoupenými potravinami“

Z pohledu kuchařky je však popis *algoritmu* nákupu nezajímavý. Kuchařka potřebuje pouze vědět, jak se *metoda* pro zajištění surovin jmenuje a jak ji má použít.



Zapamatujme si, že při počítačovém zpracování se využívají *algoritmy* (například recept na přípravu pokrmů) a *data* (nákupní seznam, potraviny, přepravní taška). Průchod algoritmem lze (za nějaké podmínky) opakovat pomocí *iteračních* příkazů programovacího jazyka.

4.3 Paměť

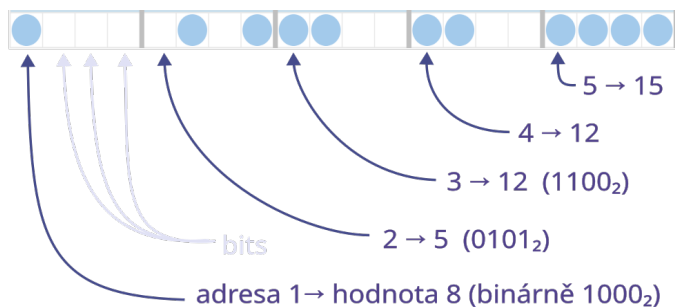
Paměť je deník, který všichni nosíme s sebou.

— Oscar Wilde, irský dramatik

Počítačová *paměť* je místo, kam si počítač ukládá informace, které potřebuje k práci – krátkodobě nebo natrvalo. Díky paměti může počítač uchovávat data i instrukce, podle kterých má pracovat. Existují dva hlavní druhy paměti:

1. **Paměť RAM (Random Access Memory)** je krátkodobá, „pracovní“ paměť počítače. Ukládají se do ní data a programy, se kterými počítač právě pracuje. Když se zařízení vypne, obsah paměti RAM se vymaže. Aby mohl program paměť používat, musí ji nejdříve *alokovat* – tedy požádat operační systém o přidělení volného prostoru. Poté získá adresu paměti, kam může data zapisovat. Adresa je v podstatě pořadové číslo paměťové buňky, které určuje umístění informace. Po dokončení práce je nutné paměť uvolnit, aby ji mohl použít jiný program. Různé programy potřebují různé množství paměti – například kalkulačka využije méně než webový prohlížeč.
2. **Pevná paměť (např. pevný disk nebo SSD)** je trvalá paměť, která uchovává data i po vypnutí počítače. Jsou zde uloženy všechny soubory, dokumenty, aplikace i operační systém počítače.

4.3.1 Ukládání čísel

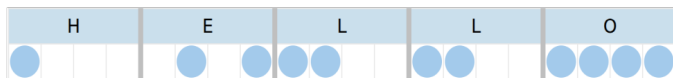


Obrázek 3. Číselné hodnoty v paměti

Každý malý čtvereček na obrázku představuje jeden **bit**, který si pamatuje dva stavy: například „ano“ (má kuličku) a „ne“ (nemá ji). Pokud tyto stavy vyjádříme počtem obsažených kuliček (1 nebo 0), lze na spojení více bitů nahlížet jako na *dvojkovou číselnou soustavu*. Ve **čtyřech bitech** lze uložit 16 různých čísel (v rozsahu od 0 do 15 včetně). Kuličky z první adresy (1) pak můžeme přepsat do binárního čísla 1000, které lze převést (třeba kalkulačkou) na číslo desítkové

soustavy s hodnotu 8. Pro přístup k hodnotě potřebuje počítač znát její **adresu**. Není však nutné znát adresy všech jednotlivých bitů. Pro skupinu bitů stačí znát adresu pouze toho prvního. Moderní počítače adresují paměť většinou po 8 **bitech**, které tvoří jeden **byte**.

4.3.2 Ukládání znaků



Obrázek 4. Ukládání znaků v paměti

Pro ukládání znaků využíváme **znakovou sadu**, což je prakticky očíslovaný seznam obrázků. Pro vykreslení znaku na obrazovku (či tiskárnu) pak počítač vyhledá příslušný obrázek pomocí jeho čísla. Grafická podoba sady znakových obrázků se nazývá **font**. Pro lepší představu si můžete vyzkoušet interaktivní formulář z kapitoly **Binární podoba textu**, který je součástí příkladů.

- 1 → A ... znamená, že číslo 1 odkazuje na znak A
- 2 → B ... číslo 2 odkazuje na znak B
- 12 → L ...
- 15 → O

4.3.3 Ukládání logických hodnot

Logická hodnota vyjadřuje, zda je určité tvrzení pravdivé, či nikoliv. K jejímu uložení by stačil v principu **jeden bit**. Počítač však nedokáže adresovat jednotlivé bity a tak musí použít celou nejmenší adresovatelnou jednotku (*bajt*). Skupinové adresování pomáhá šetřit počet paměťových adres.

- 0 → stav „ne“ (není to pravda)
- 1 → stav „ano“ (je to pravda)

4.3.4 Jak počítač rozumí informacím v paměti

Počítač sám o sobě nezná význam uložených bitů; potřebuje informaci o tom, jak je má interpretovat. Tuto informaci poskytuje *datový typ*, který říká, zda se mají bity číst jako **číslo**, **písmeno** nebo **logická** (pravdivostní) hodnota. Datové typy určuje *programovací jazyk*. Podrobnější vysvětlení přijde v kapitole o **datových typech**.

4.4 Proměnná

Pojem *proměnná* označuje *symbolické jméno* pro úschovu informace v paměti RAM za běhu programu. Proměnnou si lze představit jako krabici na úschovu předmětů: pro uložení sezonní obuvi použijeme krabici na boty opatřenou štítkem „*Petrovy zimní boty*“. Podle štítku pak krabici snadno najdeme a její obsah využijeme. Jedna krabice na boty však pojme právě jeden pár; vložením nového páru o původní obsah přijdeme. Pro úschovu mýdla si přirozeně zvolíme menší krabičku. Podobnost s krabicemi nejlépe vystihují *primitivní* datové typy. Vysvětlení pojmu bude brzy následovat.



Při tvorbě programů používáme pro úschovu dat *proměnné*. Pamatujme, že využití proměnné má omezenou platnost v rámci nějakého bloku, detaily probereme později.

4.5 Primitivní datové typy a přetypování

V jazyce *Java* rozlišujeme tři základní skupiny datových typů:

- *primitivní typy*,
- *třídy* objektů a
- *pole*.

V jazyce *Java* má každá **proměnná** svůj pevně daný datový typ, který se uvádí při její deklaraci. Tento typ může kompilátor také odvodit automaticky na základě první přiřazené hodnoty, v takovém případě zápis typu v kódu nahradíme klíčovým slovem **var**. Pod pojmem *deklarace* si představme určité **prohlášení**, při programování třeba o datovém typu. Tato kapitola se věnuje **primitivním** typům, k ostatním se dostaneme postupně. Primitivní typy obsahují číslo z vymezeného rozsahu, případně jeho speciální interpretaci, jako je *znak* nebo *logická hodnota*. Rozsah hodnot primitivního typu je dán specifikací jazyka. *Java* nabízí celkem osm primitivních typů, jejich názvy začínají vždy *malým písmenem*. Slovo *literál* označuje ve zdrojovém kódu zápis konkrétní hodnoty. Například celočíselný literál s hodnotou **deset** zapíšeme jako **10**, literál desetinného čísla ve formátu **65.731F**. Literály v následující tabulce nejsou jedinou možnou formou zápisu, více informací najdete v odkazu [19]. Číselné limity v tabulce zapsané ve vědecké notaci jsou zaokrouhleny. *Java* umožňuje získat přesné mezní hodnoty i během běhu programu. Význam posledního sloupce si vysvětlíme v souvislosti s funkcí **Autoboxing**.

Tabulka 1. Přehled primitivních datových typů jazyka Java

#	Typ	Literál/ Konstanta	Rozsah hodnot	Nejmenší hodnota	Největší hodnota	Objektový ekvivalent
1.	<code>boolean</code>	<code>true</code>	1 bit	<code>false</code>	<code>true</code>	<code>Boolean</code>
2.	<code>byte</code>	<code>65</code>	8 bitů	-128	127	<code>Byte</code>
3.	<code>short</code>	<code>65</code>	2 bajty	-32 768	32 767	<code>Short</code>
4.	<code>char</code>	<code>'A'</code>	2 bajty	0	65 536	<code>Character</code>
5.	<code>int</code>	<code>2_000_000</code>	4 bajty	-2.1e9	2.1e9	<code>Integer</code>
6.	<code>long</code>	<code>65L</code>	8 bajtů	-9.2e18	9.2e18	<code>Long</code>
7.	<code>float</code>	<code>65.7313F</code>	4 bajty	-3.4e38	3.4e38	<code>Float</code>
8.	<code>double</code>	<code>65.731</code>	8 bajtů	-1.8e308	1.8e308	<code>Double</code>

Popis hodnot

- `boolean` – logická hodnota; nabývá pouze stavů `true` (pravda) a `false` (nepravda).
- `byte` – nejmenší celočíselný typ (česky *bajt*); stačí pro *znakovou sadu ASCII*, ale nepokryje všechny národní znaky. Velikost datových souborů se často uvádí jako násobek tohoto typu. Celočíselný *literál* lze do tohoto typu přiřadit přímo, pokud je hodnota v požadovaném rozsahu.
- `short` – krátké celé číslo o velikosti 16 bitů. Tento typ přijímá také celočíselné *literály* přímo, pokud je hodnota v požadovaném rozsahu.
- `char` – reprezentuje grafický symbol, jako je písmeno, číslice, mezera nebo piktogram. Povolené znaky definuje standard *Unicode* [11]. Hodnota tohoto typu je ve skutečnosti pořadové číslo znaku z tabulky. Při zobrazování se však na monitoru vykreslí příslušný grafický symbol. Více najdeme v kapitole o [sestavování textu](#).
- `int` – nejpoužívanější celočíselný typ v Javě. Mezní hodnoty v tabulce jsou orientační. Hodnota se zapisuje jako číslo bez desetinné části. Pro lepší čitelnost lze v číselných literálech používat podtržítko (`_`), které nemá na hodnotu vliv.
- `long` – velké celé číslo; literál se doplňuje znakem `L`.
- `float` – desetinné číslo s plovoucí čárkou; doplňuje se znakem `F`. Jeho přesnost je omezená, proto může při výpočtech docházet k zaokrouhlování.
- `double` – velké desetinné číslo; lze (volitelně) doplnit znakem `D`. I zde je přesnost omezená a může docházet k zaokrouhlování.

Zdrojový kód 1. Ukázka zápisu primitivní hodnoty do proměnné

```
int age = 25; ①  
double price = 19.99; ②  
char code = 'A'; ③  
var amount = 77; ④
```

- ① Zápis čísla do nové proměnné s názvem **age**. Operátorem přiřazení je znak rovnítko (=). Nalevo je uveden **datový typ** a **název proměnné**, napravo číselný literál. Příkaz se zakončuje středníkem (;).
- ② Literál typu **double** uložíme do proměnné **price**.
- ③ Literál typu **char** uložíme do proměnné **code**.
- ④ Klíčové slovo **var** označuje, že typ proměnné si kompilátor odvodí z přiřazené hodnoty. Proměnná **amount** tak bude typu **int**. Typ proměnné nelze později změnit. Moderní editory (IDE) zobrazují typ proměnné již při psaní. Pro přehlednost budeme v této knize používat hlavně **explicitní** deklarace typů.

Někdy je nutné změnit typ hodnoty na jiný, což nazýváme *typová konverze* nebo *přetypování*. Číselné hodnoty lze snadno přiřadit do proměnných s **přesnějším** typem. Pokud však hrozí ztráta informace, musíme použít **explicitní** přetypování v kulatých závorkách.

Zdrojový kód 2. Příklad přetypování čísla.

```
long age = 25; ①  
byte code = (byte) age; ②  
short value = (short) 654321; ③  
int price = (int) 19.99; ④
```

- ① Literál typu **int** se na typ s **větším** rozsahem přetypuje automaticky.
- ② Pro přetypování na typ s **menším** rozsahem musíme hodnotě předřadit cílový typ v závorkách.
- ③ Při explicitním přetypování na menší rozsah může dojít k odříznutí bitů a vzniku nekorektního výsledku **-1039**. Tento příklad slouží jako varování před nevhodným přetypováním.
- ④ Přetypováním desetinného čísla na **celé** dojde k odříznutí desetinné části.

4.6 Java výrazy a příkazy

Pojem *Programový výraz* si lze představit jako matematický **vzorec**, jehož vyhodnocením získáme **výsledek**. Přesněji jde o literál, proměnnou, volání metody nebo jejich kombinaci s využitím **operátorů**. Pokud výsledek výrazu zapíšeme do

proměnné, vytvoříme *přiřazovací příkaz* zakončený středníkem. Vedle *přiřazovacích příkazů* rozeznáváme ještě *příkazy volání metod* a *příkazy jazyka*.

Připomeňme, že zápis hodnoty do proměnné vyjadřujeme rovnítkem (=), kde levá strana určuje cíl zápisu a pravá strana jeho obsah. Při prvním použití proměnné uvádíme její datový typ; lze ji však deklarovat i bez okamžitého přiřazení.



Zapamatujme si, že kód zakončený středníkem (uvnitř metody) představuje **příkaz**. Existují však i příkazy jazyka, kde se středník neuvádí.

Ve sloupci **Kód** následující tabulky najdeme vybrané ukázky výrazů a příkazů, ukázky bez středníku reprezentují výraz. Zalamování řádků v mezerách je pro kompilaci nevýznamné.

Tabulka 2. Ukázka výrazů a jejich zápisu do proměnných

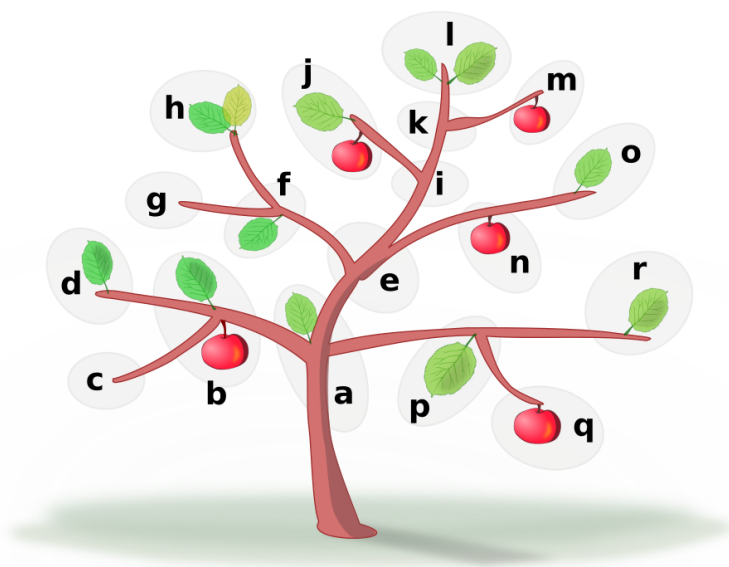
# Kód	Výsl. Typ	Poznámka
1. <code>boolean agreement = true;</code>	true boolean	Přiřadí logický <i>literál</i> do nové proměnné <i>agreement</i> (česky <i>souhlas</i>).
2. <code>char myCharacter = 'A';</code>	A char	Zápis znaku <i>A</i> ; znakové literály se ohraničují jednoduchou uvozovkou.
3. <code>int count = 5;</code>	5 int	Zápis čísla <i>5</i> do nové proměnné <i>count</i> (počet).
4. <code>int goodsPrice = count * 100;</code>	500 int	Výpočet ceny zboží pomocí součinu. Hvězdička <i>*</i> je operátor násobení, lomítko <i>/</i> slouží pro dělení.
5. <code>int myCash = 2 * 100 + 3 * 20 + 1;</code>	261 int	Výpočet mé hotovosti. Operátor plus <i>+</i> provádí sčítání, minus <i>-</i> odčítání. Násobení má přednost (precedenci) před sčítáním.
6. <code>boolean shopping = myCash >= goodsPrice;</code>	false boolean	Porovnání hotovosti. Použít lze operátory <i><</i> , <i>></i> , <i><=</i> , <i>>=</i> , <i>==</i> (rovná se) a <i>!=</i> (nerovná se).
7. <code>myCash = myCash + 1_000;</code>	1261 int	Změna hodnoty existující proměnné: k původní hodnotě přičteme 1000.
8. <code>myCash += 1_000;</code>	2261 int	Zkrácený zápis přičtení hodnoty. Analogicky lze použít <i>--</i> , <i>*</i> a <i>/=</i> .
9. <code>shopping = myCash >= goodsPrice;</code>	true boolean	Nová kontrola hotovosti po navýšení; výsledek se změnil na <i>true</i> .
10. <code>myCash -= goodsPrice;</code>	1761 int	Odečtení ceny po zaplacení zboží.
11. <code>myCash <= Short.MAX_VALUE</code>	true boolean	Srovnání s maximální hodnotou typu <i>short</i> . Minimální hodnotu získáme přes <i>Short.MIN_VALUE</i> .
12. <code>boolean evenNumber = (myCash % 2) == 0;</code>	false boolean	Operátor <i>%</i> (<i>modulo</i>) vrací zbytek po dělení. Výsledek <i>0</i> u dělení dvěma značí sudé číslo.
13. <code>Math.max(2, 3)</code>	3 int	Volání funkce, která vrátí větší z uvedených hodnot.

# Kód	Výsl. Typ	Poznámka
14. <code>2 + 3 == 6 - 1</code>	true boolean	Porovnání výsledků dvou operací. Matematické operátory mají přednost před porovnáním.
15. <code>2 + 3 <= 6 - 2</code>	false boolean	Porovnání dvou číselných výrazů.
16. <code>true false</code>	true boolean	Logický součet se vyznačuje párem svislých znaků (anglicky <i>pipe</i>).
17. <code>true && false</code>	false boolean	Logický součin se vyznačuje párem znaků ampersand (anglicky <i>ampersand</i>).
18. <code>! false</code>	true boolean	Negace logického výrazu.

Znak podtržítka v číselném literálu se používá pro grafické oddělení, jeho použití není povinné. Velikost hodnot primitivních typů lze porovnávat pomocí operátorů: `<` (je menší než), `>` (je větší než), `<=` (menší či rovno), `>=` (větší či rovno), `==` (rovná se), `!=` (nerovná se). Výsledkem porovnání je logická hodnota typu `boolean`. Zde jsou uvedeny jen vybrané operátory. Vlastní preference lze vynutit kulatými závorkami. Primitivní typy jsou výhodné pro svou nízkou paměťovou náročnost a přímou podporu operátorů. Další příklady najdete v kapitole [Programové výrazy](#).

4.7 Stromová struktura dat

Data lze seskupovat do struktur, které zrychlují vyhledávání nebo usnadňují manipulaci. Se **stromovou** strukturou se budeme setkávat často, proto si představíme základní pojmy.



Obrázek 5. Schéma stromu

Pro lepší představu o tomto datovém modelu si vezmeme na pomoc strom jabloně s jejími listy a plody. Místo, kde se větve dělí, se nazývá *uzel* (anglicky *node*). Tento pojem obecně označuje jakékoliv místo, ke kterému se připojují další potomci. Na obrázku jabloně to jsou další větve, plody či listy stromu.

Koncový *uzel*, který již nemá žádné potomky, se nazývá *list* (anglicky *leaf*). Každý uzel má právě jednoho *rodiče*, s výjimkou prvního uzlu, kterému říkáme *kořen* (anglicky *root*). Uzlům i listům můžeme přiřadit názvy. Spojnice mezi dvěma uzly se nazývá *hrana*. Přesnou adresu uzlu zapíšeme jako postupný seznam názvů od kořene, například oddělený tečkou.

```
a.e.f.h.žlutý_list
```

Podobně bychom mohli lokalizovat i červené jablko vpravo nahoře.

```
a.e.i.k.m.jablko
```

Stromovou strukturu má také adresa na poštovní obálce (stát → město → ulice). Jiným příkladem může být *hierarchická klasifikace organismů*, dalších příkladů by se našlo určitě více.

4.8 Model reálného světa

Vysvětlení pojmu *model* uvedeme krátkou definicí:

Model je zjednodušená reprezentace určitého objektu reálného světa či systému – pojatá z určitého úhlu pohledu.

— inspirováno zdrojem [10]

Děti si hrají s modely autíček nebo s panenkami, což jsou v napodobeniny předmětů reálného světa. Často se zachovává tvar či barva, zatímco jiné vlastnosti jsou potlačeny kvůli bezpečnosti nebo ceně. Pokud se model skládá z informací pro počítačové zpracování, říkáme mu *datový model*.

4.9 Fyzikální objekt

Při programování si *objekt* můžeme představit jako digitální model *tělesa*, který napodobuje jeho vlastnosti i chování. Slovem *těleso* se označuje hmotný předmět, který má své **místo** (či adresu), **velikost** v prostoru a také **čas** vzniku i zániku. Za *těleso* lze považovat libovolný konkrétní předmět (např. dům nebo veverku),

mezi *tělesa* však nepatří abstraktní pojmy jako chuze nebo spánek. Tělesa mají své **vlastnosti** (např. hmotnost, barvu nebo teplotu) a mohou poskytovat také **služby**. Příkladem může být automat na jízdenky, který za určitých podmínek vytiskne cestovní lístek. Pojem *těleso* pokrývá podmnožinu objektů reálného světa, ale kvůli některým podobnostem s počítačovými objekty ho připomeneme za chvíli.

4.10 Programový objekt

Programový objekt (v našem případě *Java objekt*) je datový **model** uložený v operační paměti počítače, který popisuje reálné či abstraktní objekty reálného světa pomocí *dat* a *algoritmů*. Popisuje reálné či abstraktní věci pomocí *dat* a *algoritmů*. Objekt může představovat konkrétní auto nebo třeba jen zelenou barvu. Podobně jako těleso má i počítačový objekt svou **adresu** v paměti, **velikost** (obsazené místo) a **čas** existence. Objekty v Javě vznikají příkazem a zanikají, když už nejsou potřeba (automatický sběr odpadu). Všechny zaniknou nejpozději při vypnutí počítače, pokud nejsou uloženy na disk. Programové objekty mají své vlastnosti a nabízejí služby, jako jsou výpočty nebo vyhledávání.



Programový objekt je základním stavebním kamenem pro tvorbu komplexních modelů reálného světa.

4.11 Třída objektů

Pokud mají dva objekty společné vlastnosti, říkáme, že patří do **stejně třídy**. Na parkovišti vidíme mnoho objektů sloužících k dopravě. Většina má motor, kola a volant, kufr – říkáme jim automobily. Najdeme tam ale i motocykly, které mají místo volantu řídítka a nemají kufr. Obě skupiny patří do třídy **motorových vozidel**.

Třída objektů (dále jen *třída*) je technický pojem, který označuje *datový typ objektu*. Definuje vlastnosti a nabízí služby prostřednictvím metod. Vznik konkrétního objektu zapisujeme odkazem na název třídy, proto se mu také říká *instance třídy*. Rozdíl mezi pojmy si ukážeme na třídě *Car*. Pokud máme instanci třídy *Car*, máme k dispozici *objekt* s jejími vlastnostmi. Pokud však získáme objekt typu *Car*, může to být i její *potomek*, protože třídy tvoří hierarchickou strukturu.



Zapamatujme si, že třídy definují vlastnosti v kódu, zatímco objekty vznikají až během běhu programu.

4.11.1 Diagram modelu tříd

Na tvorbě softwaru se podílí mnoho lidí – od manažerů po programátory a testery. Vývoj začíná odhadem ceny a následně prochází fázemi tzv. *životního cyklu projektu*. Tyto fáze zahrnují:

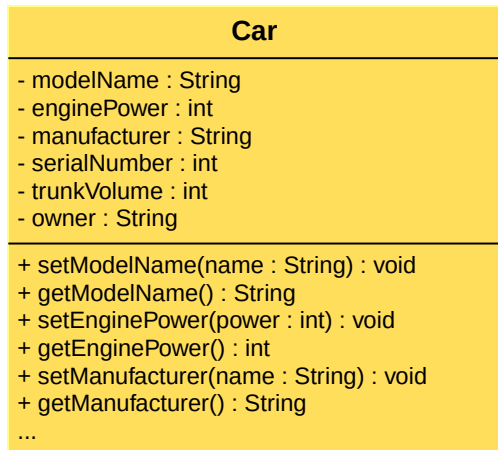
1. Sběr požadavků zákazníka.
2. Analýzu a návrh řešení.
3. Implementaci (psaní kódu).
4. Testování.
5. Nasazení produktu u zákazníka a jeho další údržbu.

Aby tým mohl spolupracovat, vytváří se dokumentace. Jedním z klíčových dokumentů je návrh tříd a jejich vztahů. K tomu se používá grafický jazyk *UML* (Unified Modeling Language) a jeho *diagram tříd* (anglicky *class diagram*).



Diagram tříd znázorňuje statickou strukturu datových typů, jejich atributy, metody a vzájemné vazby.

Představme si zadání pro **evidenci vozidel**. Zde je první návrh *třídy* pro automobil:



Obrázek 6. Grafické znázornění třídy pro automobil

Barevný obdélník představuje třídu. Barva má jen estetický význam a běžně se používá barva bílá. Obdélník je rozdělen vodorovnými čarami do tří pater, která popisují *název*, *atributy* a *metody*.



Diagramy *tříd* mohou být různě podrobné. Seznamy atributů a metod jsou nepovinné, stejně jako jejich typy. V nejjednodušší formě může diagram obsahovat jen název třídy.

Návrh *třídy* *Car* není dokonalý, ale teď nám dobře poslouží k vysvětlení pojmů. Vylepšovat ho budeme později.

4.11.2 Název třídy

Horní část diagramu obsahuje *název třídy*. Názvy *metod* se skládají z písmen a číslic, nesmí však číslicí začínat. Nesmí obsahovat Java operátory, mezery ani jiné bílé znaky. Názvy se nesmí shodovat s **rezervovanými slovy** (klíčovými slovy) Javy [26]. Velká a malá písmena jsou v Javě rozdílná (např. *Company* není totéž co *ComPany*). Tato pravidla platí i pro atributy a metody.

V Javě se doporučuje začínat název třídy **velkým písmenem**. Pokud je název složen z více slov, použije se notace zvaná *CamelCase* (česky velbloudí notace), kde spojovaná slova se uvádí velkým písmenem.

4.11.3 Atributy

Programový objekt má své vlastnosti zvané *atributy*, které jsou určeny datovým typem a názvem. Výčet atributů je konečný a v diagramu se píše do střední části. Za názvem lze uvést typ oddělený dvojtečkou. Například *String* slouží pro texty. Význam atributů v našem diagramu:

1. *modelName* – název modelu auta typu *String* (například: *Auris*),
2. *enginePower* – výkon motoru v kilowattech jako celé číslo,
3. *manufacturer* – název výrobce vozidla typu *String* (například *Toyota*),
4. *serialNumber* – výrobní číslo karoserie vozu,
5. *trunkVolume* – objem kufru v litrech,
6. *owner* – poslední majitel vozu.



Schopnost objektu obsahovat jiné objekty se nazývá *kompozice* (anglicky *composition*) a je klíčovou vlastností objektového programování.

4.11.4 Metody

Metody jsou pojmenované části *třídy*, které obsahují *algoritmy* vyjádřené v programovacím jazyce. Použití *metody* se nazývá **volání metody**. Zapisuje se

jménem, za kterým následují závorky s případnými parametry. Výčet *metod* se uvádí ve spodní části grafického znázornění třídy. Každý název *metody* reprezentuje určitou službu, přičemž tato metoda může dostávat vstupní **parametry** a může **vracet** právě jednu hodnotu. Parametry se uvádí v závorkách, návratový typ za dvojtečkou. Pokud metoda nemá parametry, zůstávají závorky prázdné. Pokud nevrací nic, používá se typ `void` (prázdný).

Třída může mít více metod stejného jména, pokud se liší jejich parametry; mluvíme o *přetížených metodách*. Uvnitř metody lze vytvářet *lokální proměnné*, které existují jen během jejího běhu. Metoda pracuje se svými parametry, atributy objektu a lokálními proměnnými. Může také volat metody jiných objektů nebo vytvářet nové. Pokud diagram obsahuje jen název třídy, oddělovače se nekreslí. Použití *metody* pak může připomínat zápis matematické funkce, například: $y = \sin(x)$.



Při návrhu *tříd* se doporučuje oddělovat *třídy* s datovým obsahem od tříd, kde dominují algoritmy. První skupině se pak říká *datové třídy* a druhá skupina se nazývá *servisní třídy*.

4.11.5 Konstruktor

Kromě běžných metod má třída i tzv. *konstruktor*, který se do diagramu obvykle nepíše. *Konstruktor* je speciální metoda, kterou se vytváří nová instance třídy. V kódu se podobá metodě, ale nemá návratový typ a jeho název se vždy shoduje s názvem třídy. Konstruktor může mít parametry, kterými lze nastavit počáteční hodnoty atributů. Třída může mít více *přetížených* konstruktorů. Pokud ve třídě není žádný *konstruktor*, Java automaticky doplní jeden konstruktor bez parametrů. Nový objekt se vytváří operátorem `new`.

Zdrojový kód 3. Ukázka použití konstruktoru

```
Car automobile = new Car(); ❶
```

- ❶ Pomocí bezparametrického *konstruktoru* vytvoříme novou instanci *třídy* `Car` a uložíme ji do proměnné `automobile`. Případné parametry v závorce by byly odděleny čárkou.

4.11.6 Třída ve zdrojovém kódu

Nyní přepíšeme grafický návrh `Car` do kódu Javy. Použijeme minimalistickou formu, aby vynikla **struktura** nové třídy. Ukázku omezíme na první **dva** atributy Jednotlivá patra diagramu v kódu symbolicky naznačíme řádkovým komentářem (`//`) s pomlčkami.

Úplný kód *třídy* najdete v souboru `Car.java` ve složce přiloženého projektu s názvem `Samples\src\test\java` (na Linuxu jsou lomítka opačná: `Samples/src/test/java`).

Zdrojový kód 4. Minimalistický zápis třídy *Car*

```
class Car { ❶
    // -----
    String modelName; ❷
    int enginePower; ❸
    // -----
    String getModelName() { ❹
        return modelName; ❺
    }
    void setModelName(String name) { ❻
        modelName = name; ❼
    }
    int getEnginePower() { ❽
        return enginePower;
    }
    void setEnginePower(int power) {
        enginePower = power;
    }
}
```

- ❶ Název *třídy* odpovídá hornímu patru diagramu. Složená závorka ohraničuje obsah.
- ❷ Seznam *atributů* odpovídá prostřednímu patru diagramu. Atribut `modelName` může obsahovat pouze objekty typu `String`. Deklarace končí středníkem.
- ❸ Atribut `enginePower` pro výkon motoru obsahuje celá čísla. Na dalších řádcích by seznam atributů pokračoval.
- ❹ Spodní blok kódu obsahuje metody, např. pro čtení hodnoty. Zápis metody v kódu začíná návratovým typem a názvem. Názvy metod začínají malým písmenem, pro čtení se používá předpona `get`.
- ❺ Klíčové slovo `return` vrací hodnotu a příkaz končí středníkem. Tato metoda pouze poskytuje obsah atributu. Typ atributu se shoduje s návratovým typem metody.
- ❻ Metoda pro zápis používá `void`. Výraz deklaruje, že metoda nevrací **nic**. V závorkách je jeden parametr `name` typu `String`.
- ❼ Forma zápisu hodnoty do *atributu* se neliší od zápisu do proměnné.
- ❽ Podle stejného vzoru bychom připravili *metody* pro čtení a zápis zbývajících *atributů*.

5 Od třídy k reálnému kódu

Objekty nejsou jen datové struktury; jsou to živé entity, které mají své chování a zodpovědnost.

— Grady Booch, softwarový inženýr a spoluautor UML

5.1 Užití objektu

V kapitole si ukážeme tvorbu objektů, použití primitivních typů a práci s textem. Začneme texty.

5.1.1 Texty v jazyce Java

V Javě často pracujeme s objekty, které představují nějakou řadu znaků (hodnot typu `char`). Mohou to být například slova nebo celé věty. Texty bývají nejčastěji instancemi třídy `String`. Objekt tohoto typu lze vytvořit konstruktorem nebo **textovým literálem**, který se zapisuje mezi dvojité uvozovky (například `"text"`).

Zdrojový kód 5. Jednoduché použití objektu `String`

```
String name = "Joe"; ①  
name = name + " Black"; ②
```

- ① Text v **dvojitých uvozovkách** je objekt typu `String`, který přiřadíme do proměnné.
- ② Dva texty spojíme operátorem `+`. Výsledkem je **nový** objekt, který nahradí ten původní. Operátor `+` lze u objektů použít pouze pro spojování textu s dalším objektem. Výsledkem takové operace je vždy nový objekt typu `String`.

Operátor `+` lze v Javě použít ke spojování objektu typu `String` s dalším objektem nebo i primitivním datovým typem. Všechna data se pak nejdříve převedou na text a výsledkem je nový objekt typu `String`. Ostatní objektové typy však operátor `+` nepodporují. Práce s textem se velmi podobá práci s primitivními typy. Tohle je nezbytné minimum potřebné pro další výklad. K textům se ještě vrátíme v kapitole [Třída `String`](#).

5.1.2 Vytvoření instance a užití třídy

Při volání konstruktoru Java vyhledá místo v paměti RAM pro novou instanci. Odkaz na toto místo se nazývá *reference*. Referenci si představte jako poštovní adresu domu. Pokud někomu zkopírujeme adresu domu, nevznikne tím druhá

stejná stavba. Analogicky, pokud zkopírujeme *referenci* objektu do jiné proměnné, nevznikne tím nový objekt. Následující ukázka vytvoří a vrátí objektový model automobilu.

Zdrojový kód 6. Práce s objektem

```
public Car createCar() { ❶  
    Car car = new Car(); ❷  
    car.setModelName("Auris"); ❸  
    car.setEnginePower(97); ❹  
  
    String model = car.getModelName(); ❺  
    Car theSameCar = car; ❻  
    boolean identical = car.getEnginePower() == theSameCar.getEnginePower(); ❼  
  
    return car; ❽  
}
```

- ❶ Deklarace *metody* začíná klíčovým slovem **public**, které určuje rozsah viditelnosti této *metody*. Podrobnosti zmíníme v kapitole o **viditelnosti metod**. Následuje návratový typ **Car** a název *metody* s kulatými závorkami **createCar()**. Závorky zde neobsahují žádný argument. Složené závorky ohraničují odsazený kód metody.
- ❷ Operátor **new** vytvoří instanci a výsledek uloží do proměnné **car**. Název proměnné bývá někdy odvozen od názvu třídy. Konstruktor tady nemá žádný parametr.
- ❸ Hodnoty do objektu nastavujeme voláním jeho metod. Používá se tečková notace: **promenna.metoda()**.
- ❹ Zápis výkonu motoru.
- ❺ Metoda bez parametrů má prázdné závorky. Návratový typ *metody* se shoduje s typem proměnné. Proměnná nemá další využití, slouží jen pro ukázkou čtení *atributů*.
- ❻ Kopírujeme referenci; obě proměnné nyní ukazují na stejný objekt automobilu.
- ❼ Proměnná bude **true**, protože obě metody vrací stejné číslo.
- ❽ Poslední řádek těla *metody* vrací hodnotu proměnné **car**, která se zapisuje za klíčovým slovem **return**.

Důležité postřehy:

1. Operátor **==** u objektů porovnává *reference*, nikoliv obsah.
2. Pro porovnání obsahu dvou **objektů** slouží metoda **equals()**. Operátor **==** bude **true** pouze tehdy, jde-li o úplně stejnou instanci. Říká se tomu, že objekty jsou **identické**.

3. Tvorbu objektů lze svěřit i tzv. *statickým metodám*. V čem jsou jiné?

Statická metoda je metoda samotné **třídy**, která nemá přístup k běžným atributům konkrétního objektu. Může však používat statické *metody*, nebo i *statické atributy*, což je pojem, který si vysvětlíme za chvíli. Ve zdrojovém kódu se metoda deklaruje modifikátorem **static**. Je užitečná tam, kde nepotřebujeme konkrétní instanci.

Zdrojový kód 7. Příklady volání statické metody

```
int i = Integer.parseInt("465"); ❶  
int v = Math.max(i, 100); ❷  
LocalDate date = LocalDate.of(2010, 10, 22); ❸
```

- ❶ Metoda `parseInt()` je statická metoda *třídy* `Integer`, která převede argument typu `String` na primitivní číslo. Název *metody* předchází název třídy.
- ❷ Třída `Math` nabízí statické metody pro matematické operace. Tady získáme větší číslo. Metody `Math` nabízí i pokročilejší matematické operace včetně mocnin.
- ❸ Třída `LocalDate` ze standardní *Java* knihovny je určena pro práci s datem (míněno kalendářní datum) bez časové zóny. Parametry *metody* obsahují postupně: rok, měsíc a den. *Metoda* určená pro tvorbu nového objektu se někdy nazývá *tovární metoda*.

Podobně jako statické *metody třídy* existují i statické *atributy třídy* a také se označují modifikátorem **static**. Názvy statických *atributů* se podle konvence zapisují velkými písmeny a víceslovné názvy se oddělují znakem podtržítka (`_`). Takové *atributy* mají využití často jako konstanty *primitivních typů* nebo *neměnných objektů* (anglicky *immutable objects*). Typickým reprezentantem neměnných objektů je *třída* `String`, která neposkytuje žádnou metodu pro změnu svého obsahu. Neměnné objekty lze s výhodou **bezpečně** sdílet při zpracování v dalších *metodách* – včetně zpracování paralelního (souběžného). Podrobnější popis zdrojového kódu *třídy* najdeme v kapitole [První servlet](#).

5.1.3 Autoboxing

Každý primitivní typ má svůj objektový ekvivalent (viz tabulka). *Java* umožňuje automatický převod mezi nimi; tomu říkáme *autoboxing*. Objektové typy začínají velkým písmenem. I když zde nepíšeme **new**, kompilátor jej na pozadí doplní. Časté používání *autoboxingu* může mírně zpomalit běh programu. Pro zrychlení může *Java* některé objekty používat opakovaně. Praktické příklady:

Zdrojový kód 8. Proměna primitivních typů na objekt a zpět

```
Integer objCount = 5; ❶  
Character objChar = 'A'; ❷
```

```
Boolean objAgreement = true; ③  
int count = objCount; ④
```

- ① Automatický převod z `int` na `Integer`.
- ② Vytvoření objektu pro znak z tabulky Unicode.
- ③ Objekt pro logickou hodnotu.
- ④ Převod z objektu zpět na primitivní typ.

5.1.4 Neplatné objekty null

Vložení nového obsahu do proměnné ten původní zmizí. Lze však proměnnou zcela **vyprázdnit**? To závisí na jejím typu:

1. Primitivní typ nelze vyprázdnit, musí vždy obsahovat nějakou platnou hodnotu. Typy s plovoucí čárkou mají speciální konstantu `NaN` (není číslo, z anglického *not a number*).
2. Objektové typy lze vyprázdnit klíčovým slovem `null`.

Zdrojový kód 9. Příklady zápisu výrazu `null`

```
Integer objCount = null; ①  
Character objChar = null;  
Boolean objAgreement = null;  
String text = null;  
int count = objCount; ②  
double value = Double.NaN; ③
```

- ① Proměnná nyní neodkazuje na žádný objekt. Hodnotu `null` lze nastavit jakémukoliv objektovému typu.
- ② Pozor, prázdný objekt nemá žádné metody a nelze jej převést na číslo! Taková operace způsobí chybu za běhu programu, která může vést k pádu celé aplikace.
- ③ Speciální konstanta pro desetinná čísla. K neplatné hodnotě může vést i dělení nulou.



Jedním z důležitých rozdílů mezi **primitivními** datovými typy a jejich **objektovým** ekvivalentem je schopnost vyjádřit nedefinovanou hodnotu pomocí výrazu `null`, který je pro všechny objektové typy společný.

5.1.5 Vztahy mezi třídami

Představme si čtyři instance třídy **Car** naplněné daty. Pokud bychom hodnoty atributů každého objektu vypsalí do jednoho řádku tabulky, získáme následující přehled:

Tabulka 3. Přehled fiktivních vozidel

Výrobce	Model vozu	Vyrobeno	Výrobní číslo vozu	Pohon	Velikost kufru v litrech	Vlastník
ŠKODA AUTO	Fabia	2005-06-01	123-654-987	benzin	260	Jiří Mexedolid
Škoda Auto	Rapid	2013-01-15	359-985-785	benzin	550	Lucie Zixerfxodf
Toyota	Auris	2010-10-22	987-654-123	Benzin	354	Donald Choresisdi
Ford	Focus	2015-03-30	958-232-497	benzin	375	Donald Choresisdi

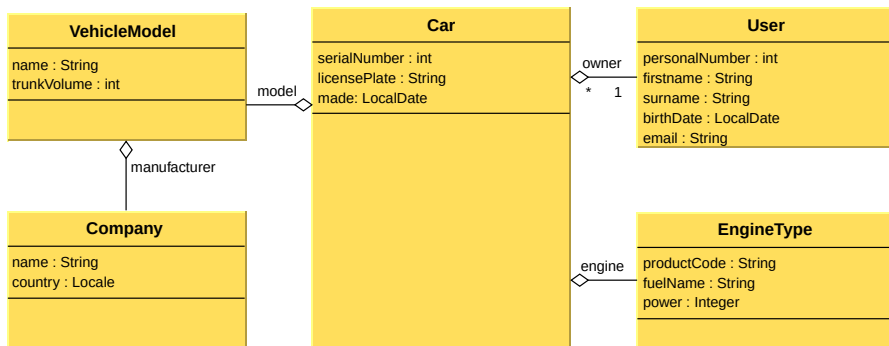
Zde vidíme rizika pro budoucí vyhledávání:

1. Názvy výrobců či modelů mohou být zapsány s překlepy (např. Škoda vs ŠKODA).
2. Dva různí majitelé mohou mít stejná jména.

I když za chyby může člověk, systém by jim měl předcházet. Proto rozšíříme náš model o další třídy.

1. Pro vlastníka navrhne třídu **User**, která dostane jedinečné ID (**personalNumber**). V diagramu nakreslíme vazbu (čáru) s kosočtvercem u **Car**, což znamená, že auto **má** vlastníka. U vazby se uvádí i počet (multiplicita). Hvězdička znamená libovolný počet. Jedno auto má právě jednoho vlastníka, ale uživatel může mít aut více. Relace se v kódu zapíše jako běžný atribut. V diagramu ji však jako textový atribut podruhé nepíšeme. Vazba může mít svůj název. Pokud chybí, používá se název odkazované třídy s malým písmenem.
2. Podobně vytvoříme třídu **VehicleModel**. Atribut výrobce přesuneme sem, protože patří k modelu, nikoliv ke konkrétnímu kusu auta.
3. Nakonec přidáme třídu **Company** (výrobce).

Po zakreslení všech připomínek do původního diagramu může výsledek vypadat takto:



Obrázek 7. Diagram tříd

Podle diagramu připravíme nové *třídy* a upravíme také tu původní metodu pro vytvoření objektu jednoho auta. Podívejme se na výsledek...

Zdrojový kód 10. Komplexní sestavení objektu

```

public Car createCar() {

    Car car = new Car();
    car.setMade(LocalDate.of(2010, 10, 22)); ❶

    VehicleModel model = new VehicleModel();
    model.setManufacturer(new Company("Toyota", Locale.JAPAN)); ❷
    model.setName("Yaris");
    model.setTrunkVolume(436);
    car.setModel(model); ❸

    User user = new User();
    user.setPersonalNumber(10);
    user.setFirstname("Donald");
    user.setSurname("Choresisdi");
    user.setBirthDate(LocalDate.of(1990, Month.OCTOBER, 30)); ❹
    car.setOwner(user); ❺

    return car;
}
  
```

- ❶ Vytvoření objektu pro datum.
- ❷ Konstruktor se dvěma parametry. Atribut **JAPAN** je statický. Objekt lze vytvořit a rovnou předat jako parametr bez ukládání do proměnné.
- ❸ Propojíme model s autem.
- ❹ Datum lze vytvořit i pomocí konstanty reprezentující měsíc.
- ❺ Propojíme uživatele s autem jako vlastníka.

```

public void printCar() {
    Car car = this.createCar(); ❶

    LocalDate made = car.getMade(); ❷
    String manufacturer = car.getModel().getManufacturer().getName(); ❸
    String modelName = car.getModel().getName(); ❹
    Integer trunkVolume = car.getTrunkVolume();
    String firstname = car.getOwner().getFirstname();
    String surname = car.getOwner().getSurname();
    String owner = firstname + " " + surname; ❺

    MyPrinter.print(made, manufacturer,
        modelName, trunkVolume, firstname, surname, owner); ❻
    return; ❼
}

```

- ❶ Zavoláme metodu pro vytvoření auta. `this` znamená, že voláme metodu na stejném objektu, kde se zrovna nacházíme. Slovo `this` lze v při volání metod zpravidla vynechat.
- ❷ Načtení data výroby do proměnné `made`.
- ❸ Ukázka řetězení metod: postupně se „prokoušeme“ až k názvu výrobce. `car.getModel()` nám dá model, na něm zavoláme `getManufacturer()` a nakonec `getName()`. Chybný název metody odhalí kompilátor nebo chytrý editor.
- ❹ Další ukázka řetězení.
- ❺ Spojení tří textů do jednoho (celé jméno).
- ❻ Předání dat k tisku.
- ❼ Příkaz `return` ukončí metodu. Na konci metody s návratovým typem `void` se ten příkaz psát nemusí.

Všimněte si, že v diagramu přibyla nová třída popisující typ energie automobilu (například benzin, nafta, elektřina z baterie). Může nás také zajímat, jak se zachová volání *metody* `getName()` v případě, že metoda `getManufacturer()` **neposkytuje** žádný přiřazený objekt. Volání *metod* na neexistujícím objektu vyvolá chybu při běhu programu, která předčasně ukončí provádění *metody*. Jak už jsme zmínili dříve, hodnota prázdných objektů se v kódu popisuje klíčovým slovem `null`, a takové prázdné objekty jsou výchozí (anglicky *default*) hodnotou každého *objektového atributu* či *pole*. Přesněji bychom mohli říci, že tento případ vede k vyhození *výjimky*. Více informací o *výjimkách* (anglicky *exceptions*) se dozvíme v kapitole [Výjimky](#). Hodnotu `null` lze také zapisovat do proměnných, posílat do *metody*, či použít při porovnání instancí. Příkládám několik příkladů použití.

```
public void writeNulls() {  
    Car car = createCar();  
    car.getOwner().setEmail(null); ❶  
    car.setOwner(null);  
    car = null; ❷  
  
    boolean undefinedCar = (car == null); ❸  
}
```

- ❶ Nastavení atributu na prázdnou hodnotu.
- ❷ „Vymazání“ lokální proměnné.
- ❸ Kontrola, zda je proměnná prázdná; zde bude výsledek `true`.

Připomeňme, že hodnotu `null` mohou obsahovat všechny *objektové typy* a *pole*, nikoliv však *primitivní datové typy*, které mají výchozí hodnotu `0` (nula, případně její ekvivalent). Ukázky kódu, ve kterých se předchází chybám tohoto typu včetně řešení následků, zmíníme později v kapitole o výjimkách.

5.2 Další poznámky ke třídám

Teď už víme, že *třída* objektů má své jméno, *atributy*, *metody* a *konstruktor*. Zapamatujme si, že sloučení *atributů* a *metod* do jedné třídy se nazývá *zapouzdření* (anglicky *encapsulation*) a patří mezi důležité vlastnosti objektového programování, protože umožňuje návrh bezpečného, rozšiřitelného a znovupoužitelného zdrojového kódu.

5.2.1 Viditelnost metod, atributů a tříd

Někdy *metody třídy* mají význam pouze pro aktuální *třidu* a hodilo by se zabránit jejich veřejnému využití – na rozdíl od *metod* jiných, které by měly být dostupné všem. Z toho důvodu je možné přiřadit každé *metodě* takzvaný *modifikátor*, který deklaruje míru její viditelnosti ze strany **objektů jiných tříd**, přitom stejnými prostředky se deklaruje také viditelnost *konstruktorů*, *atributů* ale i celých *tříd*. Jak se viditelnost označuje v diagramu tříd? Znaménko minus (-) uvedené před *atributem* či *metodou* označuje lokální viditelnost, která omezuje použití pouze na metody své *třídy* a ve zdrojovém kódu se označuje klíčovým slovem `private`. Znaménko plus (+) označuje naopak veřejnou dostupnost, která se v kódu vyznačuje klíčovým slovem `public`.

Abychom omezili zápis špatných dat, atributy děláme soukromé a přistupujeme k nim přes veřejné metody zvané *settery* a *gettery*. Názvy označují metody zápisu a čtení. V metodách zápisu můžeme kontrolovat platnost dat před jejich uložením.

Výhodou je, že vnitřek třídy můžeme měnit, aniž bychom ovlivnili ostatní části programu. Soukromé atributy s veřejným přístupem se nazývají *vlastnosti* (properties). V diagramech této knihy lze považovat všechny **neoznačené atributy** za *private* a všechny **neoznačené metody** za *public*. Podívejme se na úplný přehled viditelností (včetně dvou nových), které jsou seřazeny od nejvíce omezujících směrem k volně přístupným. Každá další viditelnost v tabulce zahrnuje též všechny předchozí viditelnosti.

Tabulka 4. Přehled modifikátorů viditelnosti jazyka Java

Znak UML	Viditelnost	Klíčové slovo	Popis
-	soukromá	private	Soukromý přístup v rámci třídy. Mezi různými instancemi stejné třídy není omezení.
~	balíček		Přístup pouze ze tříd ve stejném balíčku (anglicky <i>package</i>), v diagramu se označují znakem vlnovky (~).
#	chráněná	protected	Přístup pro potomky třídy (souvisí s dědičností). K tomu i třídy ze stejného balíčku.
+	veřejná	public	Přístup bez omezení.

Třetí sloupec obsahuje název modifikátoru jazyka Java. Pokud jsou data v pořádku, říkáme, že objekt je *datově konzistentní*. Správné nastavení viditelnosti je klíčem k této konzistenci.

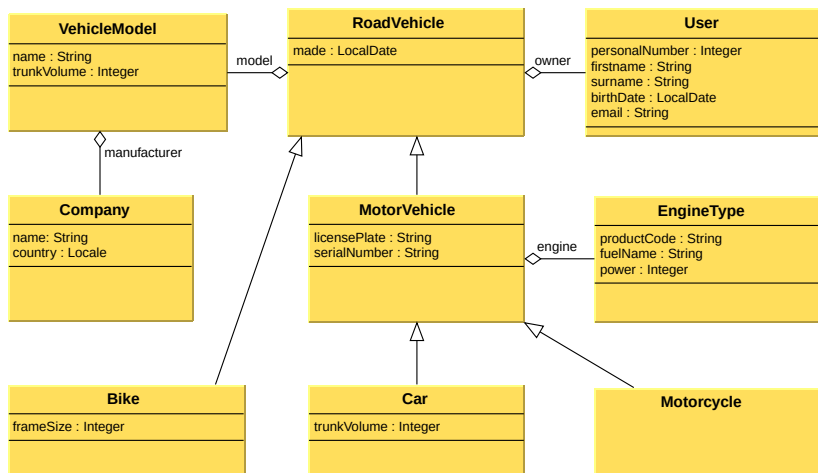
5.2.2 Dědičnost

Vraťme se zpět k diagramu *tříd* evidence vozidel, popsanému v kapitole [Vztahy mezi třídami](#). Mohlo by se zdát, že náš diagram je téměř hotový, ale do toho přijde zákazník, že by rád evidoval také motocykly a šlapací kola. Pro nás je to dobrá příležitost vysvětlit si *dědičnosti tříd* (anglicky *inheritance*).

Pro lepší představu o dědičnosti *tříd* je dobré začít hledáním odpovědi na otázky:

1. Co mají auto a motorka společného? Z pohledu naší evidence.
2. Čím se liší?
3. Co má s nimi společného bicykl?
4. Čím se bicykl liší od motorky?

Společné vlastnosti přesuneme do nadřazené třídy (předka). Společné pro auto a motorku: SPZ, výrobní číslo, výkon, typ paliva. Tuto třídu nazveme **MotorVehicle** (motorové vozidlo). Společné pro bicykl i motorová vozidla: datum výroby, model, vlastník. Vytvoříme třídu **RoadVehicle** (silniční vozidlo). V **Car** zbude jen objem kufru. Přidáme třídu **Bike** pro velikost rámu a **Motorcycle**. Nový diagram:



Obrázek 8. Diagram tříd s dědičností

Pokud některá třída dědí vlastnosti (*metody, atributy*) některého rodiče, pak tohoto rodiče označí uzavřenou šipkou, kterou můžeme interpretovat slovem **je**. Všechna následující tvrzení jsou pravdivá:

1. Auto **je** motorové vozidlo.
2. Motorové vozidlo **je** silniční vozidlo.
3. Auto **je** silniční vozidlo.
4. Bicykl **je** silniční vozidlo.
5. Bicykl však **není** motorové vozidlo (nebo ho nechceme evidovat jako motorové vozidlo).

Nadále však platí všechny původní vztahy (asociace):

1. Auto **má** motor.
2. Motorové vozidlo **má** motor.
3. Motorové vozidlo **má** vlastníka.
4. Bicykl **má** vlastníka.
5. Bicykl však **nemá** motor (nebo ho nechceme evidovat jako motorové vozidlo).

Potomek *třídy* má tedy k dispozici všechny *metody* rodičů, pokud jsou typu *public* nebo *protected*. Potomek však může mít svoji **vlastní implementaci** stejné *metody*, pokud má metoda **stejně jméno**, *vstupní parametry* i *návratový typ*. V reálném životě si můžeme takové chování přiblížit na zjednodušeném ovládání automobilu:

pro zrychlení používáme pedál plynu, pro zpomalení brzdu a pro změnu směru volant, řidiče však nemusí zajímat detailní funkce spalovacího motoru, brzdového systému či převodu pohybu volantu na natočení kol. Právě díky takové vlastnosti může řidič zpravidla používat automobily různých značek a výrobců bez větších problémů. Této vlastnosti se říká *polymorfismus* (anglicky *polymorphism*) a jedná se o významnou vlastnost objektově orientovaného programování. Technika překrývání *metod* se označuje anglickým slovem *overriding*.

Ukázku překrývání *metody* najdeme v kapitole [Piškvorky s defenzivní strategií hry](#). Teď nás může zajímat, jak se změní **použití** upravené *třídy* `Car` po zavedení dědičnosti. Dobrá zpráva je, že z pohledu obsluhujícího kódu je použití *třídy* `Car` identické bez ohledu na to, kolik má rodičů a ve kterém předkovi se implementace metody právě nachází.

Pro připomenutí zápisu a čtení se tedy můžeme vrátit k ukázkám zdrojového kódu v kapitole [Vztahy mezi třídami](#), protože tento kód zůstane **stejný**. Při zavedení dědičnosti se nám však otevírají nové vlastnosti, seznam těch nejdůležitějších je tady:

1. Každá třída (jazyka *Java*) má právě **jednoho rodiče**, výjimkou je prarodič všech *tříd* s názvem `Object` (pozor, nezaměňovat s obecným názvem instance třídy, pro začátečníka může být podobnost slov zavádějící).
2. Název rodiče se v kódu zapisuje za název *třídy* pomocí klíčového slova `extends`, a pokud třída nemá žádného výslovně označeného rodiče, *Java* kompilátor tam dosadí automaticky třídu `Object`.
3. Třída `Object` má několik metod, mezi nejpoužívanější patří metoda `toString()`, která poskytuje textovou informaci o vnitřním stavu objektu. Každý potomek *třídy* může obsahovat jinou implementaci *metody* a u tříd **našeho** projektu implementace záleží jen na nás.
4. Polymorfismus se týká pouze *metod* označených viditelností `public` a `protected`.
5. Konstruktor se nedědí.
6. Potomek může používat *atributy* a *metody* rodiče s některou z viditelností: `protected`, `package` a `private`.
7. Pokud chceme zakázat dědičnost konkrétní třídy, označíme ji klíčovým slovem `final`, pokud chceme zakázat překrytí naší konkrétní *metody*, lze ji také označit klíčovým slovem `final`. Typickým příkladem třídy, které není dovoleno vytvářet žádné potomky, je `String` pro reprezentaci textů.

```
public class Car extends MotorVehicle { ❶
    private int trunkVolume; ❷
    public void setTrunkVolume(int trunkVolume) {
        this.trunkVolume = trunkVolume; ❸
    }
    public int getTrunkVolume() {
        return this.trunkVolume; ❹
    }
}
```

❶ Třída `Car` rozšiřuje rodiče `MotorVehicle`. Rodič se uvádí za názvem aktuální *třídy* — pomocí klíčového slova `extends`. Každá Java třída má právě jednoho předka. Pokud předek není výslovně uveden, kompilátor mu přiřadí automaticky třídu `Object`, což je společný prarodič všech *tříd*.

❷ Soukromý atribut.

❸ Pokud se jména parametru a atributu shodují, použijeme zápis `this.atribut`.

❹ Zde `this` není nutné, protože nehrozí záměna názvů. Metody rodiče v kódu potomka nepíšeme.

5.2.3 Výjimky

Někdy se program dostane do situace, kdy neví, jak dál (např. chyba v datech). Tehdy je nutné zpracování ukončit pomocí **výjimky**. K tomu slouží operátor `throw` následovaný objektem typu `Throwable`. Většinou používáme konkrétnější typy, které popisují daný problém. Ukázka:

```
public void setTrunkVolume(int trunkVolume) throws IllegalArgumentException { ❶
    if (trunkVolume < 0) { ❷
        throw new IllegalArgumentException("Invalid trunkVolume: " + trunkVolume); ❸
    }
    this.trunkVolume = trunkVolume; ❹
}
```

❶ Výraz `throws` v hlavičce metody uvádí seznam možných výjimek.

❷ Kontrolujeme, zda je objem kufru kladné číslo. Pokud je hodnota záporná, vyhodíme výjimku.

❸ Příkazem `throw` vytvoří chybový objekt a ukončí metodu. Chyba se šíří nahoru, dokud ji někdo nezachytí nebo program nespadne.

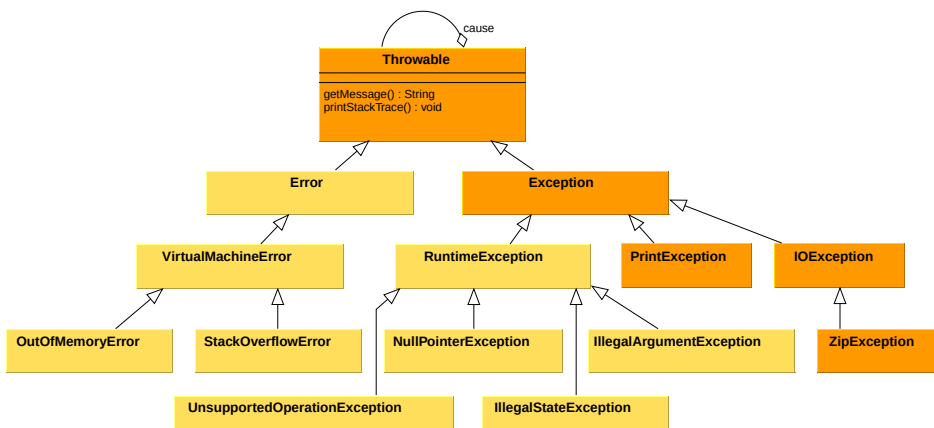
- ④ K tomuto zápisu dojde, jen když je hodnota v pořádku. Jak výjimku zachytit:

Zdrojový kód 15. Tvorba objektu

```
public Car createCar(int trunkVolume) {  
    Car car = new Car();  
    try { ①  
        car.setTrunkVolume(trunkVolume); ②  
        // ... assign other attributes  
    } catch (IllegalArgumentException e) { ③  
        e.printStackTrace(); ④  
    }  
    return car; ⑤  
}
```

- ① Začátek bloku `try`, ve kterém očekáváme problém.
- ② Zde hrozí vyhození výjimky.
- ③ Blok `catch` zachytí chybu daného typu.
- ④ Zde chybu zpracujeme, např. ji vypíšeme. `printStackTrace()` vypíše cestu, kde přesně se chyba stala (tzv. *stack trace*).
- ⑤ Program po zachycení chyby pokračuje dál.

Výjimku `NullPointerException` vyhazuje Java automaticky, když se pokusíte volat metodu na `null`. Všechny výjimky jsou objekty s předkem `Throwable`. Můžeme si vytvářet i vlastní typy chyb. Hierarchie výjimek:



Obrázek 9. Diagram výjimek (zdroj Javadoc)

Základní typy výjimek:

1. `Throwable` – rodič všech výjimek; umí popsat chybu (`getMessage()`) a najít příčinu

(`getCause()`). Umí také vypsat *stack trace*, který obsahuje detailní technický zápis volání jednotlivých metod, umožňujících lokalizovat zdrojový řádek chyby.

2. **Error** – závažné chyby systému (např. došla paměť RAM – `OutOfMemoryError`).

Běžně se nezachycují. Pro představu uveďme dva z jejích potomků:

- `OutOfMemoryError` – *JRE* nemá dostatek paměti RAM (zpravidla při vytváření nových objektů).
- `StackOverflowError` – dochází k rekurzivnímu volání *metod* do takové úrovně, kterou *JRE* vyhodnotí jako chybu.

3. **Exception** – reprezentuje *výjimky*, které má smysl zachytávat (pomocí výrazů `try-catch`), aby nevedly k pádu celé aplikace. Uveďme pár potomků:

- `PrintException` – obecná *výjimka* vznikající při tisku dat.
- `IOException` – chyba vznikající při čtení či zápisu do souborů, třeba na disk.

4. **Kontrolované výjimky** – pokud je riziko, že metoda vyhodí *výjimku*, je třeba ji propagovat za názvem *metody* pomocí klíčového slova `throws`. Existují však dva typy výjimek (`Error` a `RuntimeException`), které se v metodách propagovat **nemusí**, a tohle pravidlo platí také pro všechny jejich potomky. Takovým *výjimkám* se říká *nekontrolované výjimky* (anglicky *unchecked exceptions*) a těm ostatním se říká *kontrolované výjimky* (anglicky *checked exceptions*). Propagace **kontrolovaných** výjimek je v metodě nezbytná, jinak kompilace kódu skončí chybou. Uveďme několik případů **nekontrolovaných** výjimek:

- `IllegalArgumentException` – *výjimka* se používá při kontrole platnosti argumentů *metod*.
- `IllegalStateException` – *výjimka* se používá při kontrole platnosti stavů objektů.
- `UnsupportedOperationException` – *výjimka* se používá v metodách, které (z nějakého důvodu) postrádají korektní implementaci.
- `NullPointerException` – *výjimku* vyhazuje *JRE* při nedovoleném volání *metod* nedefinovaného objektu.

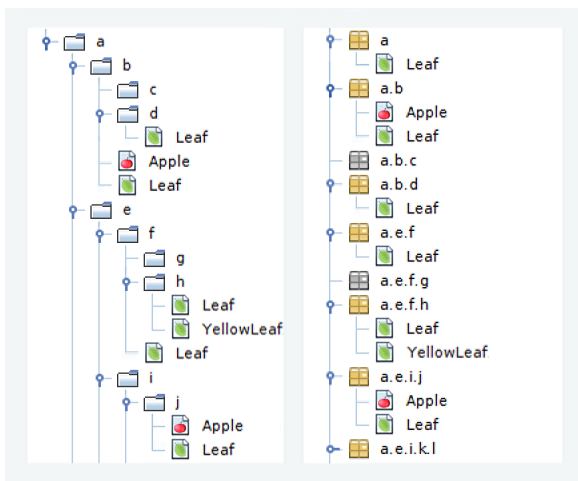
Se znalostí diagramu *tříd* lze zachytávat pouze určité typy výjimek – včetně jejich podtypů. Vyhazování výjimek má zvýšené nároky na zpracování, a tak není doporučováno tímto mechanismem řídit běžné algoritmy.



Za mimořádnou událost běhu programu lze považovat situaci, kterou nelze řešit v rámci kompetence aktuální metody. Příkladem může být neplatná hodnota *parametrů* metody, nebo třeba neočekávaný stav objektu (neočekávaná hodnota jednoho či více *atributů* objektu).

5.2.4 Balíčky

Aby se nepletly třídy se stejným jménem, ukládají se do *balíčků* (anglicky *package*). Balíčky tvoří stromovou strukturu, která odpovídá složkám na disku.



Obrázek 10. Dva náhledy na stejný balíček tříd

Překreslením jabloně z kapitoly [Stromová struktura dat](#) do podoby prezentované editorem *NetBeans* bychom dostali dva nové pohledy na strom, kde kořen stromu má prázdný název. Levý pohled připomíná běžnou adresářovou strukturu z nějakého správce diskových souborů, pro druhý pohled je typický výpis úplné cesty ke každé skupině *tříd*, které tvoří listy stromu. Třídy uložené v jednom balíčku musí mít unikátní název, přitom použití velkých a malých písmen v jeho názvu je významné. Balíčky mohou obsahovat i další soubory, tzv. *zdroje* (resources). Například textové šablony, jazykové lokalizace, nebo jiná data. Názvy *balíčků* se odvozují od internetových domén (pozpátku). Struktura: doména → firma → projekt → podrobnosti. Oddělují se tečkou. Například: `a.e.f.g.YellowLeaf`. První řádek v souboru s kódem musí být deklarace *package*. Seznam použitých tříd se v hlavičce uvádí klíčovým slovem `import`, pořadí je nevýznamné. Ukázka:

Zdrojový kód 16. Ukázka deklarace balíčků a importu tříd

```
package net.ponec.jbook.plainSamples.domains; ❶

import java.time.LocalDate; ❷
import java.lang.String; ❸
import static java.lang.Integer.MAX_VALUE; ❹
```

❶ Balíček třídy musí být na prvním řádku.

- ❷ Importujeme třídu pro datum.
- ❸ Importy z `java.lang` jsou automatické a nemusí se psát.
- ❹ Výraz `import static` umožní používat konstanty bez psaní názvu třídy.

V importech se mohou vyskytovat jen *třídy* (či konstanty) unikátního jména. Při použití dvou (a více) *tříd* stejného jména z různých balíčků je třeba uvádět neimportované *třídy* s názvem balíčku. Praktická ukázka deklarace balíčků a importu je v kapitole [Hello, World!](#).

5.2.5 Komentáře kódu a JavaDoc

Už jsme zmínili, že *řádkový* komentář začíná dvojicí lomítek `//` a končí koncem řádku. V jazyce *Java* se používají ještě *víceřádkové* komentáře uváděné mezi párem znaků `/*` a `*/`, takové komentáře do sebe nelze vnořovat.

JavaDoc jsou speciální víceřádkové komentáře určené pro generování technické dokumentace *Java* projektu ze zdrojového kódu. Forma zápisu se podobá víceřádkovým komentářům, pouze první hvězdička se zdvojí. Popisovat lze samotné třídy, jejich *atributy*, *metody* (včetně parametrů a návratových typů), a dokonce i celé balíčky. Dokumentace metod a tříd ve formátu *JavaDoc* by měla obsahovat popis chování (případně i návod k použití) pro jiného programátora, který je bude používat v jiné třídě. Zde můžeme shledat podobnost s návodem k použití vysavače, kde nového majitele bude zajímat základní obsluha přístroje pravděpodobně více nežli popis rotujícího elektromagnetického pole jeho motoru. Následuje ukázka použití *JavaDoc* nad metodou, která by měla vyhledávat automobil podle výrobního čísla motoru.

Zdrojový kód 17.

```
/** ❶  
 * Find a car by its engine number. ❷  
 * @param serialNumber Serial number of the vehicle ❸  
 * @return The required car or the {@code null} value, ❹  
 *         if no card was found.  
 */  
❺  
public Car findCar(Integer serialNumber) {  
    throw new UnsupportedOperationException("Not implemented yet"); ❻  
}
```

- ❶ Začátek JavaDocu.
- ❷ Popis chování metody. Lze používat i HTML značky. Píše se zpravidla anglicky.
- ❸ Značka `@param` uvádí popis parametru.
- ❹ Značka `@return` uvádí popis výsledku.

- 5 Konec komentáře.
- 6 Pokud implementace ještě chybí, vyhodíme výjimku.

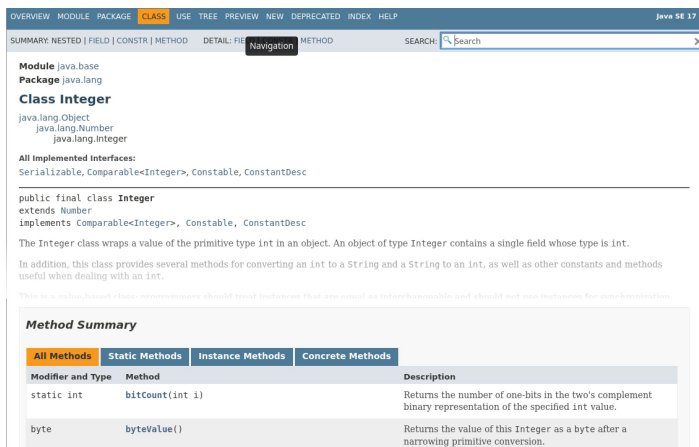
Z takových komentářů lze vygenerovat webové stránky s dokumentací.

Alternativní návrat `null` hodnoty totiž může působit falešným dojmem (za běhu programu), že implementace je hotová, jen auto pro daný parametr nebylo nalezeno.

JavaDoc nad projektem lze sestavit příkazem *Maven*:

```
mvn javadoc:javadoc
```

JavaDoc se generuje do souborového systému ve formátu *HTML*, kterému rozumí internetový prohlížeč.



Obrázek 11. Zkrácený popis třídy *Integer* (zdroj: dokumentace firmy Oracle)

Obrázek je pouze ilustrační, jeho obsah byl zkrácen pro potřeby tisku. V horní části stránky je hlavička s navigační lištou, která zobrazuje verzi *Java* a nabízí také pole pro vyhledávání *tříd* či balíčků. Pod ní je uvedeno označení *modulu*, jedná se o členění tematicky podobných balíčků a zdrojů do skupin pro úsporu paměti a snadnější distribuci aplikace, s tímto pojmem však dále pracovat nebudeme; podrobnější informace lze dohledat na internetu pomocí klíčových slov *Java Platform Module System* (JPMS). Následuje uvedení názvu *balíčku*, *třídy* a znázornění *dědičnosti*. Dále vidíme popis všech rozhraní, která třída implementuje. Na stránce pak najdeme detailní popis třídy, někdy i doporučené použití. Obrázek naznačuje také popis některých *metod* objektu, v reálné dokumentaci najdete informaci mnohem více.

5.2.6 Interface

Pojem *interface* (česky *rozhraní*) popisuje *veřejné metody*, které má objekt nabízet. Interface lze považovat zvláštní druh *abstraktní třídy*, která nemá atributy ani konstruktor. Její metody jsou výhradně veřejné a obvykle nemají tělo (implementaci). Metoda s implementací se označuje klíčovým slovem **default**. Jaký má vlastně přínos *rozhraní* ?



Zatímco každá *Java* třída dědí vždy právě z **jedné** třídy, implementovat může (volitelně) hned **několik** *rozhraní* zároveň. Využití najdeme při simulaci vícenásobné dědičnosti tříd. Interface je příslibem, že třída bude implementovat všechny deklarované metody.

Rozhraní se deklaruje v *Java* kódu podobně jako běžné třídy, výchozí implementace *metod* však bývá prázdná. Místo složených závorek s kódem se uvádí jen středník.

Zdrojový kód 18. Ukázka rozhraní pro počítadlo událostí

```
public interface Counter { ❶  
    /** Add new events to the counter. */ ❷  
    void add(int value); ❸  
  
    /** Get the current number of registered events. */  
    int getCount();  
  
    /** Get the multiple number of registered events. */  
    default int getMultiCount(int i) { return i * getCount(); } ❹  
}
```

- ❶ Používáme slovo **interface**.
- ❷ JavaDoc je zde velmi důležitý pro budoucí programátory.
- ❸ Metoda bez těla. Říká se jí *abstraktní metoda*.
- ❹ Metoda s výchozím (**default**) tělem.

Instance rozhraní nelze vytvořit přímo. Musíme vytvořit třídu, která rozhraní implementuje (**implements**).

Zdrojový kód 19. Typická ukázka implementace rozhraní

```
public class CounterImpl implements Counter { ❶  
    private int count = 0; ❷  
  
    public void add(int i) { ❸  
        count = count + i;  
    }  
}
```

```
    public int getCount() { ❹  
        return count;  
    }  
}
```

- ❶ Třída slibuje splnit interface `Counter`.
- ❷ Inicializace počítadla.
- ❸ Musíme napsat tělo všech slíbených metod.
- ❹ Poslední povinná metoda.

Implementované *metody* musí být **veřejné**, viz [kapitola o viditelnosti](#). Teď máme svůj interface a alespoň jednu třídu, která jej implementuje. Jak to použít?

Zdrojový kód 20. Příklad využití rozhraní

```
private Counter createCounter() { ❶  
    return new CounterImpl();  
}  
  
public int useCounter(int value) {  
    Counter counter = createCounter(); ❷  
    counter.add(value); ❸  
    return counter.getMultiCount(10); ❹  
}
```

- ❶ Metoda vrací typ `Counter`. Vnitřní implementace (`CounterImpl`) může zůstat skrytá.
- ❷ Pracujeme jen s rozhraním, což je bezpečnější pro budoucí změny.
- ❸ Volání metod rozhraní.
- ❹ Použití `default` metody.

5.2.7 Generické datové typy

Generický datový typ je parametr, kterým při použití třídy, rozhraní nebo metody určujeme, s jakým konkrétním datovým typem má daný prvek pracovat. Tento přístup umožňuje opakované využití logiky pro různé *datové typy* při současném zachování plné typové bezpečnosti. Parametry se uvádějí ve špičatých závorkách a případné vícečetné typy se oddělují čárkou. Například zápis rozhraní `Function<String, Integer>` definuje, že metoda tohoto rozhraní přijímá parametr typu `String` a vrací výsledek typu `Integer`. Hlavní výhodou generik je striktní typová kontrola již během kompilace, což eliminuje riziko vzniku typových chyb za běhu programu.

5.2.8 Inline třídy a lambda výrazy

Místo tvoření celého souboru pro třídu ji můžeme zapsat přímo na místě (anglicky *inline*). Používá se to u rozhraní, která mají jen jednu metodu.

Zdrojový kód 21. Použití inline třídy

```
Function<String, Integer> converter = new Function<String, Integer>() { ❶  
    public Integer apply(String t) {  
        return Integer.parseInt(t); ❷  
    }  
}; ❸
```

- ❶ Tvoříme bezejmennou třídu přímo za `new`.
- ❷ Implementace převodu.
- ❸ Celé je to jeden přiřazovací příkaz zakončený středníkem. Zápis inline tříd je zdlouhavý, proto existují stručné **lambda výrazy**. Podmínkou je rozhraní s jedinou abstraktní metodou.

Zdrojový kód 22. Příklad konverze textu na číslo

```
int number = converter.apply("1");
```

Pamatujme, že *inline třídy* mají **neomezený** přístup také k soukromým *metodám* i *atributům* hostitele – viz kapitolu [o viditelnosti metod](#). Protože však zápis *inline tříd* je stále pracný, vývojáři jazyka zapracovali do kompilátoru další zjednodušení, kde ke stejnému výsledku dojdeme stručnějším zápisem. Nezbytnou podmínkou však je, aby *interface* obsahoval popis **právě jedné** abstraktní *metody*.

Zdrojový kód 23. Lambda výraz

```
Function<String, Integer> converter = (String t) -> { return Integer.parseInt(t); };
```

První závorka za rovnítkem označuje argument *metody* `apply()`, výraz za operátorem `->` obsahuje implementaci *metody*. Pokud kompilátor dokáže zjistit datový typ argumentu metody z kontextu kódu (většinou to dokáže), lze deklaraci datového typu v kódu vynechat. Pokud metoda obsahuje **jediný** argument, kulaté závorky kolem jsou nadbytečné. Pokud implementace metody obsahuje **jediný příkaz**, složené závorky lze vynechat, stejně jako slovo *return* a středník za příkazem.

Zdrojový kód 24. Zjednodušený lambda výraz

```
Function<String, Integer> converter = t -> Integer.parseInt(t);
```

Úplně nejkratší je *reference na metodu* pomocí operátoru `::`. Lambda výraz se pak vytvoří pouhým odkazem na existující metodu.

Zdrojový kód 25. reference na metodu

```
Function<String, Integer> converter = Integer::parseInt;
```

Všechny čtyři (výše uvedené) formy zápisu jsou, z pohledu běhu programu, rovnocenné. Lambda výrazy se využívají často ve spojení s rozhraním [Stream](#) (zmíníme později). Další ukázky použití najdeme v kapitole [Bodové kreslení](#) a v příkladech použití [AJAX](#).

5.2.9 Třída typu Record

Record je speciální třída pro neměnná data. Java za nás automaticky vytvoří konstruktor, gettery a metody jako `equals` nebo `toString`.

Zdrojový kód 26. Record typu Car

```
record Car (String modelName, int enginePower) {}
```

Atributy třídy se uvádějí v kulatých závorkách. Tělo třídy může zůstat prázdné. Krátký zápis nahradí desítky řádků běžné třídy.

Zdrojový kód 27. Přepis třídy Car

```
public final class Car {
    private final String modelName; ❶
    private final int enginePower;

    public Car(String modelName, int enginePower) {
        this.modelName = modelName;
        this.enginePower = enginePower;
    }

    public String getModelName() { return modelName; }
    public int getEnginePower() { return enginePower; }

    @Override
    public boolean equals(Object o) { ❷
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Car car = (Car) o;
        return enginePower == car.enginePower
            && Objects.equals(modelName, car.modelName);
    }

    @Override
    public int hashCode() { ❸
        return java.util.Objects.hash(modelName, enginePower);
    }
}
```

```
@Override
public String toString() { ❷
    return "Car{modelName='" + modelName + "', enginePower=" + enginePower + "}";
}
```

- ❶ Atributy jsou neměnné, hodnoty se přiřadí pouze v konstruktoru.
- ❷ Metoda pro porovnávání dvou objektů. Výchozí implementace rodiče považuje za shodné pouze dvě totožné instance.
- ❸ Metoda vypočítá číselný otisk parametrů objektu, který může urychlit porovnávání objektů na shodu.
- ❹ Metoda poskytuje textovou informaci o obsahu objektu. Pokud použijeme operátor `+` pro připojení k nějakému textu (objektu typu `String`), tato metoda se zavolá automaticky. Příklad zápisu na standardní výstup:
`System.out.println("Info: " + new Car("Ford", 100));`

5.2.10 Knihovny

Knihovny jsou sady hotových tříd pro konkrétní účely. Šetří čas a pomáhají udržet jednotný styl. Oblíbené jsou otevřené (open-source) knihovny.

5.3 Datový typ pole

Posledním důležitým typem je *pole* (array). Může obsahovat seznam prvků stejného typu a pevné délky. K prvkům přistupujeme přes číselný **index** v hranatých závorkách `[]`. V Javě se vždy indexuje od **nuly**. Maximální délka pole je omezena typem `int`. Ukázky pole znaků:

Tabulka 5. Výrazy s polem znaků

# Kód	Výsledek Typ	Poznámka
1. <code>char[] array = {'A', 'B', 'C'};</code>	A, B, C <code>char[]</code>	Deklarace datového typu pole začíná uvedením typu prvku následovaného párem hranatých závorek <code>[]</code> . Napravo od rovnítko je složená závorka obsahující seznam hodnot (obecně výrazů) oddělených čárkou. Takto jednoduchý zápis lze využít pouze v <i>příkazu</i> , který zároveň deklaruje novou proměnnou.

# Kód	Výsledek Typ	Poznámka
2. <code>array = new char[] { 'N', 'E', 'X', 'T' };</code>	N, E, X, T char[]	Pokud vytváříme pole do již zavedené proměnné, je třeba před složenou závorku vložit výraz typu <code>new char[]</code> , kde výraz <code>char</code> reprezentuje požadovaný datový typ výsledného pole.
3. <code>array = new char[2];</code>	■, ■ char[]	Výraz vytvoří prázdné pole požadované velikosti. Všechny prvky číselného primitivního typu budou obsahovat hodnotu <code>0</code> (což znázorňují tmavé čtverce), prvky objektu budou obsahovat hodnotu <code>null</code> .
4. <code>array[0] = 'O'; array[1] = 'K';</code>	O, K char[]	Do každého prvku zapíšeme jeden znak.
5. <code>array.length</code>	2 int	Výraz poskytující délku pole.
6. <code>array[0]</code>	O char	Pomocí toho výrazu přečteme hodnotu prvního prvku. Tento výraz lze přiřadit proměnné typu <code>char</code> .
7. <code>array[1]</code>	K char	Čtení posledního prvku pole. Čtení z prvků pole mimo rozsah pole by skončilo výjimkou.
8. <code>array = "HELLO" .toCharArray();</code>	H, E, L, L, O char[]	Znakové pole můžeme získat z objektu typu <code>String</code> voláním metody <code>toCharArray()</code> .
9. <code>Arrays.sort(array);</code>	E, H, L, L, O char[]	Obsah pole je možné seřadit od nejmenších hodnot po největší zavoláním metody <code>Arrays.sort()</code> .
10. <code>new String(array)</code>	EHLLO String	Nový <code>String</code> vytvořený z pole znaků.
11. <code>new String[] { "AB", "CD", "EF" }</code>	AB, CD, EF String[]	Poslední příkaz demonstruje sestavení pole objektů typu <code>String</code> .

Java podporuje i vícerozměrná pole (pole v polích). Další ukázky použití lze najít v příložených příkladech, více informací je uvedeno v kapitole [Programové výrazy](#). Proměnná typu pole může obsahovat nedefinovanou hodnotu `null`.

5.3.1 Pole vytvořené metodou třídy

Metody mohou (za určitých podmínek) přijímat libovolný počet argumentů stejného typu pomocí mechanismu *varargs* (z anglického *variable arguments*). Vlastnost se deklaruje pomocí tří teček (`...`) za typem argumentu. V těle metody se pak s daty pracuje jako s polem. Aby mohl kompilátor jednoznačně určit, které hodnoty patří do pole a které k ostatním parametrům, platí pro **varargs** dvě pravidla:

1. v seznamu parametrů se smí vyskytovat pouze jednou a
2. musí být vždy uveden jako **poslední** parametr v pořadí.

```
String[] toArray(String... texts) { ❶
    return texts; ❷
}
void test() {
    String[] words = toArray("Hello", "Word!"); ❸
}
```

- ❶ Metoda toArray() má parametr typu String proměnné délky.
- ❷ V těle metody už pracujeme s parametrem stejně jako s polem, deklarovaným výrazem String[]. Metoda vrátí vstupní parametr jako běžné textové pole, metoda však může implementovat také libovolné jiné zpracování tohoto parametru.
- ❸ Do argumentu můžeme předat kolik textů chceme, metoda je vrátí jako pole.

5.4 Vybrané třídy standardní knihovny

Standardní knihovna jazyka Java má k dispozici značné množství tříd a{space}rozhraní, které se liší svým zaměřením, a některé z nich si tady představíme.

5.4.1 Třída String

Připomeňme, že objekt typu String reprezentuje neměnnou sekvenci znaků, která může reprezentovat i běžná slova či věty. Nabízí mnoho metod pro vyhledávání, ořezávání či nahrazování textu. Protože objekt je **neměnný**, každá **změna** ve skutečnosti vytvoří nový objekt.

Tabulka 6. Výrazy s objektem typu String

# Kód	Výsledek Typ	Poznámka
1. String name = "Joe";	Joe String	Zápis textového literálu.
2. name = name + " Black";	Joe Black String	Spojení textů.
3. name.charAt(0)	J char	Čtení prvního znaku.
4. name.charAt(1)	o char	Čtení znaku na další pozici. Pokud metoda dostane index mimo rozsah, vyhodí výjimku.
5. name.length()	9 int	Celkový počet znaků Stringu.
6. name.indexOf('B')	4 int	Metoda vrátí pořadí (index) prvního shodného znaku zleva, jinak hodnotu -1, pokud znak shodu nenajde. K dispozici je i přetížená metoda pro vyhledávání objektů typu String.

# Kód	Výsledek Typ	Poznámka
7. <code>name.substring(4, 9)</code>	Black String	Metoda vytvoří nový objekt, který obsahuje sekvenci znaků od čtvrtého (včetně) do znaku, který předchází devátému indexu. K dispozici je i <i>přetížená metoda</i> s pouhým parametrem počátku, konec si implementace získá z délky hodnoty <code>String</code> .
8. <code>name.toUpperCase()</code>	JOE String BLACK	Převod malých znaků na velké.
9. <code>name.toLowerCase()</code>	joe black String	Převod velkých znaků na malé.
10. <code>name.equals("Joe Black")</code>	true boolean	Porovnání obsahu metodou <code>equals()</code> .
11. <code>name == "Joe Black"</code>	false boolean	Porovnání stejné instance objektu.
12. <code>name == name</code>	true boolean	Porovnání stejné instance objektu.
13. <code>name.startsWith("Joe")</code>	true boolean	Ověření začátku textu.
14. <code>name.endsWith("Black")</code>	true boolean	Ověření konce textu.
15. <code>" Joe ".trim().equals("Joe")</code>	true boolean	Metoda odstraní <i>bílé znaky</i> z obou stran textu.
16. <code>"This is ".concat(name)</code>	This is Joe String Black	Spojení dvou textových objektů. Analogie k operátoru <code>+</code> .
17. <code>String.join(" ", "This", "is", name, "!")</code>	This is Joe String Black !	Spojování textů s oddělovačem z prvního parametru.
18. <code>"This is %s!".formatted(name)</code>	This is Joe String Black!	Spojování objektů v šabloně. První značka <code>%s</code> bude nahrazena prvním argumentem. Podporuje i jednoduché formátování.

5.4.2 Rozhraní List

Pole má pevnou délku, což je někdy omezující. Rozhraní `List` reprezentuje dynamický seznam objektů, který roste podle potřeby. Prvky listu nemohou být primitivní typy, lze však využít *autoboxing*. Populární je implementace `ArrayList`.

Tabulka 7. Ukázka použití

# Kód	Výsledek Typ	Poznámka
1. <code>List list = new ArrayList();</code>	ArrayList	Vytvoříme nový objekt typu <code>ArrayList</code> a zapíšeme ho do proměnné typu <code>List</code> . Nový objekt je prázdný.
2. <code>list.add(8); list.add('K');</code>	8, K ArrayList	Do vytvořeného objektu zapíšeme postupně dva prvky různých typů. Pro zápis využijeme <i>autoboxing</i> .
3. <code>list.size()</code>	2 int	Přidáním každého prvku se délka listu zvětší, aktuální délku pole zjistíme metodou <code>size()</code> .

# Kód	Výsledek	Typ	Poznámka
4. <code>list.get(0)</code>	8	Integer	První hodnotu listu získáme metodou <code>get()</code> na pozici <code>0</code> .
5. <code>Object c = list.get(1);</code>	K	Character	Obsah hodnoty na pozici <code>1</code> zapíšeme do proměnné. Protože však v době kompilace nelze určit, jaký datový typ bude vlastně získán, lze hodnotu zapsat přímo pouze do proměnné obecného datového typu <code>Object</code> .

Třída ArrayList

Jak název *třídy* napovídá, vnitřní implementace *třídy* `ArrayList` je postavena na **poli objektů**, přitom interně obsahuje dva důležité *atributy*:

1. `elementData` typu `Object[]` je *pole* s výchozí velikostí **deseti (10)** prvků, pokud v konstruktoru není uveden požadavek na velikost jinou,
2. `size` je celočíselný *atribut* typu `int`, který obsahuje počet **skutečně vložených** prvků, výchozí hodnota je **nula (0)**.

Po vytvoření objektu výchozím konstruktorem *třídy* `ArrayList` si můžeme schematicky popsat chování toho objektu pro několik scénářů:

1. Pokud do prázdného listu vložíme metodou `add()` **první** hodnotu, metoda zapíše hodnotu na pozici **prvního** prvku pole (s indexem `0`) a *atribut* `size` se zvýší na hodnotu `1`.
2. Další hodnota (vložená stejnou metodou) bude zapsána do pole na pozici určené aktuální hodnotou *atributu* `size` a velikost seznamu se pak zvýší o jedničku. Takové vkládání hodnoty lze ještě několikrát opakovat.
3. Při vkládání **jedenácté (11)** hodnoty však máme všechny prvky pole obsazené, co s tím? V takovém případě musí metoda `add()` vytvořit nějaké nové a větší pole, nakopírovat do něj všechny hodnoty pole původního a výsledek zapsat do *atributu* `elementData`, čímž se to původní pole připraví automaticky k uvolnění z paměti. Další postup už bude shodný s předchozím krokem.
4. Pokud budeme chtít metodou `remove()` odstranit hodnotu na pozici `3`, metoda musí přesunout všechny hodnoty zapsané pod vyšším indexem o jednu pozici níže, přitom hodnota *atributu* `size` se o jedničku sníží.

Z uvedeného popisu je zřejmé, že použití *třídy* `ArrayList` je optimální v případech, kdy umíme zhruba odhadnout cílový počet prvků a kdy četnost odebírání (či vkládání) počátečních a středových prvků není vysoká. V opačných případech je vhodné zvážit implementaci jinou, která však může mít zase vyšší nároky na paměť.

List s typem prvku

Použitím datového typu `List` jsme získali výhodu dynamického přizpůsobení velikosti, ale přišli jsme o typovou kontrolu prvků – ve srovnání s *polem*. Standardní rozhraní `List` má naštěstí už implementovanou podporu deklarace typu prvku a využívá k tomu dříve zmíněné **generické typy**. Použití generických typů není povinné, ale je silně doporučeno v případech, kde třída *generiky* podporuje. Podívejme se na ukázky zdrojového kódu.

Tabulka 8. Ukázka použití generik v Listu

# Kód	Výsledek Typ	Poznámka
1. <code>List<Character> list = new ArrayList<>();</code>	ArrayList	Seznam určený pro znaky.
2. <code>list.add('O');</code> <code>list.add('K');</code>	O, K ArrayList	Zápis jiného typu než <code>Character</code> by skončil chybou kompilace.
3. <code>list.get(0)</code>	O Character	Čtení prvků je analogické.
4. <code>Character c = list.get(1);</code>	K Character	Prvky lze zapsat přímo do proměnné toho generického typu.
5. <code>list = Arrays.asList('H', 'E', 'L', 'L', 'O');</code>	H, E, L, L, O ...ArrayList	Vytvoření seznamu znaků statickou metodou.
6. <code>Collections.sort(list);</code>	E, H, L, L, O ...ArrayList	Abecedního řazení prvků.
7. <code>Arrays.asList("AB", "CD", "EF")</code>	AB, CD, EF ...ArrayList	Vytvoření seznamu textů statickou metodou.

5.4.3 Rozhraní Map

Implementace rozhraní *Map* ukládá data jako páry **klíč:hodnota**. Místo číselných indexů používáme klíče (např. jméno nebo ID). Jednou z implementací je třída `TreeMap`, která klíče udržuje seřazené. Klíče musí být vzájemně porovnatelné, jejich typ musí implementovat rozhraní `Comparable`. Taková implementace vyhodnotí, který ze dvou klíčů má vyšší pozici. Rozhraní `Comparable` mají implementované všechny objektové ekvivalenty primitivních typů a třídy `String`, proto můžeme jejich instance použít rovnou jako klíč mapy. Hodnotou může být `null`, ale klíč musí být platný objekt.

Tabulka 9. Ukázka použití mapy

# Kód	Výsledek Typ	Poznámka
1. <code>Map<Integer, String> map = new TreeMap<>();</code> <code>map.put(7, "K");</code> <code>map.put(3, "O");</code>	{3=O, TreeMap 7=K}	Mapa obsahuje (ve špičatých závorkách) dva generické typy, první udává datový typ klíče a druhý typ hodnoty . Postupně přiřadíme dvě hodnoty.

# Kód	Výsledek	Typ	Poznámka
2. <code>map.get(3)</code>		O String	Metoda vrátí hodnotu uloženou s klíčem <code>3</code> , výsledek lze uložit přímo do proměnné toho druhého generického typu.
3. <code>map.get(4)</code>		null ?	Metoda se pokusí najít hodnotu s klíčem <code>4</code> , a pokud ji nenajde, vrátí hodnotu <code>null</code> .
4. <code>map.size()</code>		2 int	Metoda vrací aktuální počet přiřazených prvků.
5. <code>map.keySet().toArray()</code>		O, K Object[]	Tento výraz poskytuje pole všech přiřazených klíčů řazených podle abecedy. Jiná implementace však může vracet klíče řazené jinak.
6. <code>map.remove(3)</code>		O String	Metoda odstraní hodnotu uloženou s klíčem <code>3</code> a tuto odstraněnou hodnotu vrátí.

5.4.4 Rozhraní Stream

I když se může jevit implementace *třídy* `List<>` jako ideální nástroj pro zpracování skupinových dat, můžeme narazit na limity. Hlavním faktorem nemusí být jen omezený počet prvků seznamu, ale i značná **paměťová náročnost** jeho obsahu. Příkladem nevhodného použití může být hromadné zpracování fotografií, kde velikost každého obrázku může přesahovat 5 MB. Pod pojmem „zpracování“ si představme třeba zmenšování obrázku pro tvorbu náhledů. Pokud bychom chtěli umístit všechny obrázky do seznamu typu `List`, pravděpodobně by zpracování skončilo výjimkou na nedostatek paměti v počítači.

Jak můžeme takový typ úlohy řešit lépe? Naznačme alespoň dvě cesty:

1. Do objektu typu `List` můžeme umístit pouze zástupný identifikátor obrázku (třeba cestu k fotografii na lokálním disku).
2. Pro zpracování můžeme využít datový typ `Stream`, který umí odebírat data (z nějakého zdroje) **postupně**.

Objekt typu `Stream` (česky se překládá jako *proud* nebo *datovod*) si lze představit jako potrubí na datové objekty, do kterého můžeme připojovat (postupně) různé služby pro zpracování protékajících dat, mohou to být třeba **filtry** nebo **funkce** umožňující i změnu datového typu. Instalace takových **služeb** se provádí vkládáním servisních objektů (do `Streamu`), které pak požadovanou službu realizují. Zmíněné servisní objekty jsou nuceny implementovat **rozhraní** (*interface*) obvykle s právě jednou abstraktní metodou tak, aby pro jejich implementaci šlo využívat *Lambda výrazy* zmíněné dříve. Typ servisních *metod* určuje **argument** metody *Streamu*, který

službu připojuje. Objekt typu `Stream` data obsahovat nemusí, jeho přínosem je především poskytování *programového rozhraní* (API) pro dávkové zpracování dat, což lze využít i pro úsporu paměti. Do zdrojového kódu přináší *Streamy* stručnější a zpravidla lépe organizovaný zápis – zejména ve spojení s Lambda výrazy. Čtení zdrojového kódu *Streamů* je třeba zažít, zde ukážeme pouze jednoduché využití tohoto rozhraní.

Zdrojový kód 29. Použití rozhraní `Stream`

```
List<Integer> numbers ❶
= Stream.of(9, null, 2, 5, 1) ❷
  .filter((Integer i) -> i != null) ❸
  .map(i -> i * 10) ❹
  .sorted() ❺
  .limit(2) ❻
  .toList() ❼
; ❽
```

- ❶ Cílem je získat seznam čísel.
- ❷ Vytvoříme stream z několika hodnot.
- ❸ Zahodíme hodnoty `null`. Pro stručnější zápis *třídy* filtru využijeme Lambda výraz, kde potřebný algoritmus uvedeme vpravo za šipkou `->`. Argument metody (vlevo od šipky) nemusí (v tomto případě) obsahovat explicitní definici **datového typu**, a tento argument nemusí být ani obalen v kulatých závorkách, jak si ukážeme na dalším řádku.
- ❹ Metoda `map()` slouží k transformaci objektů, včetně změny typu.
- ❺ Metoda `sorted()` provádí řazení hodnot od nejmenší k největší. Pravidla řazení lze dodat druhým argumentem.
- ❻ Použijeme pouze první dva výsledky.
- ❼ Výsledek převedeme na `List`.
- ❽ Celý výraz má formu jediného *Java příkazu* (zakončeného jediným středníkem). Výsledek uložený do proměnné `numbers` bude obsahovat posloupnost dvou čísel: `10` a `20`. Třída `Stream` nabízí také API poskytující **součet** či **aritmetický průměr** výsledků, možností je hodně.

Praktické použití *třídy* `Stream` si lze prohlédnout a vyzkoušet v kapitole [Programové výrazy](#). Pro zájemce o další studium přikládám internetový odkaz [14].

5.4.5 Třída `BigDecimal`

Primitivní číselné datové typy (popsané [úvodem knihy](#)) mají jasně vymezenou velikost, která ovlivňuje jejich minimální a maximální hodnoty. Číselné typy

s (plovoucí) desetinnou čárkou mají navíc omezenou **přesnost**. Pokud mají zmíněná omezení negativní vliv na výpočet, je vhodnější použít objekty typu `BigDecimal`. Objekty této třídy reprezentují kladná a záporná desetinná čísla s libovolnou velikostí a s libovolnou přesností. Limity jsou dány jen velikostí dostupné operační paměti počítače při běhu programu. Protože však zmíněný objektový typ nemá svůj primitivní ekvivalent, pro matematické operace je třeba volat jeho *metody* – použití si ukážeme na příkladech. Třída `BigDecimal` je neměnná, její matematické metody proto vytváří **nové** objekty.

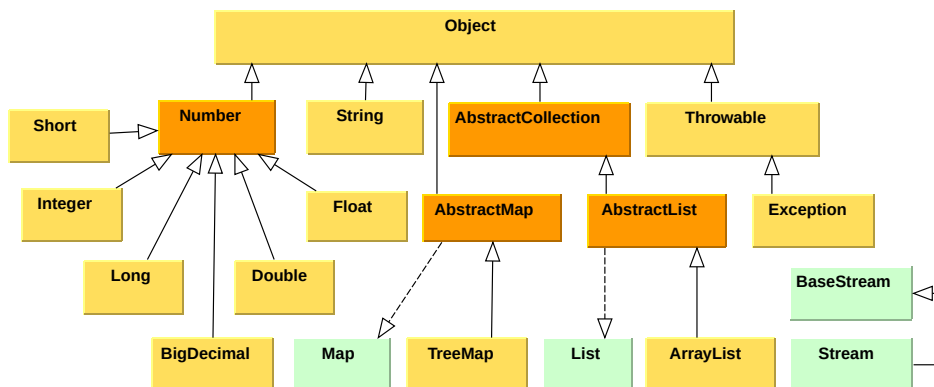
Zdrojový kód 30. Třída `BigDecimal`

```
BigDecimal a = new BigDecimal("1.45"); ❶  
BigDecimal b = new BigDecimal(1.45f, new MathContext(10)); ❷  
BigDecimal c = b.setScale(1, RoundingMode.HALF_UP); ❸  
b = a.add(c); ❹  
b = a.subtract(b); ❺  
b = a.multiply(c); ❻  
b = a.divide(c, new MathContext(10)); ❼  
a.setScale(9).compareTo(a) == 0 ❽  
a.setScale(9).equals(a) ❾
```

- ❶ Objekt se doporučuje vytvořit z textu pro požadovanou přesnost.
- ❷ Objekt lze vytvořit z primitivního typu `float`. Přesnost (anglicky *precision*) se udává v konstruktoru třídy `MathContext` a udává *platný počet číslic* nového čísla.
- ❸ Zaokrouhlení na desetiny s výsledkem `1.5`.
- ❹ Součet dvou čísel s výsledkem `2.95`.
- ❺ Rozdíl dvou čísel, který ukazuje nepřesnost datového typu `float`. Výsledkem je nový objekt typu `BigDecimal` s hodnotou `-0.000000048`.
- ❻ Násobení čísel, výsledkem je hodnota `2.175`.
- ❼ Při dělení čísel určíme přesnost výsledku. Jeho hodnota je `0.9666666667`.
- ❽ Číslo je vhodné porovnávat metodou `compareTo()`. Z pohledu řazení mají obě čísla stejnou váhu. Pokud by měl první argument vyšší pozici, výsledek by byl větší než `0` (a opačně). Pro stejné hodnotě metoda vrátí nulu.
- ❾ Shoda pomocí `equals()` porovnává navíc stejnou **přesnost**.

5.4.6 Shrnutí kapitoly o třídách

Standardní knihovna jazyka *Java* je plná užitečných *tříd* a *rozhraní*, které se řadí do tematických *balíčků*. Všechny třídy mají společného předka *Object*. Každá *třída* může (ale nemusí) implementovat jedno či více *rozhraní*. Každé *rozhraní* může (ale nemusí) mít svého předka. Číselné objektové typy (popsané v této knize) mají společného abstraktního předka *Number*, abstraktní třídy (podobně jako *rozhraní*) však nemohou vytvářet objekty přímo, ale potřebují implementaci potomka. Standardní číselné objekty jsou neměnné.



Obrázek 12. Diagram vybraných tříd

Třída *ArrayList* je jedna z několika implementací rozhraní *List* a třída *TreeMap* je jedna z několika implementací rozhraní *Map*. *Výjimky* jsou speciální objekty typu *Throwable*, které se řídí stejnými pravidly jako ostatní třídy. Ve spojení s klíčovým slovem *throw* umožňují předčasně ukončit běh programu. Pokud je nikdo nezachytí blokem *try-catch*, vedou ke shození celé aplikace.

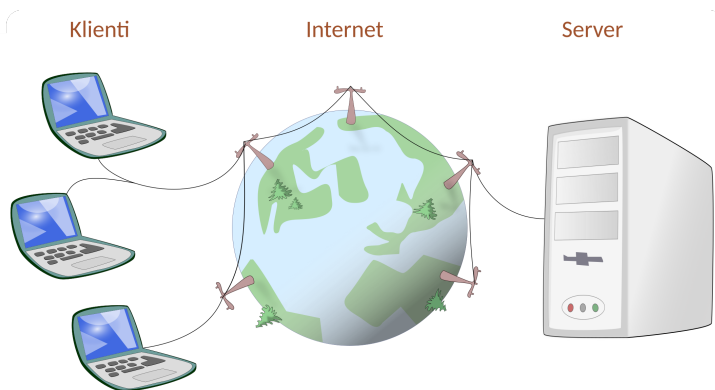
6 Webové technologie a pojmy

Síla webu spočívá v jeho univerzálnosti.

— Sir Tim Berners-Lee, vynálezce World Wide Webu a architekt internetových standardů

Na chvíli opustíme Javu a podíváme se na web. všechny nové pojmy brzy zhodnotíme v [reálných příkladech](#).

Podívejme se ale nejdřív na zjednodušené fungování webových aplikací a zkusme si představit běžného čtenáře webových stránek, který v internetovém prohlížeči klikne na nějaký odkaz. Internetový prohlížeč je aplikace, která primárně zobrazuje webové stránky v graficky pěkné podobě. Nejznámější produkty jsou: *Chrome*, *Firefox*, *Microsoft Edge*, existuje však celá řada dalších. Internetový **odkaz** si lze představit jako poštovní **adresu** stránky na internetu, která se označuje běžně zkratkou URL (je odvozena z anglického termínu: *Uniform Resource Locator*). Pro zjednodušení můžeme říci, že internetový prohlížeč předá prostřednictvím internetové sítě žádost webovému serveru o získání nějaké webové stránky. Webový server tuto žádost posoudí a vrátí zpět buď stránku očekávanou, nebo i nějakou nestandardní, která může obsahovat třeba zprávu o chybě. Taková komunikace využívá síťovou architekturu, která se označuje pojmem *klient-server*, a je pro ni typické, že **klient** žádá (prostřednictvím počítačové sítě) službu nějakého serveru a čeká na odpověď. Popsanou architekturu znázorňuje schéma:



Obrázek 13. Schéma internetové sítě

Server může používat různé technologie, my využijeme Javu. Stránky reagují na myš a klávesnici a posílají události zpět na server.

6.1 XML – rozšiřitelný značkovací jazyk

Jazyk XML (z anglického *Extensible Markup Language*) slouží k zápisu stromové struktury dat v textovém formátu. Jazyk se využívá nejen při tvorbě webových aplikací: pro svou dobrou čitelnost a podporu mnoha knihoven (například technologiemi) našel uplatnění i v oblastech zpracování dat. Uzly stromu se zapisují jako *elementy* v ostrých závorkách.



Pojem *element* označuje *uzel* stromu včetně všech jeho dětských *uzlů* (přesněji elementů).

S touto znalostí můžeme zkusit přepsat část struktury jabloně (z kapitoly [Stromová struktura dat](#)) do formátu XML, výsledek je zde:

Zdrojový kód 31.

```
<?xml version="1.0" encoding="UTF-8"?> ❶
<a> ❷
  <b> ❸
    <leaf/>
    <apple/> ❹
    <c></c>
    <d>
      <leaf/>
    </d>
  </b> ❸
  <e>
    <f>
      <leaf/>
      <g></g>
      <h>
        <leaf color="green"/> ❺
        <leaf color="yellow"/>
      </h>
    </f>
    <!-- Tady mohou být popsány další větve stromu. --> ❻
  </e>
</a>
```

- ❶ Hlavička určuje kódování, zpravidla *UTF-8*.
- ❷ Kořen stromu je popsán párem značek `<a>` – `</a>`, ukončovací značka obsahuje lomítko.
- ❸ Vnořený element je popsán párovými značkami `<b>` – `</b>`. Názvy reálných elementů bývají většinou určitá slova (či zkratky), více slov se běžně spojuje notací *CamelCase*.
- ❹ Pokud se jedná o list stromu (*uzel bez dalšího větvení*), je možné závěrečnou značku vynechat, jen lomítko se přesune před závorku uzavírající první značku podle vzoru `<apple/>`.

- 5 Atributy se píší do první značky a jejich hodnoty musí být v uvozovkách.
- 6 XML dokument může obsahovat komentáře, které se při zpracování budou ignorovat. Komentáře nelze do sebe vnořovat.

Všechny elementy v jednom *uzlu* mohou mít shodný název – na rozdíl od *tříd* z jednoho balíčku. Každý element může obsahovat *atributy*, které se zapisují za název elementu (prvního z páru) v pořadí: název, rovnítko, hodnota v uvozovkách (jednoduchých či dvojitých), viz příklad: `<apple color="red"/>`. *Bílé znaky* (mezery, tabulátory, odřádkování) umístěné mezi elementy jsou nevýznamné, a tak celý XML soubor může být zapsán na jediném řádku, pro lepší čitelnost se však každá nová úroveň elementů odsazuje (třeba mezerami). Prostor mezi elementy lze prokládat textem, pokud to má při zpracování smysl. Dokument formátu XML má několik speciálních znaků, které vyžadují zvýšenou pozornost, jejich popis je zde:

- Špičaté závorky jsou vyhrazeny pro značkování; pokud je třeba zapsat je do textu, použije se náhradní výraz `<` (pro znak `<`) a `>` (pro znak `>`).
- Ze stejného důvodu je znak `&` určen výhradně pro zápis speciálních znaků; pokud je třeba zapsat ho do textu, použije se náhradní výraz `&`.
- Aby nedošlo k záměně uvozovky v hodnotě *atributu* s jeho ohraničením, lze dvojitě uvozovky nahradit výrazem `"`; a jednoduché výrazem `'`. Alternativou je použít pro ohraničení hodnoty parametru jiné uvozovky, než obsahuje jeho tělo, například: `<apple character='"/>`



Pokud dokument splňuje vše uvedené, říkáme, že je *správně strukturovaný* či *syntakticky správný* (anglicky *well-formed*).

Pro potřeby strojového zpracování však často potřebujeme pevná pravidla obsahující seznam povolených elementů, jejich *atributů* (včetně datových typů), a také strukturu celého dokumentu. Definice takových pravidel se dnes běžně popisují souborem s příponou *XSD* (z anglického *XML Schema Definition*), což je opět dokument formátu *XML* sestavený podle určitých pravidel. Elementy mohou mít svůj *jmenný prostor*, který umožňuje sestavit XML soubor podle několika různých pravidel. V této knize se jmenným prostorem pracovat (přímo) nebudeme.



Pomocí *XSD* souboru lze ověřit platnost XML dokumentu. Pokud jeho struktura splňuje požadavky *XSD*, říkáme, že XML dokument je *platný* či *validní* (anglicky *valid*). Kontrola platnosti XML proti *XSD* dokumentu se využívá běžně před zahájením strojového zpracování (pomocí programových knihoven), v případech neshody pak dochází k přerušení dalšího zpracování dat. Pro ruční kontrolu platnosti sestaveného XML souboru lze využít (například) specializovaný editor, označovaný zkratkou *IDE*.

6.2 HTML – hypertextový značkovací jazyk

Jazyk HTML (z anglického *Hypertext Markup Language*) slouží k vytváření webových stránek. Přesněji pro grafickou prezentaci informací v prostředí webového prohlížeče. Obsah souboru ve formátu *HTML* si lze představit jako XML soubor z předchozí kapitoly, který splňuje definici jazyka HTML. V takovém případě hovoříme o specifikaci jazyka *XHTML*. Výhodou takového pojetí je snadné využití standardních nástrojů XML dokumentů, jako jsou editory, validátory kódu a různé knihovny pro strojové zpracování dokumentu. Nevýhodou jsou problémy způsobené různou mírou tolerance chyb internetových prohlížečů. Některé prohlížeče totiž zobrazují **příjemně** i nevalidní XHTML, zatímco jiné prohlížeče mohou chybný kód interpretovat jinak. To vnímají uživatelé negativně bez ohledu na skutečnou příčinu chyby.

Z toho důvodu se dnes více ujal novější formát *HTML5*, který akceptuje různé odchylky proti XML. Tímto sjednocuje vzájemnou kompatibilitu internetových prohlížečů. Specifikace *HTML5* nabízí také některé nové funkce (kreslení, přetažení a přehrávání videa) a umožňuje stručnější zápis hlavičky dokumentu. Vývojáři stránek *HTML5* však mají i nadále k dispozici formát podporující *XSD* – jako přísnější alternativu. Ukázky v této knize budou mít správně párované elementy, přesto využívají volnější specifikaci jazyka *HTML5*. Podívejme se na ukázkou...

Zdrojový kód 32. HTML5

```
<!DOCTYPE html> ❶
<html lang="en"> ❷
  <head> ❸
    <meta charset="UTF-8"/>
    <title>Demo</title>
    <link href="css/hello.css" rel="stylesheet"/>
  </head>
  <body> ❹
    <h1>Hello, World!</h1> ❺
  </body>
</html> ❻
```

- ❶ Deklarace typu dokumentu se od XML formátu liší.
- ❷ Hlavička obsahuje technické údaje a název stránky. Specifikuje také národní jazyk textu stránky v kódu ISO [3]. Kořenový element obsahuje dvě hlavní části.
- ❸ **head** – Hlavička obsahuje spíše technické informace o znakové sadě celého dokumentu, titulek okna internetového prohlížeče a odkaz na soubor kaskádových stylů (CSS, budeme se mu věnovat krátce v následující kapitole).
- ❹ **body** – Tělo obsahuje *elementy* grafických komponent zobrazených na stránce.

- 5 **h1** – Stránka má jediný element, který reprezentuje **nadpis nejvyšší úrovně**. Jeho název je odvozen z anglického *heading level 1*.
- 6 Konec obsahu HTML stránky.



Hello, World!

Obrázek 14. Zobrazení HTML kódu v prohlížeči

Začíná být zřejmé, že při tvorbě HTML stránky se bez popisu základních elementů neobejdeme, podívejme se tedy na několik nejčastějších:

- **h1** (heading level 1) – jak jsme už zmínili, jedná se o nadpis první úrovně, přitom úroveň nadpisu lze zvyšovat až do velikosti 6,
- **p** (paragraph) – odstavec běžného textu,
- **div** (division) – ohraničí obdélníkovou oblast uvnitř HTML stránky,
- **span** – ohraničí část textu na řádku,
- **a** (anchor) – element zvaný **odkaz** označuje část textu, která po kliknutí myši vede k načtení nové HTML stránky,
- **table** – tělo tabulky, které obsahuje elementy řádků (**tr**) a jednotlivé buňky (**td**), které pak tvoří sloupce,
- **form** – formulář je neviditelná obálka grafických komponent podporujících interakci uživatele. Typickým obsahem formuláře je následující *element*:
- **input** – element s textem od uživatele, který je určen k odeslání server.

Pro další studium lze použít třeba tento odkaz [20]. Je obecně doporučované, aby HTML kód tvořil především formální strukturu dokumentu, a výsledný grafický vzhled se dotváří spíše prostředky kaskádových stylů (CSS).

6.3 CSS – Kaskádové styly

Kaskádové styly (anglicky *Cascading Style Sheets*) popisují, jak se mají HTML elementy a texty zobrazovat v internetovém prohlížeči (nebo třeba i na tiskárně či jiném zařízení). Běžně nastavujeme velikost, tvar, odsazení či barvu písma, rámečků, obrázků a dalších grafických objektů. Popis kaskádových stylů je výhodné umístit do samostatného souboru (často s příponou **css**). V hlavičce HTML stránky se pak uvádí jen reference na CSS soubor – pomocí elementu **link**...

Zdrojový kód 33. Ukázka reference na soubor *hello.css*.

```
<link href="css/hello.css" rel="stylesheet"/>
```

V další ukázce vidíme zjednodušený obsah odkazovaného CSS souboru, který obsahuje dva bloky vlastností uvedených ve složených závorkách.

Zdrojový kód 34. Zkrácený obsah CSS souboru

```
body { ❶  
    color: Black; ❷  
    background-color: WhiteSmoke; ❸  
}  
h1, h2, h3 { ❹  
    color: SteelBlue;  
}
```

- ❶ Bloky zde začínají **názvem** elementu. Uvedené vlastnosti písma se aplikují také na texty **vnořených** elementů – pokud ovšem vnořený element nemá svůj vlastní styl.
- ❷ Definice barvy písma začíná názvem vlastnosti (v souladu s CSS specifikací), za kterou následuje **kód barvy** oddělený dvojtečkou. Výraz se zakončuje středníkem, *bílé znaky* okolo jsou nevýznamné. Kód barvy je třeba vybrat ze vzorkovníku pojmenovaných HTML barev, ale použít lze i vlastní barvu, která se uvádí pomocí číselného kódu. Podrobnější informace k barvám lze najít třeba zde [\[32\]](#).
- ❸ Definice barvy **pozadí** elementu *body* připomíná *barvu papíru*.
- ❹ Blok nastaví barvu pro všechny elementy nadpisů první, druhé a třetí úrovně. Seznam názvů elementů se odděluje čárkou. Stojí za povšimnutí, že barva písma nadpisu zde přepisuje barvu zapsanou v elementu *body*.

Obecně platí pravidlo, že vlastnosti textu daného *elementu* mají **větší váhu** (vyšší prioritu) než vlastnosti definované jeho rodičem. Pokud existují dvě vlastnosti se **stejnou vahou**, internetový prohlížeč aplikuje tu, která je uvedena jako poslední. Hlavička HTML stránky může obsahovat i několik CSS souborů. Pořadí odkazů na CSS soubory je důležité, protože pravidlo větší váhy se uplatňuje napříč všemi uvedenými CSS definicemi.

Výrazu uvedenému před složenou závorkou říkáme *CSS selektor*. Tento *selektor* však nemusí obsahovat jen **název** elementu, ale může využívat také jejich *atributy* a vzájemně je kombinovat. Jasnější to bude na příkladech formátování **nadpisu** z předchozí HTML stránky, který doplníme několika málo atributy.

```
<body class="intro">
  <h1 class="topTitle" id="code7">Hello, World!</h1>
</body>
```

Jinými slovy lze vyjádřit, že CSS selektory vymezují působnost kaskádových stylů (CSS) na HTML stránce. Následující tabulka obsahuje příklady nejčastějších použití selektorů. Všechny uvedené příklady obarví nadpis červenou barvou.

Tabulka 10. Příklady selektorů

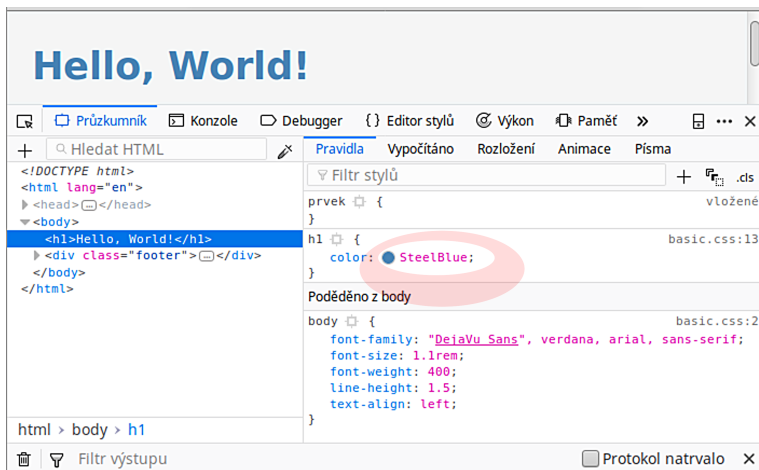
# Selektor	Typ	Ukázka	Popis
1. <code>h1</code>	typový selektor	<code>h1 {color:red;}</code>	Selektor typu elementu je identický s názvem elementu. Ukázka obarví na stránce všechny nadpisy první úrovně.
2. <code>.topTitle</code>	třída	<code>.topTitle {color:red;}</code>	Selektor třídy začíná tečkou (.). Výraz označí všechny elementy obsahující uvedenou CSS třídu.
3. <code>#code7</code>	identifikátor	<code>.code7 {color:red;}</code>	Selektor identifikátoru začíná mřížkou (#). Uvedený selektor označuje element s identifikátorem <code>code7</code> . Protože identifikátor elementu je unikátní, na jedné HTML stránce by se měl vyskytovat právě jeden.
4. <code>h1:nth-child(1)</code>	pseudo-selektor	<code>h1:nth-child(1) {color:red;}</code>	Selektor označuje první nadpis ve skupině. Číslo v kulatých závorkách uvádí pořadí elementu, ale může obsahovat také pokročilejší výrazy.
5. <code>h1.topTitle</code>	kombinace	<code>h1.topTitle {color:red}</code>	Výše uvedené typy selektorů lze kombinovat. Zde uvedený selektor označuje všechny nadpisy (první úrovně) obsahující třídu <code>topTitle</code> . Pokud selektor třídy oddělíme mezerou, povolíme hledání i ve vnořených elementech.
6. <code>body h1</code>	kombinace	<code>body h1 {color:red;}</code>	Tento zápis označí element (<code>h1</code>), který je vnořený do nějakého rodiče. Selektory s více elementy mají obecně větší váhu než ty stručnější.
7. <code>.intro .topTitle</code>	kombinace	<code>.intro .topTitle {color:red;}</code>	Selektor označuje všechny elementy s třídou <code>topTitle</code> , které jsou v těle elementu s třídou <code>intro</code> .



U nového webového projektu je vhodné začít výběrem nějaké hotové CSS knihovny a v HTML kódu používat především *názvy stylů* z této knihovny. Ve vlastním CSS souboru pak popisujeme už jen specifické požadavky. Pro začátek můžeme zkusit třeba

populární knihovnu *Bootstrap* [25]. Vlastními CSS styly pak lze realizovat jen specifické požadavky na grafický vzhled.

Styly dostupných HTML stránek lze studovat pomocí nástrojů internetového prohlížeče pro vývojáře. V prohlížeči *Firefox* zkusme otevřít [první ukázkou](#) k této knize: levým tlačítkem klikněte na text nadpisu, pak vyskočí kontextové menu a v něm vyberme volbu **Prozkoumat**. Na pravé straně vidíte název barvy elementu **h1**, v tomto režimu lze styly také (dočasně) upravovat. Výchozí vzhled stránky lze obnovit stiskem klávesy **F5**. Měli bychom spatřit podobnou obrazovku:



Obrázek 15. Zobrazení stylů v prohlížeči *Firefox*

Použití CSS může být tématem na samostatnou knihu. Pro další studium můžete použít třeba tyto odkazy: [21] či [31]. Znalost selektorů využijeme ještě v kapitole [AJAX](#).

6.4 URL – Adresa internetové stránky

Pojem *URL* (z anglického *Uniform Resource Locator*) je jednořádkový řetězec, který slouží k adresování webových zdrojů, mezi něž patří zejména HTML stránky, CSS styly a obrázky. V *URL* je možné najít určitou podobnost s adresováním souborů na disku lokálního počítače, *URL* však podporuje některé další části, například parametry. Většina internetových prohlížečů zobrazuje aktuální *URL* ve své horní části, kde mají uživatelé možnost ji změnit. Velká a malá písmena mohou být významná.

Na příkladu konkrétního *URL* si popíšeme důležité části, přitom je dobré vědět, že některé z nich jsou nepovinné.

1. **https** – Označuje název komunikačního protokolu pro bezpečnou výměnu dat v počítačové síti pomocí protokolu *SSL*. Nezabezpečený protokol má kód *http*, pro čtení souborů z lokálního disku lze použít protokol *file*.
2. **://** – Oddělovač protokolu.
3. **stackoverflow.com** – Doména druhého a prvního řádu se odděluje tečkou a společně definují jedinečný **název hostitelského počítače**. Tento název se převádí pomocí DNS serverů na jedinečný identifikátor hostitelského počítače zvaný **IP adresa** (z anglického *Internet Protocol address*). Při sestavování *URL* adresy lze použít také IP adresu, ale častěji se používají doménová jména. Pro oddělení případné domény třetího řádu a dalších se používá rovněž tečka.
4. **/** – Každé další lomítko odděluje popis detailního umístění zdroje na serveru.
5. **:443** – Číslo portu, lze si ho představit jako komunikační kanál serveru. Pokud číslo portu chybí, klient použije nějaké výchozí číslo dané konfigurací serveru.
6. **search** – Specifikace detailního umístění na serveru může někdy sloužit i jako doplňkový parametr (příkaz).
7. **?** – Oddělovač parametrů.
8. **q=java** – Zápis *URL* parametru, kde **q** je jeho název a **java** je jeho hodnota. Pokud *URL* obsahuje více parametrů, oddělují se znakem **&**.

Připomeňme si, že HTML stránky nabízejí elementy typu **odkaz**, umožňující načtení jiné HTML stránky pomocí adresy zvané *URL*. Adresování může být absolutní (obsahující celou *URL*), nebo relativní k aktuální HTML stránce. Stránku na stejné adrese lze odkazovat pouhým názvem (s volitelnými parametry). To je také případ adresování CSS souboru v předchozí kapitole pomocí parametru **href**.

6.5 JSON – úsporný datový formát

Značkovací jazyk *XML* není jediný formát pro zápis strukturovaných dat, do skupiny nejpoužívanějších patří také *JSON* (zkratka z anglického *JavaScript Object Notation*). Formát vznikl původně jako komunikační protokol mezi internetovým prohlížečem a serverem, brzy však našel uplatnění i v řadě dalších oblastí.

Podobně jako *XML* umí i *JSON* formát popisovat stromovou strukturu dat v textovém formátu. Jeho zápis je úspornější, podporuje několik datových typů a dobře se používá v kombinaci se skriptovacím jazykem *JavaScript*, nepodporuje však komentáře. Rozdílů by se našlo více.

Data *JSON* formátu jsou zapsána mezi složenými závorkami (`{}`) a tvoří je párové výrazy typu *klíč: hodnota*, které jsou vzájemně oddělené čárkou (`,`). Bílé znaky kolem obou oddělovačů (`:` a `,`) jsou nevýznamné, a tak lze celý *JSON* zapsat i na jediný řádek. Podívejme se na následující ukázkou a pro zajímavost si všimněme podobnosti zápisu hodnot s *Java* literály.

Zdrojový kód 36. Datový soubor ve formátu JSON.

```
{
  "personalNumber" : 10, ①
  "fullName" : "Joe Black", ②
  "active" : true, ③
  "boss" : null, ④
  "series" : [-1, 8, 10], ⑤
  "car" : {"model" : "Scala"} ⑥
}
```

- ① Ukázka zápisu celého čísla. Specifikace *JSON* podporuje i typy s plovoucí čárkou zapsané podle vzoru: `3.1415`, platných formátů (pro zápis plovoucího čísla) je však více.
- ② Příklad zápisu textu, který musí být ohraničen dvojími uvozovkami. Případně uvozovky obsažené v textu (či další speciální znaky) je třeba předřadit zpětným lomítkem. Stejná pravidla platí i pro zápis klíče, který tak může obsahovat mezeru i speciální znaky.
- ③ Hodnota s logickým datovým typem, alternativou je hodnota `false`.
- ④ Nedefinovaná hodnota se zapisuje výrazem `null`.
- ⑤ Pole může obsahovat (kromě čísel) i ostatní podporované datové typy.
- ⑥ Platnou **hodnotou** může být i vnořená struktura *JSON* formátu, která se uvádí včetně složených závorek, přitom hloubka vnoření je prakticky neomezená. Vnořenou strukturu lze vkládat i do pole.

Pro podrobnější informace přikládám odkaz na Wikipedii [\[30\]](#), technické informace jsou uvedeny specifikaci *RFC 4627*. Znalost datové struktury *JSON* využijeme v kapitole [Interaktivní aplikace s AJAX](#).

6.6 Shrnutí

Programový objekt je instance třídy, která má své *jméno* a *balíček*, *atributy* (data) a *metody* (algoritmy). Jeho metody přijímají parametry a vracejí výsledek. *Java* používá *primitivní* datové typy, *třídy*, anebo *pole*. *Dědičnost* umožňuje sdílet kód mezi rodičem a potomkem. *Statické* prvky patří třídě, nikoliv objektu. Internetový *prohlížeč* komunikuje se serverem pomocí protokolu *HTTP* a zobrazuje *HTML* dokumenty formátované *CSS*. Teď už víme vše důležité, pojďme na příklady.

7 Řešené příklady

Žijete jenom jednou. Tak by to měla být zábava.

— Coco Chanel, francouzská módní návrhářka

Ve druhé části knihy najdeme výklad vybraných částí zdrojového kódu z příložených příkladů [2]. Projekt příkladů je uspořádán ve formátu *Maven*. Zdrojový kód lze používat v souladu s [licencí](#) popsanou v úvodu knihy. Každá z kapitol začíná stručným zadáním úkolu a pokračuje popisem programového řešení. Ke stejnému cíli však mohou vést i jiné postupy. Závěrem kapitoly jsou uvedeny tipy na další úpravy a rozšíření. Témata kapitol na sebe navazují, pracovat tam budeme s pojmy z první části knihy. Po přečtení každé kapitoly je vhodné experimentovat se staženým zdrojovým kódem projektu a studovat vliv vlastních změn na chování aplikace. Pro obnovu původního řešení stačí kořenový adresář projektu smazat a nakopírovat tam znovu soubory z původního ZIP archivu.

Jak už bylo zmíněno dříve, jednotlivé příklady lze po spuštění projektu zobrazovat v internetovém prohlížeči jako HTML stránky. Šikovný editor integrovaný do [vývojového prostředí](#) přijde vhod při vlastních úpravách či tvorbě nového kódu. Pro pouhé zobrazení kódu stačí využít odkaz na zdrojový kód, který je umístěn v patičce stránky.

Source code of the HelloWorldServlet servlet class

HelloWorldServlet.java

```
1. /* Copyright 2018-2022 Pavel Ponec, https://jbook.ponec.net */
2. package net.ponec.jbook.s01_hello;
3.
4. import java.io.IOException;
5. import java.io.PrintWriter;
6. import javax.servlet.ServletException;
7. import javax.servlet.annotation.WebServlet;
8. import javax.servlet.http.HttpServlet;
9. import javax.servlet.http.HttpServletRequest;
10. import javax.servlet.http.HttpServletResponse;
11. import net.ponec.jbook.tools.WebTools;
12.
13. /** A simple servlet implementation */
14. @WebServlet("/helloWorld")
15. public class HelloWorldServlet extends HttpServlet {
16.
17.     /** Handles the HTTP GET request */
18.     @Override
19.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

Obrázek 16. Ukázka zdrojového kódu v prohlížeči

7.1 Instalace

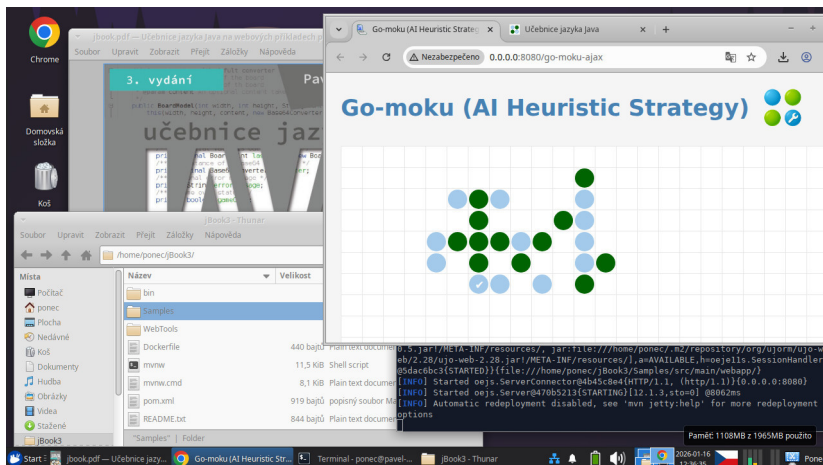
Volba operačního systému je na nás, hlavními kandidáty jsou *Windows*, *Linux* nebo *MacOS*. Kontrolu příkladů jsem zkoušel na dvou systémech s instalovaným OpenJDK 25 LTS. To je *Java* s otevřenou licencí [4] a dlouhodobou podporou (anglicky *Long Term Support*).

7.1.1 Hardware a operační systém

Operační systém je důmyslný software zavedený trvale v paměti počítače, který poskytuje služby aplikacím. Jedná se například o zpracování událostí z klávesnice a myši, zprostředkování výstupu na monitor, přidělování paměti, práci s diskem nebo síťovou komunikaci. Minimální nároky na *hardware* (dále jen *HW*) závisí na zvoleném operačním systému.

Počítač s *Windows* 11 potřebuje pro spuštění příkladů alespoň 8 GB paměti RAM a 128 GB místa na disku. Na počítači s *Linux Xubuntu* 25.10 postačovala paměť 2 GB RAM a kapacita disku 10 GB. Dvojnásobek minimální paměti RAM však pravděpodobně zrychlí každý operační systém. Instalace *Javy* a dalších nezbytných knihoven obsadila na pevném disku asi 500 MB. Ověřování minimálních požadavků probíhalo na virtuálním počítači.

Operační systém *Xubuntu* [17] je lehká linuxová distribuce s grafickým prostředím *Xfce*. Je podporována firmou *Canonical* a vychází z distribuce *Ubuntu* založené na systému *Debian GNU/Linux*. Její použití není zatíženo licenčními poplatky, což platí pro většinu linuxových distribucí.



Obrázek 17. Ilustrační obrázek upravené plochy operačního systému Xubuntu

7.1.2 Apache Maven

Pro spuštění naší výukové *Java* aplikace potřebujeme získat knihovny, na kterých aplikace závisí. Dále musíme zdrojový projekt zkompileovat a spustit. Požadavků by se však našlo více. Uvedené činnosti podporuje několik SW produktů. V této knize budeme používat populární *Apache Maven*, což je volně šiřitelný nástroj usnadňující automatizované sestavení a správu SW projektu.

Aby mohl *Maven* s projektem pracovat, je třeba zdrojové soubory uložit v předepsané adresářové struktuře. Vedle zdrojových kódů *Javy* může projekt obsahovat také textové, multimediální a další pomocné **soubory**, pokud jsou nezbytné pro běh programu. V hlavním adresáři projektu *Maven* se nachází textový soubor s názvem `pom.xml`. Tento soubor obsahuje formální informace o projektu (název, verzi) a údaje o závislostech na externích knihovnách či požadované verzi *Javy*. Pojem *Maven projekt* v této knize označuje zdrojový kód *Java* projektu rozložený do adresářů v souladu s požadavky nástroje *Apache Maven*. Takový projekt se může skládat z několika dílčích částí, kterým se říká *moduly*. Každý takový *modul* má strukturu *Maven* projektu, včetně vlastního souboru `pom.xml`. *Maven* sestaví projekt pomocí následujícího zjednodušeného postupu:

1. Přečte konfigurační soubor `pom.xml`.
2. Zkontroluje instalovanou verzi *Java Development Kit* (JDK).
3. Zkontroluje strukturu projektu.
4. Při prvním sestavení si *Maven* stáhne z centrálního repozitáře (na internetu) potřebné knihovny, kterým se říká *závislosti* (anglicky *dependencies*). Tyto knihovny ukládá do adresáře `.m2` v domovské složce uživatele. Opakované spuštění je pak rychlejší, protože závislosti se stahují pouze jednou.
5. Provede kompilaci zdrojového kódu.
6. Výsledné soubory zabalí do formátu vhodného k distribuci. V našem případě bude mít soubor příponu `war` (zkratka z anglického *web archive*).

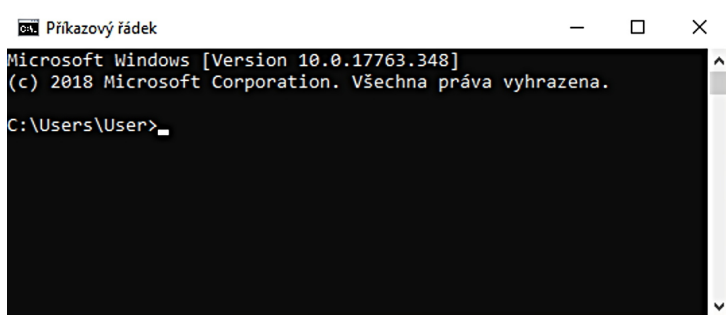
Vybrané příkazy *Mavenu* budeme používat ke spouštění aplikace, podrobnější popis nástroje je však nad rámec této knihy. Reálnou ukázkou připojení knihovny lze najít v kapitole [Datový model HTML stránky](#).

Součástí výukového projektu je skript *Maven Wrapperu*, který si při prvním spuštění automaticky stáhne potřebnou verzi *Mavenu*. Díky tomu není nutné *Maven* instalovat ručně. Další informace lze najít v rychlém úvodu na odkazu [\[7\]](#).

7.1.3 Znakový terminál

Příkazy nástroje *Maven* budeme zadávat prostřednictvím aplikace zvané *emulátor znakového terminálu*. Pojem *znakový terminál* původně označoval fyzické zařízení pro ovládání počítače pomocí klávesnice a textového monitoru. Textové příkazy se používají dodnes, ale vkládají se do SW *emulátoru*, který v grafických systémech vypadá jako okno s textem. Místo pro zápis příkazů se nazývá *příkazový řádek* a je označeno blikajícím *kurzorem*. Jednotlivé příkazy se potvrzují klávesou **Enter**. Potvrzený příkaz obecně nelze vrátit zpět. *Emulátor znakového terminálu* lze spustit z menu operačního systému:

1. ve Windows se aplikace jmenuje *Příkazový řádek* (nebo *Terminal*),
2. v systému Linux Xubuntu je uveden pod názvem *Emulátor terminálu*.



Obrázek 18. Ukázka znakového terminálu spuštěného ve Windows 10

7.1.4 Získání příkladů

1. Stáhneme zdrojové kódy příkladů z URL adresy [\[2\]](#) ve formátu ZIP.
2. Soubory z archivu ZIP zkopírujeme do nové *složky* (v Linuxu se častěji používá pojem *adresář*). Název složky doporučuji zvolit bez mezer a diakritiky.

7.1.5 Instalace vývojového prostředí

Postup instalace vývojového prostředí Java je na různých systémech podobný:

1. Stáhneme instalační archiv *OpenJDK* verze 25 [\[4\]](#).
2. Seznámíme se s licenčními podmínkami.
3. Archiv rozbalíme do vhodného adresáře na disku.

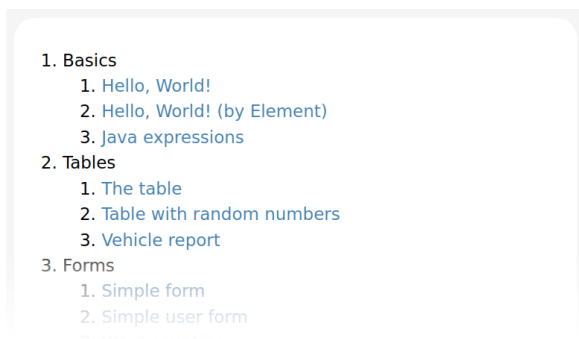
Pro instalaci jiné distribuce *Javy* postupujte podle pokynů dodavatele. Doplňující informace lze najít také na domovské stránce knihy [\[1\]](#).

7.1.6 Spouštění příkladů

V terminálu se přepneme do adresáře projektu a ve *Windows* spustíme příkazy:

1. `.\mvnw clean install`
2. `.\mvnw -f Samples/pom.xml jetty:run`

V Linuxu je postup stejný, pouze se používají opačná lomítka (`./mvnw`). Pro nastavení spustitelnosti skriptu v Linuxu je třeba použít příkaz: `chmod +x mvnw`.



Obrázek 19. Zkrácená nabídka příkladů

Zbývá spustit váš oblíbený internetový prohlížeč (například z menu operačního systému) a vložit do adresního řádku text <http://localhost:8080/>. Měla by se zobrazit nabídka řešených příkladů. Aplikaci lze ukončit v terminálu klávesami `Ctrl+C` nebo zavřením okna konzole. Skript `mvnw` nepíšeme ručně, ale přebíráme jej z projektu *Maven Wrapper*.

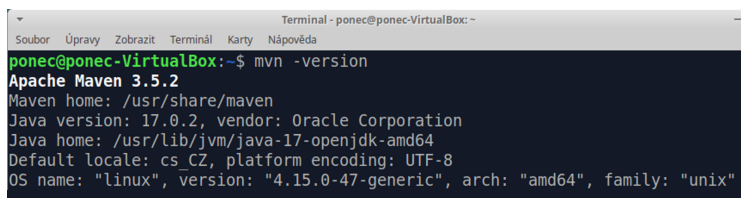
7.1.7 Co dělat při potížích?

Pokud prohlížeč zobrazí stránku s nabídkou, lze zbytek této kapitoly přeskočit. V opačném případě zvažte následující kroky:

1. Zkontrolujte instalaci Mavenu: v terminálu zadejte příkaz:

```
./mvnw --version
```

Měli byste vidět informaci o verzi Mavenu a JDK.



```
Terminal - ponec@ponec-VirtualBox: ~
Soubor  Úpravy  Zobrazit  Terminál  Karty  Nápověda
ponec@ponec-VirtualBox:~$ mvn -version
Apache Maven 3.5.2
Maven home: /usr/share/maven
Java version: 17.0.2, vendor: Oracle Corporation
Java home: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: cs_CZ, platform encoding: UTF-8
OS name: "linux", version: "4.15.0-47-generic", arch: "amd64", family: "unix"
```

Obrázek 20. Výpis verze nástroje Maven v Linuxu

Je možné, že bude nutné v textovém editoru upravit skript `mvnw` (nebo `mvnw.cmd`) tak, aby proměnná `JAVA_HOME` ukazovala na adresář s instalovanou Javou. Lze použít i relativní cestu, pokud je Java rozbalena přímo v projektu. Uživatelé Windows editují `mvnw.cmd`, uživatelé Linuxu soubor `mvnw`.

Zdrojový kód 37. Upravený řádek souboru `./mvnw.cmd` ve Windows

```
set "JAVA_HOME=C:\Users\User\Documents\Java\jdk-25"
```

2. Zkontrolujeme dostatek volného místa na pevném disku. Pokud zbývá méně než 100 MB, smažeme nepotřebné soubory.
3. Ověřte dostatek operační paměti a ukončete nepotřebné aplikace.
4. Prozkoumejte chybovou zprávu v terminálu. Pro podrobnější informace zkusme vložit klíčová slova chybové zprávy do internetového vyhledávače.
5. Pokud narazíte na chybu o obsazeném portu:

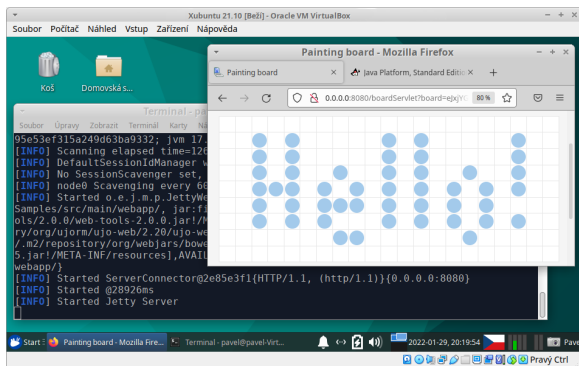
```
Failure: Failed to bind to 0.0.0.0/0.0.0.0:8080: Adresa je užívána -> [Help 1]
```

+ Máte dvě možnosti:

- a. Najděte a ukončete aplikaci, která port 8080 využívá.
- b. V souboru `Samples/pom.xml` změňte číslo portu `8080` na jiné (např. `8090`).
 1. Hledejte řešení v často kladených dotazech na domovské stránce knihy [\[1\]](#).
 2. Nechte chybu posoudit nějakým produktem umělé inteligence.

7.1.8 Virtuální počítač

Pokud projekt stále neběží, můžete zkusit využít *VirtualBox* a nainstalovat do něj čistý *Linux Xubuntu* [\[17\]](#). V tomto prostředí byl postup opakovaně a úspěšně testován. Virtuální počítač běží v samostatném okně a izolovaně od hostitelského systému.



Obrázek 21. Spuštění příkladů v prostředí aplikace VirtualBox

Z hlediska bezpečnosti procesy ve virtuálním stroji neovlivňují váš hlavní počítač. Počítejte však s určitou ztrátou výkonu a s vyššími nároky na počítač. Alternativou k plné virtualizaci je použití virtualizačních *kontejnerů*.

7.1.9 Virtualizační kontejner

Virtualizační *kontejner* si lze představit jako lehký virtuální počítač, který běží na našem počítači s minimálními nároky na zdroje, kde dochází ke spuštění prakticky okamžitě. *Kontejner* nemívá zpravidla grafické prostředí a ovládá se textovými příkazy – podobně jako v běžném znakovém terminálu. Pro kontejner je vhodné použít operační systém s otevřenou licencí. Tyto kontejnery nabývají na popularitě díky vysokému výkonu, dobré automatizovanosti, přenositelnosti aplikací a vlastnostem umožňujícím efektivnější využívání stávajících systémových prostředků.

Spuštění našeho projektu v kontejneru si můžete vyzkoušet pomocí aplikace *Docker*® [28]. Nastavení kontejneru je popsáno v souboru *Dockerfile*. Uživatelům *Windows* doporučuji nejprve ověřit technické požadavky na instalaci *Dockeru*. Projekt poté spustíme z terminálu pomocí připraveného skriptu ...

Zdrojový kód 38. Spuštění projektu v kontejneru na Windows

```
bin\docker-run.cmd
```

Uživatelé systému *Ubuntu* spustí projekt skriptem, který umí *Docker* v případě potřeby doinstalovat. Pro spuštění jsou vyžadována administrátorská oprávnění.

Zdrojový kód 39. Spuštění projektu v kontejneru na Linuxu

```
sh bin/docker-run.sh
```

Skript provede následující kroky:

1. Vytvoří linuxový [29] kontejner s OpenJDK podle souboru `Dockerfile`.
2. Zkopíruje do něj Maven projekt.
3. Spustí skript, který projekt zkompileje a spustí jako webovou aplikaci.
4. Stránky budou dostupné v prohlížeči na adrese `localhost:8080`.
5. Ukončení provedete klávesami `Ctrl+C`.

Podrobný popis Dockeru přesahuje rámec této publikace. Více informací naleznete v dokumentaci výrobce [28].

7.1.10 Vývojové editory

Pro profesionální vývoj se používají *integrovaná vývojová prostředí* (IDE). Nabízejí editor s doplňováním kódu, zvýrazňováním syntaxe, integrovaný kompilátor a nástroje pro ladění (debugging). Podívejme se na nejrozšířenější IDE, která jsou dostupná zdarma:

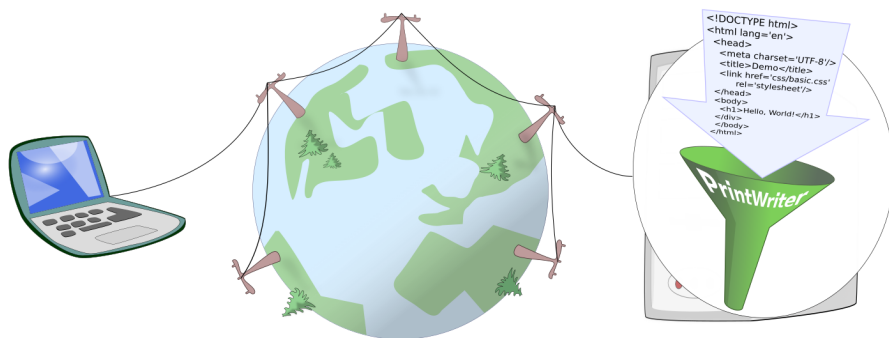
1. **IntelliJ IDEA**® je v současnosti nejpoužívanější IDE pro Javu. Nabízí inteligentní refaktoring a vysokou produktivitu. Komunitní edice (Community Edition) je zdarma. Odkaz ke stažení: [8].
2. **Apache NetBeans**® je projekt spravovaný nadací Apache. Nabízí intuitivní ovládání a výbornou integraci s Mavenem. Vhodné pro začátečníky. Odkaz: [8].
3. **Eclipse**® je tradiční open-source IDE s širokou škálou doplňků. Je napsáno v Javě, ale používá nativní grafické knihovny SWT.

7.2 Servlety

Pojem *servlet* označuje v této knize třídu odvozenou od `HttpServlet`. Tato třída zpracovává HTTP dotazy (request) a odesílá odpovědi (response) ve formátu HTML zpět do prohlížeče. Pro běh servletů potřebujeme *webový server* nebo *webový kontejner*, který požadavek přijme a předá ho správnému servletu. Náš projekt využívá odlehčený kontejner `Jetty`®. Typická komunikace probíhá takto:

1. Uživatel zadá do prohlížeče URL adresu.
2. Prohlížeč odešle *požadavek* (request) typu `GET` na server.
3. Webový kontejner požadavek přijme a podle URL identifikuje příslušný servlet.
4. Na tomto servletu zavolá metodu `doGet()` a předá jí objekt `PrintWriter` pro zápis odpovědi. Fyzicky komunikace probíhá přes síťové protokoly, což za nás řeší

knihovny a operační systém. Při lokálním testování náš počítač simuluje server i klienta zároveň.



Obrázek 22. Schéma odeslání dat třídou *PrintWriter*

1. Prohlížeč odešle požadavek přes síť.
2. Server požadavek zpracuje a vrátí HTML stránku.
3. Metoda `close()` na konci komunikaci uzavře.
4. Prohlížeč zprávu zpracuje a vykreslí její grafickou podobu na monitor.

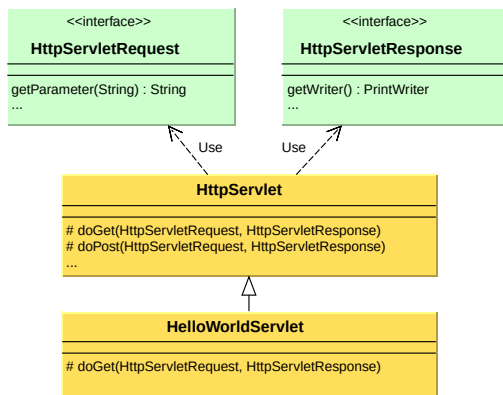
7.2.1 Hello, World!

Naším prvním úkolem je vypsát text „Hello, World!“ do prohlížeče ve formátu HTML. Kód výsledné stránky by mohl vypadat takto:

Zdrojový kód 40. Obsah výsledné HTML stránky

```
<!DOCTYPE html>
<html lang='en'>
  <head>
    <meta charset='UTF-8'>
    <title>Demo</title>
    <link href='css/hello.css' rel='stylesheet'>
  </head>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

My vytvoříme třídu dědící z `HttpServlet` a přepřekrýt metodu `doGet()`. Třidu nazveme třeba `HelloWorldServlet`.



Obrázek 23. Diagram servletu

V diagramu vidíme, že třída `HttpServlet` využívá typy pro *požadavek* (anglicky *request*) a *odpověď* (anglicky *response*). Tyto typy jsou definovány jako *rozhraní* (interface). Rozhraní předepisuje, jaké metody musí objekt obsahovat, ale samotnou implementaci provádí až konkrétní třída. Budeme potřebovat:

1. `HttpServletRequest` nabízí metody, pomocí kterých lze získat parametry URL dotazu (anglicky *request*).
2. `HttpServletResponse` nabízí také několik metod; pro náš úkol je nejdůležitější metoda poskytující objekt typu `PrintWriter`, pomocí které budeme zapisovat texty určené do internetového prohlížeče.

První servlet

Zde je kompletní implementace našeho prvního servletu:

Zdrojový kód 41. Třída `HelloWorldServlet`

```
/* License terms */ ①
package net.ponec.jbook.s01_hello; ②

import java.io.IOException; ③
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import net.ponec.jbook.tools.WebTools;
```

```
/** A simple servlet implementation */ ④
@WebServlet("/hello-world") ⑤
public class HelloWorldServlet extends HttpServlet { ⑥

    /** Handles the HTTP GET request */ ⑦
    @Override ⑧
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) ⑨
        throws ServletException, IOException { ⑩
        String html = "" ⑪
            <!DOCTYPE html>
            <html lang='en'>
            <head>
                <meta charset='UTF-8'>
                <title>Demo</title>
                <link href='css/hello.css' rel='stylesheet'>
            </head>
            <body>
                <h1>Hello, World!</h1>
                %s ⑫
            </body>
            </html>
            """.formatted(WebTools.createFooter(this)); ⑬
        PrintWriter writer = response.getWriter(); ⑭
        writer.append(html); ⑮
    }
}
```

- ① Úvodní komentář k licenci.
- ② Definice *balíčku* (package), která musí odpovídat umístění souboru v adresářové struktuře.
- ③ Importy tříd umožňují používat jejich krátké názvy (bez balíčků).
- ④ JavaDoc komentář třídy.
- ⑤ Anotace třídy s názvem `@WebServlet` obsahuje ve svém parametru **umístění** HTML stránky, která se stane součástí adresy URL. Pomocí tohoto označení pak může server spárovat za běhu aplikace přijatý webový požadavek s instancí požadovaného servletu. Pokud v anotaci nastavujeme parametrů více, je třeba uvádět je v obecném formátu dle vzoru `value="text"` a jednotlivé výrazy oddělovat čárkou. Pak je také nezbytné hodnoty parametrů zapisovat včetně jejich názvů následovaných rovnítkem. Pokud v *anotaci* nepoužijeme žádný parametr, kulaté závorky anotace jsou nepovinné. V našem případě bude URL adresa vypadat takto: `http://localhost:8080/helloWorld`. Význam *anotace* si lze představit jako nálepku na zavařenině: nálepka nemá vliv na chuť zavařeniny, ale může usnadnit její vyhledávání v regálu.
- ⑥ Posledním zápisem první úrovně je vlastní název třídy `HelloWorldServlet`, který obsahuje deklaraci třídy začínající dvěma klíčovými slovy:

1. *public* – třída je viditelná pro všechny ostatní třídy,
 2. *class* – jedná se o třídu; typ *rozhraní* (anglicky *interface*) používá na tomto místě klíčové slovo *interface*.
 3. Následuje název třídy s velkým počátečním písmenem na začátku.
 4. Klíčové slovo *extends* uvádí rodiče třídy z pohledu dědičnosti: pokud by třída implementovala nějaká rozhraní, na tomto místě by se použilo klíčové slovo *implements*.
 5. Tělo třídy je pak ohraničeno párem složených závorek { a }; v našem případě obsahuje pouze jedinou metodu.
- 7 Popis **těla třídy** se pro zvýraznění struktury zdrojového kódu odsazuje (zleva) počátečními mezerami. Následující čtyři řádky jsou nedílnou součástí popisu metody; začínáme komentářem ve formátu *JavaDoc*.
- 8 Na dalším řádku vidíme *anotaci metody* s názvem *@Override*, která deklaruje, že metoda překrývá metodu nějakého předka. Pokud bychom udělali překlep v názvu metody, jejího parametru či návratového typu, pak nás kompilátor díky této anotaci dokáže včas upozornit na problém. Anotace se považují za nedílnou součást deklarace metody, obecně jsou však nepovinné.
- 9 Deklarace metody pokračuje dvěma klíčovými slovy:
1. *protected* – je modifikátor metody povolující její viditelnost pouze potomkům nebo třídám ze stejného balíčku,
 2. *void* – je náhrada návratového typu, deklarující, že tahle metoda **nevrací nic**. Následuje název metody (*doGet*) začínající malým počátečním písmenem. V kulaté závorce jsou uvedeny vstupní parametry metody, oddělené čárkou. Každý parametr metody se uvádí deklarací datového typu.
- 10 Klíčové slovo *throws* uvádí seznam výjimek oddělených čárkou, které může metoda vyhazovat. Obě výjimky uvedené jsou kontrolované, viz kapitola [Výjimky](#). Kód algoritmu se zapisuje do prostoru ohraničeného párem složených závorek { a } a pro přehlednost se odsazuje mezerami.
- 11 Obsah HTML stránky sestavíme do pomocné proměnné *html* pomocí víceřádkového textového literálu. Připomeňme, že začátek a konec textu se označuje třemi uvozovkami.
- 12 Metoda *formatted()* dosadí obsah svého argumentu na místo značky *%s*. Tímto způsobem vložíme do HTML kódu patičku stránky obsahující odkaz na zobrazení zdrojového kódu *servletu* a odkaz pro návrat do hlavního menu. Klíčové slovo *this* v argumentu metody *createFooter()* vyjadřuje referenci na svůj vlastní objekt. Do metody tedy posílá sebe. Zdrojový kód třídy *WebTools* je součástí přiloženého projektu, ale jeho popis není předmětem našeho studia.

- 13 Z argumentu `response` získáme metodou `getWriter()` objekt typu `PrintWriter`, který zapíšeme do proměnné `writer`.
- 14 Obsah proměnné `html` pošleme do internetového prohlížeče metodou `append()` na objektu `PrintWriter`. Uzavření objektu `PrintWriter` provede služba, která tento objekt vytvořila. Metoda `doGet()` nemá žádný návratový typ a náš zápis algoritmu tím může skončit.

Výsledek servletu

Po spuštění a otevření URL v prohlížeči uvidíte výsledek:



Obrázek 24. Výsledek servletu zobrazený internetovým prohlížečem

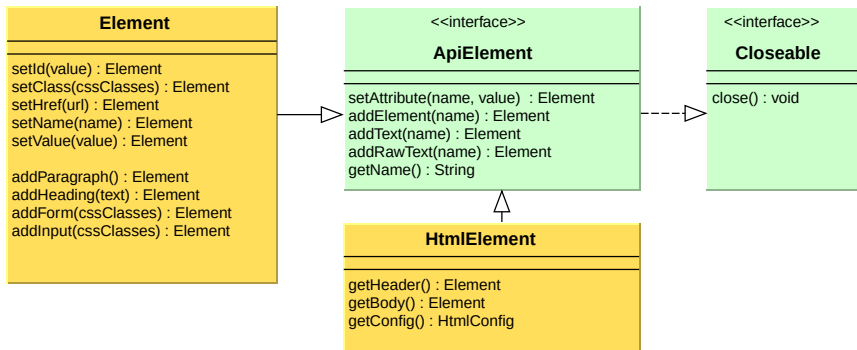
Závěrem provedme ve zdrojovém kódu vlastní změny: Upravte text pozdravu tak, aby obsahoval vaše jméno. Zatím se vyhněte diakritice.

7.2.2 Datový model HTML stránky

Předchozí způsob sestavování HTML je náchylný k překlepům a chybám v sintaxi. Problematické může být i ošetření speciálních znaků a diakritiky. Proto je lepší využít **objektový model**, kde HTML strukturu vytváříme pomocí metod Javy. Použijeme volně dostupnou knihovnu *Ujorm*, která poskytuje třídu `Element` pro modelování HTML stromu. Pro práci nám stačí znát v principu tři základní metody:

- `setAttribute()` pro nastavení atributů elementu,
- `addElement()` pro přidání nového vnořeného elementu,
- `addText()` pro bezpečné přidání textu. Pro zápis surového textu (například těla *JavaScriptu*) je k dispozici alternativní metoda `addRawText()`.

Třída `Element` nabízí množství metod připravených pro snadné vkládání běžných HTML elementů (například: nadpis, odstavec, formulář, tabulka) a jejich atributů (například: identifikátor, CSS třídu, URL odkaz). Výhodou tohoto přístupu je, že některé chyby HTML kódu odhalí už Java kompilátor. Vhodné sestavení HTML elementů však zůstává nadále v kompetenci vývojáře. Knihovna také nastavuje správnou kódovou stránku pro zápis textu s diakritikou a třeba omezuje dočasné ukládání stránek v internetovém prohlížeči (cache), což je nezbytné pro správné překreslování stránek. Následuje zjednodušený diagram třídy `Element`.



Obrázek 25. Diagram třídy *Element*

1. Začneme popisem rozhraní **ApiElement**, jehož metody umožňují sestavit model libovolného XML či HTML souboru.
 - a. Metody s prefixem **add** přidávají nový *objekt* do stávajícího elementu. Pokud se jedná o nový **Element**, vrací jeho instanci, jinak metoda vrací referenci na sebe za účelem možného řetězení volaných metod – v takových případech zde zpravidla o přidávání textu.
 - b. Metody s prefixem **set** zapisují do elementu atribut s hodnotou.
 - c. Metoda **close()** z rozhraní **Closeable** uzavírá element. K uzavírání elementů dochází automaticky s výjimkou kořenového elementu, který je třeba uzavřít explicitně.
 - d. Metoda **getName()** poskytuje název stávajícího elementu.
2. Třída **Element** nabízí metody poplatné jazyku HTML, konvence prefixu **add** a **set** (z rodičovské třídy) platí i zde.
 - a. Metoda **setId()** přiřadí k elementu identifikátor, který má být na HTML stránce jedinečný.
 - b. Metoda **setClass()** umí přiřadit jeden či více identifikátorů pro mapování CSS stylů.
 - c. Metoda **setHref()** je užitečná pro element typu **a** (zkratka je odvozena od anglického slova *Anchor*).
 - d. Metody **setName()** a **setValue()** jsou užitečné při tvorbě HTML formulářů, k tomu se dostaneme později.
 - e. Metoda **addParagraph()** vytvoří novou instanci elementu pro tvorbu odstavce.
 - f. Metoda **addHeading(text)** vytvoří nový nadpis první úrovně.
3. Třída **HtmlElement** reprezentuje kořen HTML stromu, poskytuje element hlavičky

i těla a obsahuje také konfiguraci, která ovlivňuje způsob zápisu kódu HTML stránky. V neposlední řadě metoda nabízí statické metody pro vytvoření výchozí instance třídy `HtmlElement`, která nám usnadní tvorbu HTML hlavičky v ukázkových příkladech.

Je užitečné vědět, že výchozí (defaultní) implementace je obálka nad třídou `PrintWriter`, a tak všechny texty zapsané do *elementu* se posílají rovnou do *writeru*. Metoda `close()` pak uzavře otevřené elementy a vyprázdní buffer. Výhodou takového řešení je dobrá performance a minimální paměťová náročnost při sestavování objemných stránek. Pro úplnost doplňuji, že chování lze ovlivnit změnou konfigurace při sestavení kořenového elementu. Pomocí takové konfigurace lze například změnit znaky pro odsazování vnořených elementů, zalamovat řádky, změnit národní lokalizaci obsahu či *znakovou sadu* obsahu HTML stránky. K dispozici je i podpora uživatelského formátování datových typů, výchozí implementace používá (pro zobrazení objektových hodnot) metodu `toString()`.

Podívejme se nyní, jak se změní implementace metody `doGet()`. Na první pohled lze pozorovat absenci textových *literálů* s HTML elementy. Všimněme si také drobné změny v importech tříd a také parametru anotace `@WebServlet` – dostal novou hodnotu pro odlišnou URL adresu servletu. Metoda také nepotřebuje deklaraci kontrolované výjimky.

Zdrojový kód 42. Třída `HelloWorldElement`

```
package net.ponec.jbook.s01_hello;

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.ujorm.tools.web.HtmlElement;
import org.ujorm.tools.web.Element;
import net.ponec.jbook.tools.WebTools;

/** An extended servlet implementation based on the Element model */
@WebServlet("/hello-element")
public class HelloWorldElement extends HttpServlet {
    /** Handles the HTTP GET method */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) {
        HtmlElement html = HtmlElement.niceOf(response, "css/world.css"); ❶
        Element body = html.getBody(); ❷
        body.addHeading("Hello, World! (by element)"); ❸
        WebTools.addFooter(body, this); ❹
        html.close(); ❺
    }
}
```

- ① Statická metoda `HtmlElement.niceOf()` vytvoří kořenový element HTML stránky a zároveň nastaví v konfiguraci požadavek na odsazování HTML elementů mezerami – podobně, jak jsme to prováděli zápisem mezer do zdrojového kódu v první implementaci.
- ② Metodou `getBody()` získáme objekt reprezentující HTML element **body**, ale použít lze i metodu s alternativním názvem `addBody()`.
- ③ Do objektu proměnné `body` zapíšeme požadovaný text nadpisu (*heading*).
- ④ Do elementu body vložíme ještě standardní patičku stránky, jako tomu bylo u první implementace.
- ⑤ Metodou `close()` uzavřeme model a stránku odešleme na internetový prohlížeč.

Moderní Java umožňuje automatické uzavírání zdrojů pomocí konstrukce `try-with-resources`. Tato konstrukce uzavírá automaticky všechny objekty zdrojů definované v hlavičce poté, co program opustí příslušný blok. Aby to bylo možné, musí tyto objekty implementovat rozhraní `AutoCloseable`. Kód pak vypadá čistěji:

Zdrojový kód 43. Ukázka automatického uzavírání zdrojů

```
try (HtmlElement html = HtmlElement.niceOf(response, "css/world.css")) { ①
    html.getBody().addHeading("Hello, World (final)"); ②
    WebTools.addFooter(html, this);
} ③
```

- ① Nastavení proměnné `html` se přesune do kulatých závorek bloku `try`.
- ② Nadpis stránky zde zapíšeme technikou řetězení metod objektů.
- ③ Konec bloku `try` volá metodu `close()` automaticky (na objektu deklarovaném v hlavičce bloku).

Do bloku `try` lze vkládat také vnořené elementy, a protože taková forma odsazování programového kódu připomíná **formátovaný** HTML kód, v dalších příkladech budeme preferovat právě tohle řešení.

Připojení knihovny do projektu

V Maven projektu stačí přidat závislost do souboru `pom.xml`:

Zdrojový kód 44. Připojení knihovny do Maven projektu

```
<dependency>
  <groupId>org.ujorm</groupId>
  <artifactId>ujo-tools</artifactId>
  <version>2.28</version>
</dependency>
```


Element `dependency` XML souboru deklaruje závislost na externích knihovnách. Taková závislost obsahuje tři části:

1. `groupId` – identifikátor skupiny projektů v rámci *Maven* repozitáře,
2. `artifactId` – identifikátor knihovny v rámci skupiny,
3. `version` – identifikátor verze.

Pohledem do souboru `pom.xml` zjistíme, že náš projekt má i další závislosti; jednou z nich je knihovna `javax.servlet-api` poskytující popis `servletů` (*API*) pro kompilaci projektu, jejich skutečnou implementaci však poskytuje až webový kontejner `Jetty` za běhu programu.

Závěrem provedme ve zdrojovém kódu vlastní změny: pod nadpis dopíšme nějaký vlastní text. Můžeme k tomu použít metodu `html.getBody().addText()`, kterou vložíme na nový řádek za nadpis (jako samostatný příkaz).

7.2.3 Programové výrazy

V této části využijeme `servlet` k zobrazení výsledků různých Java výrazů. Pro přehlednost použijeme pomocnou třídu `WebPrinter`, která k výsledku přidá i datový typ a číslování.

Zdrojový kód 45. Volání ukázkových výrazů

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) {
    try (HtmlElement html = HtmlElement.niceOf(response, "css/code.css")) { ❶
        Element body = html.getBody(); ❷
        try { ❸
            body.addHeading("Java expressions"); ❹
            printPrimitives(WebPrinter.of(body)); ❺
        } catch (Exception exception) { ❻
            html.getBody().addHeading(2,
                "A stack trace of exception"); ❼
            WebTools.writeStackTrace(html.getBody().addPreformatted(),
                exception); ❽
        }
        WebTools.addFooter(html, this);
    }
}
```

- ❶ Stránce přiřadíme CSS styl připravený pro tabulkové uspořádání dat.
- ❷ Získáme objekt elementu `body`.
- ❸ Založíme nový blok typu `try-catch`, který nám umožní zachytávat případné výjimky (vyhozené za běhu programu) za účelem jejich zobrazení v HTML stránce.

- ④ Zapišeme nadpis skupiny příkladů.
- ⑤ Pro zobrazení výrazů s primitivními datovými typy zavoláme metodu `printPrimitives()` s argumentem typu `WebPrinter`.
- ⑥ Pokud dojde k vyhození výjimky, zachytí ji blok `catch` a provedou se *příkazy* uvnitř tohoto bloku. V opačném případě bude zpracování pokračovat prvním *příkazem*, který za tímto blokem následuje. V našem případě to je tisk standardní patičky stránky.
- ⑦ Nadpis tematické skupiny bude mít druhou úroveň důležitosti.
- ⑧ Pro zobrazení výjimky využijeme statickou metodu, jejíž implementací se tady zabývat nebudeme.

Zkrácená ukázka **implementace** výrazů nám připomene základní operace s **primitivními** datovými typy. První argument metody `write()` obsahuje testovaný výraz (nebo proměnnou), druhý argument obsahuje jeho popis.

Zdrojový kód 46. Programové výrazy

```
private void printPrimitives(WebPrinter printer) {
    // ...
    printer.write(2 + 3L, "2 + 3L"); ①
    printer.write("2" + "3", "\"2\" + \"3\""); ②
    printer.write(2 * 3, "2 * 3"); ③
    printer.write(1 + 2 * 3, "1 + 2 * 3"); ④
    printer.write((1 + 2) * 3, "(1 + 2) * 3"); ⑤
    printer.write(7 / 3, "7 / 3"); ⑥
    printer.write(7 % 3, "7 % 3"); ⑦
    printer.write('A' + (char) 1, "'A' + (char) 1"); ⑧
    printer.write((char) ('A' + 1), "(char) ('A' + 1)"); ⑨
}
```

- ① Součet dvou primitivních čísel odlišných typů.
- ② Operátor `+` aplikovaný na dva objekty typu `String` vede k jejich spojení. Všimněme si, jak znaky uvozovek v textových *literálech* jsou uváděny zpětným lomítkem.
- ③ Ukázka násobení.
- ④ Priorita násobení před sčítáním.
- ⑤ Priorita vynucená závorkami.
- ⑥ Výsledek celočíselného dělení se nezaokrouhluje.
- ⑦ Ukázka výpočtu modulo (zbytku po dělení).
- ⑧ Výsledkem součtu typu `char` a `int` bude `int`.
- ⑨ Výraz typu `int` přetypovaný na `char` zobrazuje znak.

Zmíněný kód zobrazí v prohlížeči část tabulky, která začíná pořadovým číslem 16...

#	Expression/Statement	Result	Type
16.	2 + 3L	5	long
17.	"2" + "3"	23	java.lang.String
18.	2 * 3	6	int
19.	1 + 2 * 3	7	int
20.	(1 + 2) * 3	9	int
21.	7 / 3	2	int
22.	7 % 3	1	int
23.	'A' + (char) 1	66	int
24.	(char) ('A' + 1)	B	char

Obrázek 26. Zkrácená ukázka výsledku

První sloupec tabulky zobrazuje pořadové číslo výpisu, druhý výsledek výrazu, třetí sloupec výsledný datový typ a poslední sloupec obsahuje (zpravidla) zdrojový kód výrazu. Ve zdrojovém kódu můžete narazit na příkaz `printer.writeEmptyLine()`, jehož cílem je pouze grafické oddělení řádků. Všechny ukázky najdete v příložených příkladech, v případě nejasností kolem metod objektů doporučuji dokumentaci *JavaDoc* [5].

Řešenou úlohu můžeme doplňovat svými vlastními programovými výrazy podle libosti, prostor pro vlastní experimenty je značný.

7.3 Tvorba tabulek

Tabulkový design má v HTML široké využití, často se používá pro zarovnávání nejrůznějších grafických bloků na stránce, jak jsme si mohli všimnout v předchozí kapitole. V následujících kapitolách se zaměříme hlavně na využití HTML elementu `table`, je však dobré vědět, že tabulkového vzhledu lze dosáhnout i vhodným použitím CSS stylu v kombinaci s nějakým HTML elementem označujícím **blok**.

7.3.1 Jednoduchá tabulka

Dalším úkolem bude zobrazit v tabulkovém formátu výsledky, které mohou pocházet například z nějakého měření. Prezentovaná data nám bude poskytovat metoda, která bude vracet (pro začátek) konstanty zapsané ve zdrojovém kódu. Strukturu tabulky vyznačíme HTML elementem `table`, v rámci tohoto elementu se vyznačují řádky elementem `tr` (zkratka z anglického výrazu *table row*) a každý řádek má své buňky, které se vyznačují elementem `td` (z anglického výrazu *table detail*), a jejich hodnoty se zapisují jako text tohoto elementu. Protože chceme všechny hodnoty v tabulce zarovnat doprava, připojíme do HTML stránky CSS styl, který byl za tímto účelem už vytvořen. Zápis tabulky o dvou řádcích a třech

sloupcích by pak vypadal takto:

Zdrojový kód 47.

```
<table>
  <tr>
    <td>@</td>
    <td>A</td>
    <td>B</td>
  </tr>
  <tr>
    <td>1</td>
    <td>493</td>
    <td>180</td>
  </tr>
</table>
```

Začněme *metodou*, která bude **poskytovat data** tabulky. Protože tabulka je dvourozměrná, hodí se připravit data ve struktuře **dvourozměrného** pole objektů. Zkrácená ukázka kódu může vypadat takto...

Zdrojový kód 48. Metoda poskytující data

```
private Object[][] getTableModel() {
    Object[][] result = ❶
    {
        { "@", "A", "B", "C", "D", "E", "F", "G" } ❷
        , { "1", 493, 180, 103, 45, 317, 183, 67 }
        , { "2", 469, 5, 156, 98, 470, 434, 196 }
    };
    return result; ❸
}
```

- ❶ Vytvoříme **dvourozměrné pole objektů**, které uložíme do proměnné `result`. Protože třída `Objekt` je společným rodičem všech tříd, může pojmut objekty typu jak `String`, tak `Integer`. Plnou implementaci najdeme v příložených příkladech.
- ❷ Na místě prvků (vnějšího) pole se uvádí další (vnitřní) pole s prvky uzavřenými párem složených závorek `{}`. Taková vnitřní pole mohou mít obecně různé délky.
- ❸ Výsledek vrátíme *příkazem* `return`.

Příkaz for

Pro zpracování výsledku (této metody) budeme potřebovat programovou konstrukci na procházení prvků pole. V jazyce *Java* se využívá *iterační příkaz* `for` (česky *pro*). Jeho chování vysvětlíme na příkladu.

```
Object[] words = {"AA", "BB", "CC"};
for (int x = 0; x < words.length; x = x + 1) { ❶
    Object word = words[x]; ❷
}
```

- ❶ Součástí příkazu `for` je pár kulatých závorek, který obsahuje **tři** části oddělené **středníkem** podle vzoru: `(a;b;c)`.
 - a. V první části zakládá řídicí proměnnou `x` typu `int` s výchozí hodnotou nula.
 - b. Druhá část obsahuje podmínku, jejíž splnění je nezbytné pro vykonání **bloku příkazů**, vymezeného ve složených závorkách (za těmi kulatými závorkami). V tomhle případě porovnáváme hodnotu proměnné `x` s délkou pole, a pokud je hodnota proměnné menší, provede se blok *příkazů* uvedený ve **složených** závorkách `{}`.
 - c. Po dokončení **bloku příkazů** se vykoná *příkaz* uvedený ve třetí části příkazu `for`, v tomhle případě se obsah proměnné zvýší o jedničku a znovu se ověří platnost podmínky – pomocí výrazu uvedeného ve druhé části. Pokud je výsledkem hodnota `false`, iterace se ukončí a program pokračuje dalším *příkazem* za blokem `for`.
- ❷ Tělo bloku `for` má proměnnou `x` k dispozici, a tak ji lze využít rovnou jako **index** pro čtení **hodnot** pole. Protože naše pole má právě **tři prvky**, řídicí proměnná `x` nabývá postupně hodnot od 0 do 2. Při každé iteraci se načte hodnota jednoho prvku pole do proměnné `word`, a protože je typu `Object`, lze do ní zapisovat prakticky libovolný objekt či pole. Součástí bloku *příkazů* může být také **vnořený** cyklus, který může deklarovat svoji vlastní (novou) řídicí proměnnou (s jiným názvem). Pro předčasné ukončení iterací lze použít `break`, pro předčasný návrat do další iterace je k dispozici *příkaz* `continue`.

Inkrementace a dekrementace

Pamatujme, že *inkrementace* číselné proměnné primitivního typu provádí zvýšení hodnoty proměnné o jedničku, jedná se o zkrácený zápis *příkazu* typu `x = x + 1`. Analogicky se používá i *dekrementace* obsahující dva minusy (`x--`), která hodnotu proměnné o jedničku naopak snižuje. Inkrementace se používá s výhodou v iteračním *příkazu* `for`, kde operátor `++` (či `--`) lze zapisovat alternativně před názvem proměnné: pro nás je teď podstatné, že umístění operátoru nemá vliv na průchod cyklem.

Řešení úkolu

Vratme se k řešenému příkladu. Původní *servlet* na zobrazování nadpisu rozšíříme o generování tabulky. Při té příležitosti si ukážeme práci s dvourozměrným polem objektů. Už nebudeme popisovat celou třídu, zaměříme se na rozdíly.

Zdrojový kód 50. Implementace metody doGet()

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) { ❶
    try (HtmlElement html = HtmlElement.niceOf("The table",
        response, "css/table.css")) {
        html.getBody().addHeading(html.getTitle());
        // Print table: ❷
        Object[][] model = getTableModel(); ❸
        Element table = html.getBody().addElement("table")
            .setClass("numbers"); ❹
        for (int y = 0; y < model.length; y++) { ❺
            Object[] rowValues = model[y]; ❻
            Element rowElement = table.addTableRow(); ❼
            for (int x = 0; x < rowValues.length; x++) { ❽
                Object value = rowValues[x]; // model[y][x] ❾
                rowElement.addTableDetail().addText(value);
            } ❿
        } 11
        WebTools.addFooter(html, this);
    }
}
```

- ❶ Deklarace metody `doGet()` se od minulé kapitoly nezměnila.
- ❷ **Řádkový komentář**, který na běh programu nemá vliv.
- ❸ Data tabulky získáme voláním samostatné metody.
- ❹ Element `table` s názvem **CSS třídy**. Protože metoda `setClass()` vrací svůj objekt, přiřazení **CSS třídy** lze zřetězit.
- ❺ Projdeme prvky pole a zapíšeme do buněk tabulky. Průchod dvourozměrným polem provádíme dvěma **cykly**. První prochází řádky, druhý prochází sloupce.
- ❻ Do proměnné `rowValues` zapíšeme pole s prvky řádku `y`.
- ❼ Vytvoříme novou instanci elementu řádku HTML tabulky. Na konci smyčky se provede **inkrementace** proměnné a podmínka se znovu vyhodnotí. Po porušení platnosti podmínky se cyklus opustí a běh pokračuje prvním **příkazem** za za složenou závorkou cyklu.
- ❽ Ve vnořeném cyklu se celý postup opakuje. Teď procházíme prvky řádku (sloupce) tabulky.
- ❾ V prvním příkazu vnořeného cyklu získáme hodnotu prvku pole. Ke stejnému výsledku povede i výraz `model[y][x]`, vliv na rychlost provedení tady není zásadní.

10 Konec druhého (vnořeného) cyklu `for`.

11 Konec prvního cyklu `for`.



Věnujme pozornost podmínce ukončení smyčky: chyby tohoto typu zpravidla nevyhazují za běhu programu žádnou výjimku. Zpracovávání nekonečné smyčky se uživatelům projevuje zamrznutím programu (bez možnosti komfortního ukončení) při současně ztrátě výkonu celého počítače.

Po vykreslení naší HTML stránky v internetovém prohlížeči dostaneme na monitoru počítače následující obrázek:

@	A	B	C	D	E	F	G
1	493	180	103	45	317	183	67
2	469	5	156	98	470	434	196
3	73	383	413	221	156	340	79
4	324	211	41	302	26	426	115
5	123	303	250	301	416	487	315
6	326	493	16	11	346	240	280
7	468	390	497	36	242	469	174

Obrázek 27. Vykreslení HTML tabulky

Alternativní procházení pole

Pokud v těle cyklu nepotřebujeme pracovat s proměnnými (`x`, `y`), lze využít zkrácený zápis. Smyčka `for` pak obsahuje pouze dvě části oddělené dvojtečkou (`:`). Tělo metody pak bude pracovat s proměnnou definovanou v prvním cyklu.

Zdrojový kód 51.

```
for (Object[] rowValues : model) {  
    Element rowElement = table.addTableRow();  
    for (Object value : rowValues) {  
        // run body ...  
    }  
}
```

Podmíněný příkaz if-else

Při implementaci algoritmu potřebujeme často provést (jeden či více) *příkazů* pouze při splnění nějaké podmínky. Pro ten případ se používá podmíněný *příkaz if*. Následuje kulatá závorka s logickým výrazem, která podmiňuje provedení bloku v závorkách. Složené závorky lze vynechat v případě, že blok obsahuje právě jeden *příkaz*, tohle řešení se však kvůli horší přehlednosti spíše nedoporučuje. Za blokem *příkazů* může (ale nemusí) následovat klíčové slovo *else*, které uvádí další blok *příkazů* pro alternativní zpracování. Výrazy *if* a *else* lze s výhodou řetězit. Ukázka použití následuje, více příkladů si ukážeme v dalších kapitolách.

Zdrojový kód 52. Podmíněný příkaz

```
if (y == 0) { ❶  
    statementA();  
} else if (x == 0) { ❷  
    statementB();  
} else { ❸  
    statementC();  
}
```

- ❶ Podmíněný *příkaz* s podmínkou. Pokud je podmínka splněna, zavolá se metoda *statementA()*.
- ❷ Alternativní podmínka se vyhodnotí v případě, že předcházející podmínka nevyhověla. Tuhle formu zápisu lze po skončení bloku opakovat pro podmínky další.
- ❸ Do posledního bloku spadnou všechny případy, které nezachytí žádná z předchozích podmínek.

Tabulka náhodných čísel

Na posledním servletu jsme si ukázali procházení prvků vícerozměrného pole. V dalších příkladech však budeme využívat spíše metodu třídy *Element* zvanou *addTable()*, která popsaný algoritmus už obsahuje. Stačí jí předat referenci na dvourozměrné pole, metoda sestaví HTML model tabulky sama.

Zdrojový kód 53. Zjednodušené sestavení tabulky

```
html.getBody().addTable(getTableModel(8), "numbers");
```

Teď zkusme původní metodu (poskytující dvourozměrné pole) upravit tak, aby vracela (dvourozměrné) pole **náhodně** generovaných čísel v intervalu od 0 do 500. První řádek by mohl obsahovat nějaký popis, stejně tak i první sloupec. Protože se bude jednat o čtvercovou tabulku, počet řádků a sloupců bude shodný. Jejich

hodnotu dostane metoda prostřednictvím svého argumentu. Podívejme se na řešení s komentářem.

Zdrojový kód 54. Metoda pro tvorbu dvourozměrného pole objektů

```
/** Create a random number array ❶  
 * @param size Number of cells per a square table side (including labels)  
 * @return An object array  
 */  
private Object[][] getTableModel(int size) { ❷  
    Object[][] result = new Object[size][size]; ❸  
    Random random = new Random(); ❹  
    for (int y = 0; y < size; y++) { ❺  
        for (int x = 0; x < size; x++) {  
            if (y == 0) { ❻  
                result[y][x] = (char) ('A' + x - 1); ❼  
            } else if (x == 0) { ❽  
                result[y][x] = y;  
            } else {  
                result[y][x] = random.nextInt(500); ❾  
            }  
        }  
    }  
    return result;  
}
```

- ❶ Dokumentace ve formátu *JavaDoc*. Pomocí značek `@param` a `@return` popisujeme vstupní parametr a návratovou hodnotu.
- ❷ Metoda `getTableModel()` očekává parametr s názvem `size` primitivního typu `int` a vrátí dvourozměrné pole objektů.
- ❸ Vytvoříme dvourozměrné pole velikosti `size` (z parametru metody), kde každý prvek má hodnotu `null`. Pole však bude třeba naplnit hodnotami.
- ❹ Pro generování náhodných čísel vytvoříme objekt typu `Random`, což je třída ze standardní *Java* knihovny.
- ❺ Ve vnořených cyklech projdeme všechny prvky dvourozměrného pole a zapíšeme hodnoty.
- ❻ Uvnitř vnořeného cyklu máme k dispozici souřadnice `x` a `y`. Do prvního sloupce chceme zapisovat popis řádku, a k tomu využijeme podmíněný *příkaz*. Logická podmínka se zapisuje do kulaté závorky za klíčové slovo `if`. Pokud je podmínka pravdivá, provedou se všechny *příkazy* uvnitř složené závorky. Za složenou závorkou podmínky `if` může následovat klíčové slovo `else` se složenou závorkou obsahující *příkazy*, které se naopak provádějí, když je podmínka neplatná. Příkazy `else` a `if` lze řetězit.
- ❼ Při zápisu prvního řádku zapíšeme postupně znaky tabulkové sady `Unicode` od znaku `A`, protože však první sloupec data neobsahuje, přiřadíme mu znak, který písmenu `A` předchází. Pokud sečteme čísla (rozdílných) typů `char` a `int`, výsledek

bude mít typ s větším rozsahem. Protože do buňky však chceme zapsat objekt typu `Character`, je třeba číslo nejdříve přetypovat pomocí cílového datového typu v závorce (`char`). Teprve potom může *Java* převést primitivní typ na objekt typu `Character` – díky vlastnosti `autoboxing`. Tento zápis se provádí jen pro **první řádek** tabulky – tedy pokud proměnná `y` se rovná `0`.

- ⑧ Jinak otestujeme, zdali se nejedná o zápis do **prvního sloupce**, který poznáme porovnáním proměnné `x` s hodnotu `0`. V tom případě do buňky zapíšeme pořadové číslo řádku.
- ⑨ Jinak do buňky zapíšeme náhodné číslo podle zadání, k tomu využijeme metodu `nextInt()` objektu typu `Random`. Hodnota argumentu reprezentuje horní limit.

Výsledek si můžeme ověřit na spouštěném projektu. Při každém novém otevření stránky se zobrazí jiná čísla.

Závěrem provedme ve zdrojovém kódu vlastní změny: nahraďme náhodná čísla tabulky náhodně generovanými slovy. Pro začátek k tomu můžeme využít pole (s prvky typu `String`) obsahující předem připravený seznam slov, později můžeme zkusit generovat i obsah těchto slov.

7.3.2 Přehled vozidel

V kapitole [Dědičnost tříd](#) jsme si popsali diagram tříd na příkladu **evidence vozidel**. Úkolem bude zavolat službu poskytující záznamy o vozidlech, které vypíšeme do HTML tabulky. Třídy popisující *datový model* (`Car`, `Company`, `RoadVehicle`) nazýváme *doménové objekty*. Při návrhu tříd bývá zvykem datovou část oddělit do *doménových* objektů a algoritmy umístit spíše do tříd *servisního* typu.

Implementace setterů a getterů v doménových objektech je rutinní práce, kterou zvládne vygenerovat většina vývojových editorů IDE. Ukázky reálných doménových objektů lze najít v příložených příkladech. Zajímavější bude sestavení datového modelu tabulky, který implementujeme do metody `getTableModel()`. Popis následuje...

Zdrojový kód 55. Metoda pro tvorbu dvourozměrného pole

```
/**
 * A dynamic number array
 * @param size Number of cells per a square table side (including labels)
 * @return A result
 */
private List<Object[]> getTableModel() { ①
    List<Object[]> result = new ArrayList<>();
    Object[] header = { "Order", "Manufacturer", "Model",
                        "Made", "Serial number", "Power [kW]",
                        "Energy", "Trunk [l]", "Owner" };
}
```

```

    result.add(header);
    for (Car car : vehicleService.getFictionalCars()) {
        String ownerName = car.getOwner().getFirstname() ❷
            + " "
            + car.getOwner().getSurname();
        Object[] row = { result.size() ❸
            , car.getModel().getManufacturer().getName()
            , car.getModel().getName()
            , car.getMade()
            , car.getSerialNumber()
            , car.getEngine().getPower()
            , car.getEngine().getFuelName()
            , car.getTrunkVolume()
            , ownerName
            };
        result.add(row);
    }
    return result;
}

```

- ❶ Návrátová hodnota je typu `List<Object[]>`. Jak už jsme zmínili v kapitole [List](#). Typ prvků se deklaruje pomocí *generického* typu, který se uvádí ve špičatých závorkách, my zde budeme potřebovat seznam **jednorozměrných polí**. Na prvním řádku metody jsme získali prázdný návratový objekt, který zbývá naplnit daty.
- ❷ Připomeňme spojování textových objektů operátorem `+`.
- ❸ Pro každý prvek seznamu vytvoříme pole objektů, které vložíme do objektu typu `List` metodou `add()`. Počet zapsaných prvků listu lze zjistit metodou `size()`. Na rozdíl od pole přistupujeme k objektu typu `List` výhradně pomocí jeho *metod*.

Výsledná HTML stránka vykreslená v internetovém prohlížeči může vypadat takto:

Order	Manufacturer	Model	Created	Serial number	Power [kW]	Energy	Trunk [l]	Owner
1	Škoda Auto	Fabia	2005-06-01	123-654-987	47	benzin	260	Jiří Mexedolid
2	Škoda Auto	Rapid	2013-01-15	359-985-785	81	benzin	550	Lucie Zixerfxodf
3	Toyota	Auris	2010-10-22	987-654-123	85	benzin	354	Donald Choresisdi
4	Ford	Focus	2015-03-30	958-232-497	77	benzin	375	Donald Choresisdi

Obrázek 28. Přehled registrovaných vozidel

Závěrem provedme ve zdrojovém kódu vlastní změny: změňte pořadí sloupců tabulky, výkon motoru zkuste přepočítat na jednotky v **koních** [HP].

7.4 Formuláře

Do webové stránky přidejme grafické komponenty pro datové vstupy z klávesnice.

7.4.1 Formulář s jedním vstupním elementem

Začneme jednoduchým úkolem: uživateli nabídneme jednoduchý formulář s jednou položkou pro zápis textu. Formulář budeme na server odesílat tlačítkem formuláře. Server si textovou zprávu přečte a vrátí odpověď v podobně nové stránky. Následuje ukázka zápisu jednoduchého formuláře v jazyce HTML, která se bude nacházet někde uvnitř HTML elementu `body`...

Zdrojový kód 56. HTML kód formuláře (elementu form)

```
<form method="post" ①  
  action="/" ②  
  <label>Some text</label> ③  
  <input type="text" name="text" value="Enter some text"/> ④  
  <input type="submit" value="Submit"/> ⑤  
</form>
```

- ① Element `form` obsahuje grafické komponenty umožňující interakci uživatele, jejich hodnoty se budou posílat společně na server. Hodnota atributu `method` určuje **metodu**, kterou se data posílají na server. Metod je více, my budeme používat jen dvě následující:
 - a. `get` – formulářová data se odesílají jako parametry URL adresy a jejich hodnoty vidí uživatel v adresním řádku internetového prohlížeče; parametry tak mohou být ukládány do záložek či historie prohlížení. Metoda je vhodná spíše pro krátké zprávy bez důvěrných informací. Takovou zprávu `servlet` zpracuje metodou `doGet()`,
 - b. `post` – data budou obsažena v těle HTTP dotazu, a tak je uživatel v adresním řádku prohlížeče nevidí. Metoda je vhodná pro posílání **důvěrných** informací (například hesla), přitom zvládá i delší zprávy s diakritikou. Zprávu `servlet` zpracuje metodou `doPost()`.
- ② HTML formulář obsahuje další důležitý atribut s názvem `action`, který popisuje URL cílové stránky. V našem případě to je URL `servletu`. Pokud je cílem stejná adresa, ze které byla stránka načtena, jeho hodnotou může být *otazník* používaný pro oddělování URL parametrů.
- ③ Element `label` obsahuje jmenovku vstupního pole.
- ④ Element `input` typu `text` zobrazuje grafickou komponentu, do které může uživatel zapisovat zprávu. Atribut `name` obsahuje *název parametru*, který použijeme pro získání zprávy na `servletu`, atribut `value` může obsahovat

předvyplněný text vstupního pole.

- ⑤ Element `input` typu `submit` zobrazí tlačítko, které po kliknutí myší odešle na server všechny hodnoty vstupních elementů, jež se nacházejí **uvnitř** formuláře.

Následující implementace *servletu* už neobsahuje definici *balíčku*, *importy*, ani *JavaDoc*. Celý kód najdeme v příloženém projektu.

Zdrojový kód 57. Stavba formuláře

```
@WebServlet("/simple-form")
public class SimpleFormServlet extends HttpServlet {
    private static final Charset CHARSET = StandardCharsets.UTF_8; ①
    private static final String PARAMETER_NAME = "text";

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) {
        request.setCharacterEncoding(CHARSET.toString());
        try (HtmlElement html = HtmlElement.niceOf("Simple form",
            response, CHARSET, "css/form.css")) { ②
            html.getBody().addHeading(html.getTitle());
            Element form = html.getBody().addForm()
                .setMethod("post")
                .setAction(request.getRequestURI());
            try (Element line = form.addDiv()) { ③
                line.addLabel().addText("Some text");
                line.addTextInput()
                    .setName(PARAMETER_NAME)
                    .setValue(request.getParameter(PARAMETER_NAME)); ④
            }
            form.addInput()
                .setType(Html.V_SUBMIT)
                .setValue("Submit");
            WebTools.addFooter(html, this);
        }
    }

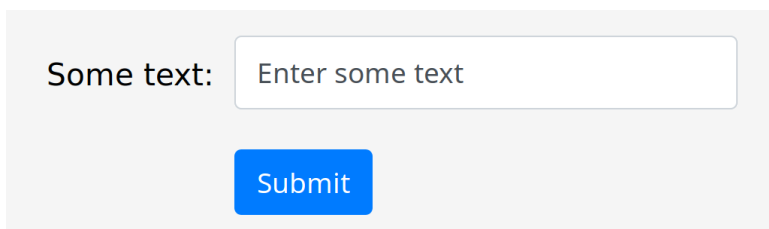
    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) ⑤
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

- ① Třída obsahuje dva statické atributy, které bývají označovány klíčovým slovem `final` pro garanci neměnnosti hodnoty za běhu programu. Takovým atributům se říká *konstanty*. Podle konvence se statické atributy třídy zapisují **velkými písmeny**.

- První atribut obsahuje *znakovou sadu* HTML stránky i přijatých parametrů.
- Druhý atribut obsahuje název parametru, pod kterým očekáváme textovou zprávu.

- ② V těle metody `doGet()` připravíme kořenový element metodou `niceOf()` s parametrem kódové stránky, který použije internetový prohlížeč při zobrazování znaků. Výchozí kódová stránka (uložená v parametrech kořenového elementu) je `UTF_8`.
- ③ Na rozdíl od ukázky HTML kódu, elementy `label` a `input` vložíme do neviditelného rámečku vytvořeného elementem `div` pro lepší formátování výsledné HTML stránky.
- ④ Důležitá metoda pro získání hodnoty parametru z objektu `request` se jmenuje `getParameter()`. Získanou hodnotu odešleme do elementu metodou `setValue()`.
- ⑤ Protože formulář odesíláme metodou `post`, zprávu je třeba zachytit v servletu metodou `doPost()`. Protože se však zpracování neliší od zpráv zasílaných metodou `get`, v metodě jednoduše zavoláme metodu `doGet()` se stejnými parametry. Díky takovému řešení bude HTML stránka reagovat na parametry zaslané v URL (skoro) stejně jako na parametry odeslané z grafického formuláře metodou `post`.

Výsledný formulář v prohlížeči může vypadat takto:



The image shows a web form with a light gray background. On the left, the text "Some text:" is displayed. To its right is a white rectangular text input field with a thin gray border, containing the placeholder text "Enter some text". Below the input field is a blue rectangular button with the word "Submit" in white text.

Obrázek 29. Formulář s jednou textovou položkou

7.4.2 Formulář s více vstupními elementy

Teď vytvoříme formulář pro sběr osobních údajů. Platnost zapsaných dat budeme ověřovat pomocí *regulárních výrazů*. Chybové zprávy budeme zobrazovat pod každým vstupním políčkem. Popis dalších *HTML* elementů lze studovat třeba tady [20].

Jmenovka	Anglicky	Podmínka	Typ
Jméno	First name	2–30 libovolných znaků	text
Příjmení	Last name	2–30 libovolných znaků	text
E-mail	E-mail	tři skupiny malých abecedních znaků bez diakritiky oddělené zavináčem @ a tečkou. První dvě skupiny tolerují číslice, tečku a pomlčku, délka poslední skupiny je v rozmezí 2–4 znaky (včetně)	text

Jmenovka	Anglicky	Podmínka	Typ
Telefon	Phone number	9–15 číslic, tolerovány jsou mezery a znaky plus (+)	text
Přezdívka	Nickname	3–10 znaků, tolerovány jsou mezery a znaky plus (+)	text
Odeslat	Submit	tlačítko pro odeslání nepotřebuje podmínku	submit

Regulární výrazy

Regulární výraz je *textový řetězec* sestavený podle určitých pravidel pro porovnávání, prohledávání či záměnu textů pomocí *zástupných výrazů*. Fungování budeme demonstrovat na anglickém citátu britské spisovatelky:

As is a tale, so is life: not how long it is, but how good it is, is what matters.

— J. K. Rowling

Což znamená: Život je jako příběh: nezáleží na tom, jak je dlouhý, ale jak je dobrý. Mějme tento anglický citát v objektu typu `String` a zkusme v něm vyhledat text o čtyřech znacích, který začíná písmenem `l` a končí `e`. Pro popis hledaného výrazu by se nám hodil znak připomínající žolíka, který v některých karetních hrách nahrazuje kartu libovolné hodnoty. Regulární výraz nabízí něco podobného, je však třeba znát pravidla jeho použití. **Běžné** znaky jsou malá či velká písmena abecedy a číslice. Nejdůležitější znaky **speciálního** významu jsou popsány tady:

1. `.` (tečka) je výraz reprezentující jeden libovolný znak. Regulární výraz pro hledání výše popsaného požadavku je `l..e`. Pro vyhledání stačí zavolat vhodnou metodu s parametry bez nutnosti složitého programování.
2. `\` (zpětné lomítko) ruší speciální význam znaků a naopak – z některých běžných znaků dělá speciální. Pokud tedy chceme v textu popsat tečku, je třeba sestavit regulární výraz: `\.`



Připomeňme, že zpětné lomítko je speciální znak také v jazyce *Java*, a proto ve *zdrojovém kódu* je nutné ho zdvojit dle vzoru: `"\\."`. Další výrazy budou už zapisovány jako *textové literály*.

3. `[]` (hranaté závorky) ohraničují seznam povolených znaků. Výraz `"[Ll]ife"` reprezentuje slovo bez ohledu na velikost prvního písmene. Znak pomlčky vymezuje interval, a tak výraz `"[A-Z]"` reprezentuje jeden znak velké abecedy (bez diakritiky) a výraz `"[0-9]"` reprezentuje číslici.
4. `*` (hvězdička) označuje **nula a více** opakování předcházejícího znaku (případně výrazu). Regulární výraz `"l.*e"` má shodu se slovy: **lake, lie, le**.

5. **+** (plus) značí **jedno a více** opakování předcházejícího znaku (případně výrazu).
6. **{ }** (složené závorky) s číslem vyjadřují počet opakování předcházejícího znaku (či výrazu). Vložením pomlčky lze vyjádřit také interval podle vzoru: **"{1-3}"**.
7. **()** (kulaté závorky) slouží k vyznačování skupin, oddělovačem je znak **|**. Například výraz **"(tale|life)"** reprezentuje jedno ze slov **tale** či **life**. Pro úplnost doplňme, že kulaté závorky mají své uplatnění také při **záměně** textů.
8. **^** a **\$** – znaky symbolizují počátek a konec textu na řádku. Uvedením obou symbolů ve výrazu lze kontrolovat obsah celého textu proti požadovanému vzoru. Uvedený citát má shodu s regulárním výrazem **"^A.*\\\$. \$"**.

Speciálních znaků je celá řada, znaky je možné v seznamech i vylučovat. Dobrá zpráva je, že regulární výrazy se dají využít v celé řadě oblastí mimo jazyk *Java*, na druhou stranu i regulární výrazy mají své meze a na některé specifické potřeby už nestačí. Podrobnější popis lze najít také v *JavaDoc* [5] u třídy *Pattern*.

Formulář osobních údajů

Vratme se zpět k implementaci našeho formuláře. Na rozdíl od minulého řešení si pro popis **editačního pole** připravíme **objektový model**, jeho třídu pojmenujme třeba *FieldDescriptor*. Tento model bude mít **čtyři** atributy, jejichž hodnoty budou neměnné po celou dobu životnosti objektu. Takovou vlastnost lze garantovat klíčovým slovem **final**, které se zapisuje před datový typ atributu. Hodnoty pak budeme vkládat do objektu pomocí *konstruktoru*. Podívejme se na ukázkou.

Zdrojový kód 58. Atributy objektu zapisované konstruktorem

```
public class FieldDescriptor {  
    private final String label; ①  
    private final String name; ②  
    private final String regexp; ③  
    private final boolean submit; ④  
  
    public FieldDescriptor(String label, String name,  
        String regexp, boolean submit) { ⑤  
        this.label = label; ⑥  
        this.name = name;  
        this.regexp = regexp;  
        this.submit = submit;  
    }  
    public FieldDescriptor(String label, String key, String regexp) { ⑦  
        this(label, key, regexp, false); ⑧  
    }  
    public String getValue(HttpServletRequest request) { ⑨  
        if (submit) {  
            return "Submit"; ⑩  
        } else {  
            return request.getParameter(name); ⑪  
        }  
    }  
}
```


- ① Jmenovka editovaného pole. Díky klíčovému slovu **final** bude pokus o další zápis do atributu odepřen už při kompilaci zdrojového kódu.
- ② Název *input* elementu na HTML stránce.
- ③ Regulární výraz pro kontrolu platnosti.
- ④ Příznak, že se jedná o odesílací **tlačítko**.
- ⑤ Seznam parametrů konstruktoru se odděluje čárkou, podobně jako je tomu u parametrů metod.
- ⑥ Pokud se název argumentu shoduje s atributem, je třeba atribut odlišit předřazením klíčového slova **this** odděleným tečkou.
- ⑦ Třída může mít několik konstruktů, pokud se liší počtem parametrů či typem. Této konstrukci se říká *přetížený konstruktor*.
- ⑧ V těle konstrukturu doplníme výchozí hodnotu argumentu **submit** a pomocí klíčového slova **this** delegujeme vytvoření objektu na jiný konstruktor. Pomocí klíčového slova **super()** lze volat také konstruktor rodičovské třídy.
- ⑨ Metoda bude poskytovat **hodnotu** parametru z objektu **HttpServletRequest**.
- ⑩ V případě **tlačítka** budeme vracet jeho jmenovku.
- ⑪ Pro ostatní položky vrátíme výsledek metody **getParameter()**, kde argumentem bude **jméno HTML elementu** z editačního formuláře.
- ⑫ Atributy zpřístupníme sadou veřejných *getterů* (*setterů* zde nemají význam).

Teď už máme připravenou třídu pro modelování položek formuláře a připravíme si metodu, která objektový model sestaví. Podívejme se na použití metody **getFieldDescriptors()**...

Zdrojový kód 59. Sestavení modelu formuláře

```
private FieldDescriptor[] getFieldDescriptors() {
    FieldDescriptor[] result = ①
    { new FieldDescriptor("First name", "firstname", ".{2,30}") ②
    , new FieldDescriptor("Last name", "lastname", ".{2,30}")
    , new FieldDescriptor("E-mail", "email",
        "[a-z0-9.-]+@[a-z0-9.-]+\.[a-z]{2,4}") ③
    , new FieldDescriptor("Phone number", "phone", "[+ 0-9]{9,15}")
    , new FieldDescriptor("Nickname", "nick", ".{3,10}")
    , new FieldDescriptor("", "submit", ".*", true)}; ④
    return result;
}
```

- ① Do proměnné **result** zapíšeme pole objektů typu **FieldDescriptor**.

- ② Vytvoříme objekt pomocí konstruktoru se **třemi** parametry. Posledním argumentem konstruktoru je *regulární výraz*, který očekává 2 až 30 libovolných znaků. Protože *regulární výrazy* budeme hodnotit metodou `matches()`, symboly počátku a konce textu jsou zde nadbytečné. Atribut `submit` dostává v tomto konstruktoru hodnotu `false`.
- ③ Popis e-mailového formátu: očekává se alespoň jeden **znak** z první skupiny (v hranatých závorkách), dále znak **zavináč** (`@`), alespoň jeden **znak** z druhé skupiny, **tečka** (`.`) a dva až čtyři **znaky** z poslední skupiny. Vzor reálných emailů by mohl být více tolerantní k použití velkých písmen a k doménám s délkou přesahující 4 znaky.
- ④ Na tomto řádku využijeme konstruktor se čtyřmi parametry. Jmenovka bude prázdná a hodnota odesílacího tlačítka by měla být vždy platná.

Popišme si metodu `doGet()`, která sestaví výsledný HTML model, a připomeňme, že metoda `doPost()` deleguje své zpracování právě sem...

Zdrojový kód 60. Formulář metody GET

```
@Override
protected void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
    try (HtmlElement html = HtmlElement.niceOf("Simple user form",
        response, CHARSET, "css/form2.css")) {
        html.getBody().addHeading(html.getTitle());
        Element form = html.getBody().addForm()
            .setMethod(Html.V_POST) ①
            .setAction(request.getRequestURI()); ②
        for (FieldDescriptor field : getFieldDescriptors()) { ③
            createInputField(field, form, request, postMethod); ④
        }
        WebTools.addFooter(html, this);
    }
}
```

- ① Pro zápis hodnoty `post` začneme využívat statickou konstantu, což omezuje některé překlady.
- ② Atribut formulářového elementu `action` bude obsahovat hodnotu `request.getRequestURI()`, což je řešení pro odeslání parametrů formuláře na ten stejný servlet. Atribut umožňuje odeslat parametry formuláře i na jiný servlet.
- ③ V cyklu projdeme postupně každý model z pole, jejichž seznam nám poskytne metoda `getFieldDescriptors()`.
- ④ Potřebný HTML model sestavíme v metodě `createInputField()` se čtyřmi argumenty, kterou si popíšeme zvlášť.

```
private void createInputField(FieldDescriptor field,
    Element form, HttpServletRequest request, boolean postMethod) {
    String cssStyle = field.isSubmit() ? "submit" : ""; ①
    String inputId = field.getName() + "Id";
    Element result = form.addDiv(cssStyle); ②
    result.addLabel().setFor(inputId) ③
        .addText(field.getLabel());
    Element inputRow = result.addDiv();
    inputRow.addInput()
        .setType(field.isSubmit() ? "submit" : "text") ④
        .setName(field.getName())
        .setValue(field.getValue(request))
        .setId(inputId);
    String errorMessage = getErrorMessage(request, field); ⑤
    if (errorMessage != null) {
        inputRow.addSpan().addText(errorMessage);
    }
}
```

- ① Výraz s otazníkem se jmenuje **ternární operátor** a jeho použití si vysvětlíme za chvíli. Prázdný text ("") nevytvoří v modelu žádný CSS styl.
- ② Příkaz vloží do formuláře nový neviditelný rámeček, který bude obsahovat **jmenovku**, vstupní **pole** a případně chybovou **zprávu**. Hodnota přiřazeného CSS stylu závisí na typu vstupního pole.
- ③ Atribut **for** v elementu **label** slouží k propojení jmenovky s konkrétním vstupním polem pomocí jeho unikátního identifikátoru **id**. Jeho hodnotu máme připravenou v *Java* proměnné **inputId**. Kliknutí na text jmenovky pak automaticky aktivuje (zaměří) přidružené vstupní pole.
- ④ Vstupní komponentě přiřadíme postupně: **typ**, **jméno**, **hodnotu** a zmíněný **identifikátor**.
- ⑤ Vyhodnotíme **platnost** hodnoty pole z HTTP parametru. Pokud je hodnota neplatná, metoda vrátí chybovou zprávu určenou uživateli, jinak vrátí **null**. Obsahem chybové zprávy se řídí její zápis do formuláře.

Ternární operátor

Příkaz typu **if-else** umožňuje vytvářet bloky příkazů, které se vykonají při splnění nějaké podmínky. Někdy se nám však hodí stručnější forma podmíněného výrazu, který umí vrátit jedinou hodnotu. Takovému výrazu se říká **ternární operátor** a skládá se ze tří částí...

Zdrojový kód 62. Ternární operátor

```
field.isSubmit() ? "submit" : ""
```

Výraz před otazníkem představuje řídicí podmínku. Za otazníkem jsou pak ještě dva výrazy **stejného typu** oddělené dvojtečkou (:). Pokud je řídicí podmínka pravdivá, použije se **první** z nich, jinak ten **druhý** výraz. Hodnotou může být také `null`.

Ternární operátor jsme využili také při zápisu **typu** `input` elementu. Pokud vytváříme tlačítko, použijeme hodnotu `"submit"`, v opačném případě zapíšeme hodnotu `"text"`. Příště budeme brát obě hodnoty raději z konstant (`Html.V_SUBMIT` a `Html.V_TEXT`). Závěrem kontrolujeme platnost vstupního parametru. Pokud se chybová zpráva liší od hodnoty `null`, pak tuto zprávu zapíšeme do stromu elementů pomocí HTML značky `span`.

Hodnocení platnosti textů

Teď si ukážeme, jak se pomocí *regulárních výrazů* vyhodnocuje platnost textových hodnot ve zdrojovém kódu jazyka *Java*. Řešení je obsažené v metodě, která sestavuje chybovou zprávu pro uživatele.

Zdrojový kód 63. Sestavení chybové zprávy

```
public String getErrorMessage(HttpServletRequest request, FieldDescriptor field) {
    boolean postMethod = "post".equalsIgnoreCase(request.getMethod()); ❶
    if (postMethod) { ❷
        final String value = field.getValue(request); ❸
        if (value == null || value.isEmpty()) { ❹
            return "Required field";
        } else if (!Pattern.matches(field.getRegexp(), value)) { ❺
            return "Wrong value for: " + field.getRegexp(); // Localize it!
        }
    }
    return null;
}
```

- ❶ Zjistíme typ HTTP metody, kterou byl požadavek z prohlížeče odeslán.
- ❷ Platnost hodnoty chceme kontrolovat jen u požadavku zasláného metodou `post`. Dále bychom mohli vyloučit i kontrolu hodnoty potvrzovacího tlačítka, protože nemá význam.
- ❸ Do proměnné `value` zapíšeme hodnotu parametru.
- ❹ Pokud je text prázdný či nedefinovaný, upozorníme na chybějící data.
- ❺ Jinak zkontrolujeme shodu textu s regulárním výrazem metodou `matches()` (česky *odpovídá*). Pokud hodnota **neodpovídá** očekávání, sestavíme zprávu doplněnou použitým regulárním výrazem. Běžní uživatelé však ocení spíše popisnější komentář.

Tak, máme hotovo a můžeme si prohlédnout výsledek po vložení testovacích dat...

Obrázek 30. Formulář s neplatnými vstupy

Na internetu lze najít tipy na sestavení nejrůznějších *regulárních výrazů* – nejen pro kontrolu e-mailu. Chybové zprávy reálných aplikací by měly být uživatelsky více přívětivé.

Závěrem provedme ve zdrojovém kódu vlastní změny: doplňme do formuláře nový vstup pro osobní číslo uživatele, kde budou povoleny jen číselné znaky. Dále zkusme do formuláře zpracovat souhlas se zpracováním osobních údajů pomocí komponenty typu `checkbox`.

7.4.3 Počítání slov

V dalším příkladu si ukážeme počítání znaků a slov v textu pro statistické účely. Za **slovo** můžeme považovat každou skupinu znaků oddělenou mezerou, novým řádkem či vzájemnou kombinací takových znaků v libovolném počtu. Připomeňme, že znakům pro oddělování slov se říká *bílé znaky* a do této skupiny patří také různé mezery a tabulátory. Na výsledné HTML stránce bude možné pozorovat, jak zásadní vliv může mít nastavení CSS třídy na umístění jmenovky. Původní *element* pro vkládání textu nahradíme novým elementem `textarea`, který umožňuje zápis více řádků ve formuláři. Výsledek zobrazíme v tabulkovém formátu. Datový model nám poskytne metoda, která obdrží požadovaný text v parametru.

Zdrojový kód 64.

```
private Object[][] getGridModel(String message) {
    if (message == null) {
        message = ""; ❶
    }
    final Object[][] result =
        { {" ", "Count", "Unit"}
        , {"Word count:", countWords(message), "words" } ❷
        , {"Non-white characters:",
            countNonWhiteCharacters(message), "characters" }
        , {"Message length:", message.length(), "characters" } ❸
        };
    return result;
}
```

- 1 Pokud zpráva obsahuje hodnotu `null`, nahradíme ji prázdným textem, abychom zabránili výjimce při dalším zpracování. Na tomto řešení lze pozorovat, že k parametrům metody můžeme přistupovat podobně jako k běžným proměnným.
- 2 Sestavit dvourozměrné pole už umíme, hodnotu prvku lze získat samozřejmě libovolným výrazem – včetně volání metody.
- 3 Připomeňme, že počet znaků objektu `String` poskytuje metoda `message.length()`.

Podívejme se na implementaci statistických výpočtů...

Zdrojový kód 65. Počítání slov

```
private int countWords(String message) {
    int result = 0; ①
    boolean lastWhitespace = true; ②
    for (int i = 0; i < message.length(); i++) { ③
        boolean currentWhitespace
            = Character.isWhitespace(message.charAt(i)); ④
        if (lastWhitespace != currentWhitespace) { ⑤
            lastWhitespace = currentWhitespace;
            if (!currentWhitespace) { ⑥
                ++result;
            }
        }
    }
    return result;
}
```

- 1 Počet nalezených slov budeme ukládat do proměnné `result`.
- 2 V textu budeme sledovat dva typy znaků – pro tvorbu **slov a mezer**. Připravíme si proměnnou pro informaci o tom, jakého typu byl **poslední** zpracovaný znak.
- 3 V cyklu projdeme postupně každý znak parametru `message`, řídící proměnná `i` zde reprezentuje zároveň *index* aktuálního znaku.
- 4 Znak parametru (na dané pozici) získáme metodou `charAt(i)`. Pro kontrolu mezery (přesněji *bílého znaku*) využijeme statickou metodu `Character.isWhitespace()` ze standardní knihovny `Java`.
- 5 Pokud dojde ke změně typu znaku, změníme obsah logické proměnné `lastWhitespace`.
- 6 Pokud aktuální znak **není** bílým znakem (a předchozí byl), přičteme k výsledku jedničku. Připomeňme si, že negace logického typu se provádí operátorem `!` (vykřičník), čímž se hodnota `false` změní na `true` a naopak.

Implementace metody `countNonWhiteCharacters()` stojí na podobných principech. HTML stránka by mohla po spuštění vypadat takto:

Some text:

How many words does this text contain?

Submit

	Count	Unit
Word count:	7	words
Non-white characters:	32	characters
Message length:	38	characters

Obrázek 31. Formulář počítání slov

Řešení celého servletu najdeme v ukázkových příkladech.

Závěrem provedme ve zdrojovém kódu vlastní změny: Doplňme tabulku výsledků o počet **velkých písmen**, variantou je počítat pouze **číslovky**. Metodu vhodnou pro rozpoznávání potřebných znaků najdeme v dokumentaci *JavaDoc* ke třídě `Character`.

7.5 Zábava

V této kapitole představíme dvě interaktivní aplikace a jednoduchou strategickou hru. Než se k nim dostaneme, podívejme se ještě na tvorbu vlastního abstraktního rodiče servletu.

7.5.1 Vlastní rodič servletu

V dosavadních příkladech jsme používali standardní servlet z knihovny Java, který ve svých metodách (určených pro zpracování požadavků) deklaroval **dva** parametry — jeden pro *request* a druhý pro *response*. V následujících příkladech si vytvoříme vlastního **abstraktního** potomka standardního servletu, jehož metody budou mít pouze **jediný** parametr typu `HttpContext`. Ten poskytuje základní služby obou původních parametrů, tedy načítání parametrů požadavku a zprostředkování odpovědi. Nový přístup zjednoduší kód a sníží jeho závislost na konkrétní verzi *Servlet API* po změně *balíčků* (jak tomu bylo ve verzi *Jakarta*). Společný rodič však může obsahovat i další specifické služby, jako třeba zachytávání výjimek. V případě potřeby nebude problém vrátit se k původnímu zápisu.

V následující ukázce najdeme mírně upravený původní servlet `HelloWorldElement`. Importy tříd pro stručnost vynecháme — celý kód je součástí příložených příkladů.

Zdrojový kód 66. Třída `HelloWorldElementV2`

```
package net.ponec.jbook.s20_board;
```

```

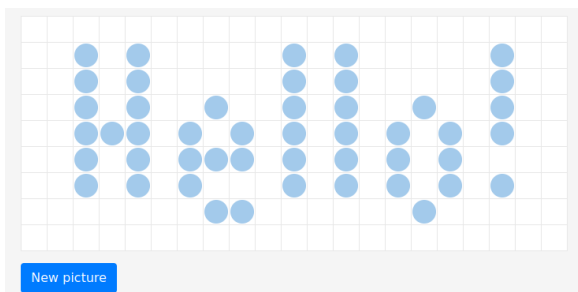
@WebServlet("/hello-context")
public class HelloContext extends MyHttpServlet { ❶
    @Override
    protected void doGet(HttpContext ctx) { ❷
        try (HtmlElement html = HtmlElement.niceOf(ctx, "css/world.css")) { ❸
            html.getBody().addHeading("Hello, World! (Context)");
            WebTools.addFooter(html, this);
        }
    }
}

```

- ❶ Implementace příkladu vychází z našeho vlastního abstraktního servletu, který se nachází v balíčku `net.ponec.jbook.tools`.
- ❷ Metoda používá jediný parametr `ctx`, navíc není nutné deklarovat kontrolované výjimky.
- ❸ Metoda pro vytvoření objektu dokáže pracovat přímo s parametrem typu `HttpContext`.

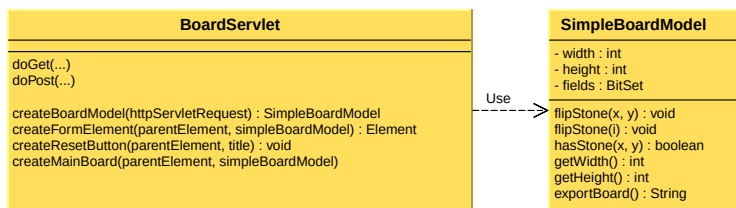
7.5.2 Bodové kreslení

Hurá na zábavu. Na dalším příkladu si vyzkoušíme sestavení dvoubarevného obrazu, kde jednotlivé body budeme zakreslovat do mřížky kliknutím myši. Výsledný vzhled by se měl blížit následujícímu obrázku.



Obrázek 32. Obrazovka bodového kreslení

Mřížku vykreslíme pomocí HTML elementu `table`, kde do každé buňky umístíme jedno tlačítko `button` typu `submit` (bez popisu) a do atributu `name` zapíšeme jednoznačný identifikátor tlačítka, celá tabulka musí být uvnitř elementu `form`. Pro práci s mřížkou budeme potřebovat *datový model*, který postavíme na třídě `SimpleBoardModel`. Podívejme se na diagram tříd servletu s modelem:



Obrázek 33. Diagram tříd bodového kreslení

Třída `SimpleBoardModel` obsahuje šířku a výšku kreslicí desky (v počtu buněk) a nějaký objekt na popis stavu jednotlivých bodů mřížky. Zde se také nabízí použití **dvourozměrné** pole primitivních logických hodnot `boolean[][]`, my však půjdeme jinou cestou a využijeme objekt typu `BitSet`. Jeho výhodou je snadná manipulace s daty a menší paměťové nároky, nevýhodou je jeho **jednorozměrnost**, kterou však lze napravit v aplikačním rozhraní (API) modelu mřížky. Každému bodu lze přiřadit **identifikační číslo**, začneme vlevo nahoře hodnotou `0` a pokračujeme směrem doprava a na konci řádku přejdeme na řádek nižší. Tímto způsobem očíslováme celou mřížku. Abychom však práci s body modelu typu `SimpleBoardModel` zpřehlednili, nabídneme metody s parametry obou souřadnic, které v implementaci přepočítáme na identifikátory buněk. Využijeme pro to operátor `%` (modulo). Popišme si jednotlivé metody modelu:

1. `flipStone(x, y)` – metoda změní logickou hodnotu buňky v mřížce, kterou si můžeme představit jako kámen tmavé barvy umístěný do mřížky: prvním zavoláním metody kámen do mřížky vložíme, druhým zavoláním kámen zase odstraníme. Buňka v tabulce je určena souřadnicemi `x` a `y`.
2. `flipStone(i)` – metoda stejného jména také změní logickou hodnotu buňky, její pozice je však určena pořadovým číslem buňky v jednorozměrném číslování, které se podobá *indexu* jednorozměrného pole. Připomeňme si, že jedna třída může obsahovat více metod stejného jména, pokud se liší počtem nebo typem parametrů. Takovému zápisu říkáme *přetěžování metod*.
3. Metoda `hasStone(x, y)` kontroluje přítomnost kamene na dané souřadnici.
4. Metody `getWidth()` a `getHeight()` poskytují výšku a šířku mřížky.
5. Metoda `exportBoard()` převede aktuální stav hry do textu. Pokud totiž uživatel klikne myší na nějakou buňku, musí formulář na server poslat (kromě jejího identifikátoru) také poslední stav hrací desky, pomocí kterého server provede rekonstrukci objektového modelu. Za tímto účelem doplníme do HTML stránky ještě *neviditelný input element* typu `hidden`, který bude ten **původní stav** hrací desky poskytovat.

Zpracování requestu v servletu `BoardServlet` bude následující:

1. Servlet při zpracování HTTP požadavku zkusí najít parametr dotazu se jménem `board`.
2. Pokud ho **nenajde**, založí logicky prázdný model, jinak se pokusí obnovit stav modelu pomocí textové hodnoty tohoto parametru. V případě zachycení výjimky založí logicky prázdný model.
3. Třída `BoardServlet` pak získá **identifikátor** tlačítka, které odeslalo formulář na server, a tento identifikátor převede na pořadové číslo bodu v datovém modelu mřížky.
4. Provede se změna stavu bodu zavoláním metody `flipStone(i)`.
5. Podle *modelu mřížky* sestavíme *datový model* HTML elementů, buňky obsahující body (kameny) přitom označíme CSS *třídou* s hodnotou `"s"` (stone). Vykreslení kolečka v tabulce však přenechme pro začátek spíše webovým grafikům.
6. Model HTML se odešle do webového prohlížeče nejpozději při uzavření kořenového elementu. A to je v principu vše.

Řešení najdeme v příložených příkladech, my se zaměříme spíše na nové programové výrazy a konstrukce.

Pomocná třída `Base64Converter`

Abychom obsah mřížky mohli posílat bezpečně v parametrech webové stránky, data získaná z objektu `BitSet` je třeba nejprve převést na *64znakovou sadu* označovanou jako *Base64*. Protože však délka takového textu je zbytečně velká, příslušná metoda `data` před konverzí zkomprimuje. Třidu `Base64Converter` (je součástí příložených příkladů) můžeme brát jako servisní službu, jejíž implementací se nebudeme zabývat. Pro zájemce je zdrojový kód k dispozici.

Parsování textu

Textová hodnota každého tlačítka je zároveň pořadové číslo buňky. Abychom z takového textu získali zpět (celé) číslo, využijeme statickou metodu třídy `Integer...`

Zdrojový kód 67. Konverze textu na číslo

```
int index = Integer.parseInt(buttonValue);
```

Pokud parser z nějakého důvodu číslo získat neumí, vyhodí výjimku. Užitečné metody pro parsování textů jsou poskytovány všemi třídami objektových alternativ k *primitivním datovým typům*.

Class – třída pro popis tříd

Již jsme zmínili dříve, že každá třída jazyka *Java* má vlastnosti jako *atributy*, *metody*, *konstruktor*, ale třeba i *třidu předka* nebo název *balíčku*. Popis vlastností každé třídy (meta-model) je k dispozici (za běhu programu) prostřednictvím objektů typu `Class`. Jeho instanci získáme zápisem názvu třídy, který je doplněn tečkou a klíčovým slovem `class`. Úplný název třídy (pro zobrazení či tisk) získáme metodou `getName()`. Podívejme se na příklad získání takového objektu.

Zdrojový kód 68.

```
Class<BoardServlet> myClass = BoardServlet.class; ❶  
String className = myClass.getName(); ❷
```

- ❶ Objekt typu `Class` získáme uvedením názvu třídy, za kterým následuje výraz `.class`. Pokud máme k dispozici objekt, jeho třídu lze získat (za běhu programu) metodou `getClass()`.
- ❷ Ukázka získání úplného jména třídy (včetně balíčku).

Logování událostí

Pokud dojde při zpracování k nějaké **nestandardní události**, je vhodné uživatele informovat v grafickém rozhraní, ale pro zpětné hledání příčiny je třeba zapisovat také okolnosti vzniku včetně časového razítka a naléhavosti. Chyby a další zajímavé události lze zapisovat do textových souborů zvaných *logy*.

Popis hierarchie volání metod se běžně označuje anglickým výrazem *stack trace*. Logování ve zdrojovém kódu se provádí pomocí třídy `Logger`, do jejího konstruktoru budeme zapisovat úplný název třídy v textovém tvaru. Typický příklad použití následuje.

Zdrojový kód 69.

```
Logger LOGGER = Logger.getLogger(BoardServlet.class.getName());  
try {  
    callSomeService();  
} catch (RuntimeException e) {  
    LOGGER.log(Level.WARNING, "An error processing parameters", e);  
}
```

Instance typu `Logger` se často zapisuje jako *soukromý statický atribut* třídy. Umístění logu závisí na nastavení projektu. Nově získanou znalost uplatníme v metodě `doGet()`, kde zachycenou výjimku lze zapisovat na konzoli (výchozí cíl), do textového souboru logů, nebo do jiného úložiště.

Třída Optional

Podívejme se na ukázkou použití *lambda* výrazu pro proměnnou typu `Consumer`. Už dříve jsme zmiňovali problém volání metody na objektu s hodnotou `null`, při které dojde k přerušení programu vyhozením výjimky. Jedním z možných řešení je vyrobit objektovou obálku s generickým typem výsledku pro uschování původní hodnoty. Konkrétní řešení nabízí standardní Java knihovna v podobě třídy `Optional`.

Zdrojový kód 70.

```
Optional<String> text = Optional.ofNullable("75");
```

Metoda `ofNullable()` je tovární metoda, která nám vyrobí hodnotu typu `Optional`. V okamžiku, kdy chceme takovou hodnotu zpracovat, zavoláme metodu `text.ifPresent(consumer)` podle vzoru:

Zdrojový kód 71.

```
text.ifPresent((String t) -> doSomething(t));
```

Do metody se posílá objekt typu `Consumer` zapsaný pomocí *lambda* výrazu, ale k volání metody `doSomething()` (s parametrem textu) dojde jen v případě, že jeho hodnota je různá od `null`. Přesně takové využití najdeme v metodě `doGet()`, která zapisuje do HTML modelu zprávu, pokud je různá od `null`. Tento zápis se vejde na **jediný řádek**. Za zmínku stojí ještě metoda `map()`, kterou lze využít například jako podmíněný konvertor textu na číslo.

Zdrojový kód 72. Podmíněný konvertor s výchozí hodnotou 0.

```
int number = text.map(Integer::parseInt).orElse(0);
```

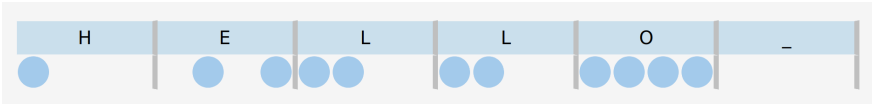
Závěrem provedme ve zdrojovém kódu vlastní změny: rozšířme naši *znakovou sadu* o několik nových znaků, můžeme doplnit třeba **vykřičník**, **otazník** a některé **číslice**. Podrobnější informace k *lambda* výrazům lze najít na internetu třeba zde [\[35\]](#).

7.5.3 Binární podoba textu

Připomeňme si, že pojem *znaková sada* představuje očíslovaný seznam vybraných grafických znaků či piktogramů. Příkladem mohou být znaky národní abecedy, čísla, mezery, znak odřádkování, symboly matematických operací a mnoho dalších. Výsledný text se vytváří seskupením pořadových čísel jednotlivých znaků a v jazyce Java se k tomu nejlépe hodí třída `String`. Připomeňme si ještě, že každý *primitivní*

datový typ má svoji pevnou délku uváděnou v bitech, což je nejmenší jednotka informace se dvěma stavy. V této kapitole si vyzkoušíme, jak by vypadal zápis textu v binární podobě ve znakové sadě, která obsahuje (pro zjednodušení) pouze 4 bity.

Zadání našeho dalšího úkolu pak zní: vytvořme HTML formulář pro simulaci **bitového pole** pro **šest** znaků. Každý znak bude mít pouze 4 bity, což vede ke stručné znakové tabulce obsahující 16 znaků (2^4). První znak bude obsahovat podtržítko, další znaky budou velká písmena z počátku abecedy.



Obrázek 34. Náhled výsledku

Pro lepší představu si takovou zkrácenou sadu znaků popíšeme:

Tabulka 11. Zjednodušená tabulka pro 16 znaků

Index	Znak	Binární číslo	Poznámka
0	_	0000	Jako oddělovač slov použijeme podtržítko.
1	A	0001	Druhý znak naší znakové sady.
2	B	0010	
3	C	0011	
4	D	0100	
5	E	0101	
6–10	F–J	0110–1010	Interval posloupnosti dalších pěti znaků.
11	K	1011	
12	L	1100	
13	M	1101	
14	N	1110	
15	O	1111	Poslední znak naší znakové sady.

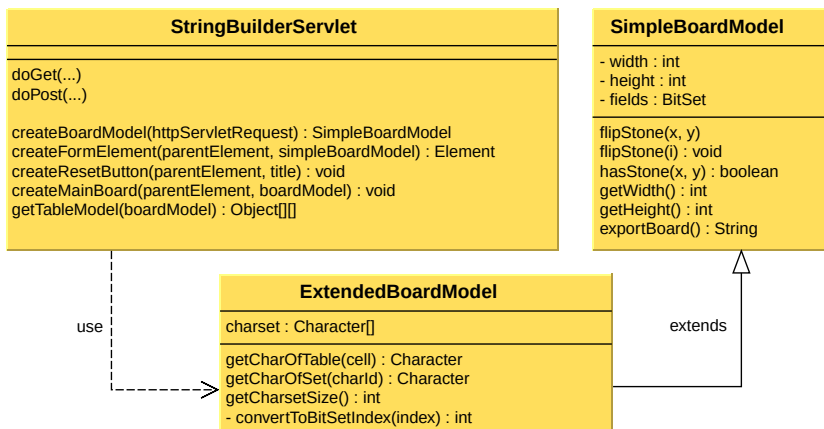
Řešení

Řešení postavíme opět na předchozím servletu, který upravíme podle zadání, a přitom využijeme také třídu pro objektový model mřížky. Pro implementaci by se nám však hodilo, kdyby *datový model* mřížky (z předchozího řešení) obsahoval navíc **seznam znaků** (z požadované znakové sady) a nabídl další veřejné metody poskytující:

1. znak z určité buňky tabulky,
2. znak na určitém indexu znakové sady,
3. velikost znakové sady jako číslo typu `int`.

Za tímto účelem vytvoříme novou třídu `ExtendedBoardModel`, která bude **potomkem** té původní třídy `SimpleBoardModel`. Původní třída přitom zůstane beze změny. Pole znaků budeme držet v atributu `charset`, vytvoření pole jsme už zmiňovali.

Za zmínku stojí soukromá metoda `convertToBitSetIndex()`, která slouží pro konverzi *veřejného indexu* znaku na interní pořadí v objektu `fields`. Algoritmus přitom projde postupně čtyři sousední *bitové* pozice a na základě získaných hodnot vypočítá výsledek.



Obrázek 35. Diagram tříd

V případě třídy `StringBuilderServlet` zmíníme hlavně vytvoření tabulkového modelu. V tabulce znakové sady chceme zobrazit také binární podobu čísla s vedoucími nulami před číslem. V jazyce *Java* však metoda `String.formatted()` binární formátování nepodporuje. Řešení existuje více, my si zde pomůžeme doplněním některých *bitů*.

Zdrojový kód 73. Tabulkový model

```

public Object[][] getTableModel(ExtendedBoardModel boardModel) {
    int mask = (int) Math.pow(2, ExtendedBoardModel.BITS_PER_CHAR); ❶
    Object[][] result = new Object[boardModel.getCharsetSize() + 1][];
    result[0] = new Object[] {"Number", "Character", "Binary"};

    for (int i = 0; i < boardModel.getCharsetSize(); i++) {
        result[i + 1] = new Object[] {
            i,
            boardModel.getCharOfSet(i),
            Integer.toBinaryString(i | mask).substring(1) ❷
        };
    }
    return result;
}

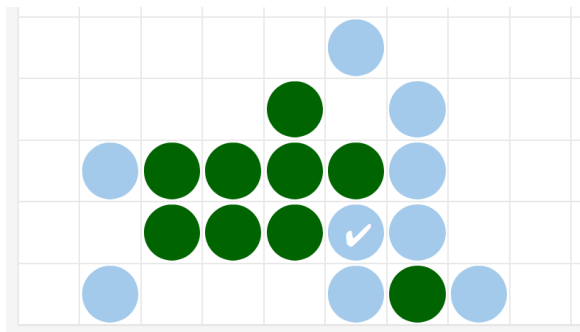
```

- 1 Nejprve si připravíme číslo, jehož bitová hodnota bude o jedničku větší než maximální hodnota čísla čtyřbitového rozsahu.
- 2 Operátor `|` (česky *svislík*, *kolmítka*, *roura*, anglicky *pipe*) provádí *bitový součet* dvou čísel, což znamená, že do výsledku promítne *logický součet bitů* na stejné pozici. Je pravda, že náš výsledek bude o jeden znak delší, ale správnou hodnotu získáme snadno metodou `substring()`. Pro úplnost uvedme také doplňkový operátor `&` pro *bitový součin*, který provádí *logický součin bitů*.

Závěrem provedme ve zdrojovém kódu vlastní změny: rozsah znakové sady rozšířme o **jeden bit** tak, aby sada pojala dvojnásobný počet znaků. Dále připravme model na vkládání znakové sady **konstruktorem**, přičemž do konstrukturu vložíme kontrolu velikosti znakové sady, která bude vyhazovat výjimku typu `IllegalArgumentException`.

7.5.4 Piškvorky s defenzivní strategií hry

Hra Piškvorky patří mezi **strategické hry**, jejichž kořeny sahají hluboko do historie. Existuje několik variant, mezi nejznámější patří zřejmě hra *Gomoku* s alternativním názvem *5 in row* (v překladu 5 v řadě). Naše pravidla hry budou následující: hráči kladou střídavě barevné kameny do mřížky s konečnou velikostí, každý hráč má svoji barvu. Vítězem se stává hráč, který jako první postaví nepřerušenou řadu alespoň pěti kamenů své barvy, povolena je i řada šikmá. Hru je možné kdykoli resetovat tlačítkem a zahájit hru novou. V naší implementaci bude protihráčem vždy počítač a výhodu prvního tahu dostane vždy člověk.

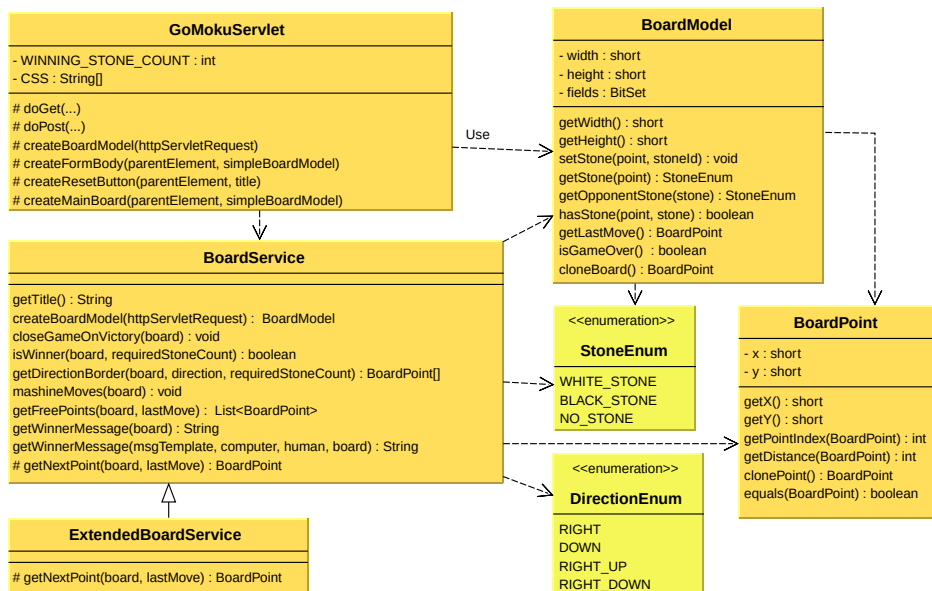


Obrázek 36. Náhled hrací desky

Strategie SW protihráče bude pouze jednoduchá obrana. Cílem hry totiž není vytvořit neporazitelný algoritmus, ale především ukázat řešení vhodné pro další studium a později třeba i pro implementaci vlastních nápadů. Čtenáři mohou nad rámec této knihy vyhledat některou knihovnu zaměřenou na hry typu *Gomoku* a zkusit napojit její algoritmy do stávající architektury.

Diagram tříd

Začneme diagramem tříd, který znázorňuje jen vybrané atributy a veřejné metody:



Obrázek 37. Diagram tříd hry Piškvorky

Na první pohled si můžeme všimnout nového typu objektu, který má nad svým názvem uveden *stereotyp* zvaný *enumeration* a místo atributů má *statické konstanty* (poznáme je podle velkých písmen). Nejdříve se podívejme na schematický popis zpracování HTTP požadavku, budeme pokračovat popisem enumerátorů a jejich možností a na konci kapitoly najdeme podrobnější popis tříd a jejich metod.

Jak to funguje?

Popis zpracování HTTP požadavku krok po kroku:

1. Servlet dostane **HTTP** požadavek metodou **GET** nebo **POST** a zavolá službu, která provede následující akce:
 - a. obnoví původní *datový model* deskové hry,
 - b. přidá do něj poslední tah hráče,
 - c. pokud tah vede k vítězství, stav se zapíše do modelu a metoda končí,
 - d. vypočítá tah počítače a přidá ho do modelu,

- e. pokud tah vede k vítězství, stav se zapíše do modelu a metoda končí,
 - f. jinak zpracování pokračuje standardním způsobem.
2. Sestaví se HTML model hrací desky (pomocí třídy `Element` pro generování HTML kódu) podle aktuálního modelu hrací desky.
 3. Doplníme standardní patičku stránky a kořenový element uzavřeme.

Obecným cílem našeho defenzivního algoritmu je najít na desce a obsadit svými kameny strategické místo protihráče, v tomto případě člověka. Následuje popis algoritmu, jehož implementace začíná voláním metody `machineMoves(board)` ze třídy `BoardService`.

1. Metoda si načte parametry z modelu hrací desky a volá jinou metodu stejné třídy s názvem `getNextPoint()`.
2. Tato metoda obsahuje tři vnořené cykly, ve kterých hledá **nejdelší nepřerušenu řadu** kamenů protihráče (člověka):
 - a. Nejdříve hledáme kameny v délce kratší o jeden kámen, než je vítězný počet. V našem případě začneme tedy hodnotou 4 (čtyři). Pokud se taková řada nenajde, snížíme podmínku o jeden kámen a hledáme znovu, až se dostaneme k číslu 2 (dva).
 - b. Směry položených kamenů jsou popsány výčtovým typem `DirectionEnum`. Kameny hledáme metodou `getDirectionBorder()` a pro rychlejší zpracování budeme prohledávat jen úsečky procházející posledním vloženým kamenem. Pokud metoda hledanou sekvenci kamenů najde, vrátí dva okrajové body **vně** této pomyslné úsečky, jinak vrátí `null`.
 - c. Poslední cyklus projde body nalezené hranice a vrátí první, který leží uvnitř desky a zároveň je volný.
3. Pokud se nepodaří najít žádnou hranici, vezmou se všechny volné body kolem posledního tahu a náhodným výběrem se vybere jeden z nich.

Brzy však zjistíme, že k vítězství stačí jen 6 tahů, a hra přestane být zajímavá. Z toho důvodu rozšíříme algoritmus pomocí metody `ExtendedBoardService.getNextPoint()`, která používá odlišnou strategii, náhodně však deleguje své zpracování i na rodičovskou implementaci pomocí klíčového slova `super`. Takové řešení ukazuje využití **Polymorfismu**. Protože nová strategie je už poměrně invazivní a málo zábavná pro hráče, použijeme tento algoritmus jen pro polovinu případů, a to náhodně.



V překryté metodě lze volat *zastíněnou* metodu pouze do **nejbližšího** předka třídy. Pro volání vzdálenějších předků nemá jazyk *Java* podporu. Tyto případy je třeba řešit vhodnějším

Jak funguje rozšířená implementace metody `getNextPoint()` ?

1. Metoda projde postupně všechna volná pole kolem posledního tahu protihráče.
2. Pro každé takové pole naklonujeme hrací desku (metodou `board.cloneBoard()`).
3. Umístí do ní na zkoušku kámen protihráče a zkontroluje, zdali takový tah nevede k vítězství protihráče či k souvislé řadě 4 po sobě jdoucích kamenů.
4. Pokud ano, počítač pole obsadí vlastním kamenem.
5. Jinak zavolá tu původní implementaci předka.

Enumerátor

Při programování je někdy výhodné pracovat s programovými objekty, které reprezentují nějaký neměnný **seznam** hodnot (říkáme jim konstanty). Typickým příkladem je seznam **měsíců** kalendářního roku. Pro takové případy nabízí jazyk *Java* výčtový typ (anglicky *enumerated type*) zvaný též *enumerátor*. **Enumerátor** je specifický objektový typ, který poskytuje pevný (neměnný) **seznam** instancí (konstant) svého vlastního typu. Každá konstanta enumerátoru poskytuje (prostřednictvím svých metod) **jméno**, **pořadové číslo** (index) a také **seznam** všech konstant stejného typu. Všechny enumerátory mají společného rodiče, kterého tvoří abstraktní třída `Enum`. Její implementace pak mohou mít (podobně jako běžné třídy) své *atributy*, *konstruktory* i *metody*_. S výčtovými typy pak lze pracovat podobně, jako by se jednalo o statické konstanty. Podívejme se na zdrojový kód výčtového typu.

Zdrojový kód 74. Ukázka kódu výčtového typu

```
public enum StoneEnum { ①
    WHITE_STONE, ②
    BLACK_STONE, ③
    NO_STONE; ④
}
```

- ① Hlavička se podobá třídě, deklarace však obsahuje klíčové slovo `enum`.
- ② První hodnota reprezentuje bílý kámen na hrací desce.
- ③ Druhá hodnota reprezentuje černý kámen, jednotlivé položky se oddělují čárkou.
- ④ Poslední konstanta se zakončuje středníkem.

Pro srovnání zkusme implementovat podobný enumerátor pomocí prostředků běžné *Java* třídy.

```
public final class MyStoneEnum { ❶

    public static final MyStoneEnum WHITE_STONE
        = new MyStoneEnum(0, "WHITE_STONE"); ❷
    public static final MyStoneEnum BLACK_STONE
        = new MyStoneEnum(1, "BLACK_STONE");
    public static final MyStoneEnum NO_STONE
        = new MyStoneEnum(2, "NO_STONE");
    private final int order; ❸
    private final String name; ❹

    private MyStoneEnum(int order, String name) { ❺
        this.order = order; ❻
        this.name = name;
    }
    public int ordinal() {
        return order;
    }
    public String getName() {
        return name;
    }
    public static MyStoneEnum[] values () { ❼
        MyStoneEnum[] result = { WHITE_STONE, BLACK_STONE, NO_STONE };
        return result;
    }
    public static MyStoneEnum valueOf(String name)
        throws IllegalArgumentException {
        for (MyStoneEnum value : values()) { ❽
            if (value.getName().equals(name)) {
                return value;
            }
        }
        throw new IllegalArgumentException(name);
    }
}
```

- ❶ Modifikátor třídy **final** znemožní tvorbu potomků.
- ❷ Veřejný statický atribut je stejného typu jako třída, modifikátor **final** garantuje jeho neměnnost.
- ❸ Pořadové číslo začíná hodnotou 0.
- ❹ Jméno výčtu vždy koresponduje s názvem statického atributu.
- ❺ Konstruktor je soukromý, aby nové instance nemohla vytvářet žádná jiná třída.
- ❻ Klíčové slovo **this** odlišuje název atributu od stejnojmenného argumentu.
- ❼ Metoda vrací všechny své statické atributy jako pole.
- ❽ Metoda vrátí statický atribut třídy nalezený podle jména, jinak vyhodí výjimku.

Ted' už je snad jasnější, že zavedení výčtových typů přispělo ke stručnějšímu zápisu kódu docela významně. Tím však výhody enumerátorů nekončí, **lze** je používat efektivně také v *příkazech* `switch`.

Příkaz `switch`

Příkaz `switch` umožňuje větvit program podle *primitivních celočíselných hodnot* (literálů či konstant). Z objektů jsou podporovány ještě typy `String` a `Enum`. Příkladám zkrácenou ukázkou použití ze třídy `BoardModel`.

Zdrojový kód 76. Ukázka použití příkazu `switch`

```
/** Set a stone by an index */
public void setStone(BoardPoint point, StoneEnum stone) {
    switch (stone) { ❶
        case BLACK_STONE: ❷
        case WHITE_STONE:
            fields.set(convertToBitSetIndex(point, stone)); ❸
            lastMove.set(point);
            break; ❹
        default: ❺
            throw new IllegalArgumentException("Illegal stone" + stone);
    }
}
```

- ❶ Příkaz `switch` vyhodnocuje proměnnou, která musí být různá od `null`. Tělo příkazu je ohraničené složenými závorkami.
- ❷ Jednotlivé případy se zachytávají pomocí klíčového slova `case` s konstantou zakončenou dvojtečkou. Výčtový typ je jednoznačně určen v hlavičce příkazu a při zachytávání případů se už neuvádí.
- ❸ Následuje blok *příkazů*, které se provádějí až do nalezení klíčového slova `break`. Zpracování je shodné pro bílý i tmavý kámen. Uložíme pozici kamene do objektu typu `BitSet`, metoda `convertToBitSetIndex()` však nejprve spočítá hodnotu potřebného indexu. Pozici posledního položeného kamene si pak uložíme pro pozdější grafické zvýraznění.
- ❹ Příkaz pro ukončení bloku *příkazů*. V případě jeho absence by zpracování pokračovalo průchodem následujících případů v příkazu `switch`.
- ❺ Hodnoty, které nejsou zachyceny žádným případem `case`, lze zachytit (volitelně) klíčovým slovem `default`, přičemž tento blok nemusí být nutně na poslední pozici. Zde se vyhazuje výjimka, protože zadanému bodu má smysl přiřadit kámen pouze bílé nebo černé barvy.

Popis tříd diagramu a jejich metod

1. `GoMokuServlet` je třída typu `HttpServlet`, která nijak nevybočuje z předchozích ukázek. Třída obsahuje dvě statické konstanty: první obsahuje minimální počet kamenů nutných k vítězství, druhá seznam kaskádových stylů HTML stránky.
2. `StoneEnum` poskytuje konečný výčet stavů (kamenů) každé buňky na hrací desce. Jedna buňka může obsahovat kámen bílý, černý, anebo může být prázdná. Poznámky k výčtovému typu najdeme v kapitole [Enumerátor](#).
3. `BoardPoint` je třída pro **adresování** jednoho bodu hrací desky (buňky), kterou budou využívat ostatní třídy. Třída podporuje jak **souřadnicové** adresování (x a y), tak i adresování **jednorozměrné** (`index`), dále nabízí metody pro posun bodu (na desce) a pro výpočet vzdálenosti od jiného bodu.
4. `BoardModel` je třída pro sestavení **datového modelu** hrací desky. Hlavní změnou proti předchozím příkladům jsou **tři stavy** buňky, protože každá buňka může obsahovat právě jednu ze tří hodnot výčtového typu `StoneEnum`. Pro popis modelu využijeme opět třídu `BitSet`, ale pro každou buňku budeme rezervovat dva *bity* s následujícími stavy: oba *bity* s hodnotou `0` (nula) značí **prázdnou** buňku, první *bit* (ten s menším indexem) s hodnotou `1` značí **bílý** kámen, jinak druhý *bit* s hodnotou `1` značí **černý** kámen. Příklad se dvěma jedničkami je neplatný. Popis vybraných metod:
 - a. `setStone(boardPoint, stone)` – metoda umístí kámen typu `StoneEnum` na zadanou pozici,
 - b. `getStone(boardPoint)` – metoda vrací kámen (typu `StoneEnum`) na zadané pozici,
 - c. `getOpponentStone()` – metoda vrací kámen protihráče,
 - d. `getBitSetIndex()` – soukromá metoda pro výpočet indexu pro objekt typu `BitSet`, který fyzicky obsahuje požadované stavy,
 - e. `isInsideBoard(point)` – metoda zjistí, zda se bod nachází uvnitř hrací desky,
 - f. `hasStone(point, stone)` – metoda kontroluje přítomnost konkrétního kamene na zadané pozici,
 - g. `getLastMove()` – metoda vrací souřadnici posledního tahu v datovém objektu `BoardPoint`,
 - h. `gameOver()` – dojde k logickému ukončení hry, pokus o vložení dalšího kamene pak vyhodí výjimku,
 - i. `isGameOver()` – vrací příznak ukončení hry,
 - j. `getErrorMessage()` je pomocná metoda, pomocí které lze získat zprávu určenou pro HTML stránku. Třída tyto zprávy sama nevytváří, ale bere si pouze text zapsaný metodou `setErrorMessage()`,

- k. `cloneBoard()` – metoda vytvoří kopii hrací desky, kterou lze použít pro simulování tahu počítače za účelem jeho vyhodnocení. Změny stavu klonované desky nesmí ovlivnit původní model.
- 5. `BoardService` – třída poskytuje aplikační logiku, což jsou algoritmy potřebné pro běh hry. Jejich vyčlenění do samostatné servisní třídy zlepšuje čitelnost kódu, usnadňuje jeho údržbu a usnadňuje sdílení společných funkcí i jeho případné rozšiřování. Význam metod:
 - a. `getTitle()` – stručný popis herní metody, který se zobrazí uživatelům v HTML stránce,
 - b. `createBoardModel()` je tovární metoda na výrobu datového modelu hrací desky, definuje jeho rozměry a nastaví poslední tah z parametru HTTP requestu,
 - c. `closeGameOnVictory()` – pokud jsou splněny podmínky vítězství na hrací desce, dojde k uzavření modelu,
 - d. `isWinner()` – metoda dostane v parametru *model desky* a minimální počet kamenů v řadě potřebný k vítězství. Metoda vždy testuje jen blízké okolí posledního vloženého kamene,
 - e. `machineMoves()` – zpracování tahu počítače,
 - f. `getWinnerMessage()` – metoda sestavuje zprávu pro vítěze hry,
 - g. `getDirectionBorder()` – metoda vrací dvojici bodů na přímce, které ohraničují počet kamenů v daném směru, jinak vrací hodnotu `null`,
 - h. `of()` – tovární metoda na výrobu instance služby.
- 6. `ExtendedBoardService` – třída překrývá metodu za účelem implementace odlišné strategie, řešení využívá *Polymorfismus*.
- 7. `DirectionEnum` – jedná se o *enumerátor*, který nabízí výčet směrů kamenů, které se počítají pro vítězství:
 - a. `move()` – nabízí metodu, která umožňuje posun bodu (zadaného parametrem) v daném směru – jedním nebo opačným směrem. Pro možnost řetězení metoda vrací stejnou instanci bodu, kterou dostala v parametru.

Zpracování dotazu

Řešení kódu naznačíme jen na úrovni vstupní metody servletu `doGet()`...

Zdrojový kód 77. Vstupní metoda servletu

```
protected void doGet(HttpContext ctx) {  
    try (HtmlElement html = HtmlElement.niceOf(service.getTitle(), ctx, css())) {
```

```
try {
    writeHeaderElements(html); ❶
    html.getBody().addHeading(html.getTitle());
    BoardModel board = service.createBoardModel(request); ❷
    Element form = createFormElement(html.getBody(), board); ❸
    board.getErrorMessage()
        .ifPresent(msg -> form.addSpan("msg").addText(msg)); ❹
    service.getWinnerMessage(board)
        .ifPresent(msg -> form.addSpan("msg").addText(msg));
    createMainBoard(form, board); ❺
    createResetButton(form, "New game");
} catch (RuntimeException e) { ❻
    String msg = "Internal server error caused by "
        + e.getClass().getSimpleName();
    LOGGER.log(Level.SEVERE, msg, e);
    html.getBody().addSpan("msg").addText(msg
        + ", see a server log for detail");
}
WebTools.addFooter(html, this);
}
```

- ❶ Smysl volání prázdné metody si vysvětlíme v kapitole o [AJAX](#).
- ❷ Metoda obnoví (z parametrů requestu) poslední *datový model* hrací desky a přidá do něj poslední tah hráče i nový tah počítače, případně nastaví i informace o stavu hry.
- ❸ Metoda sestaví HTML model hrací desky.
- ❹ Podmínečné zobrazení chybové zprávy s využitím třídy *Optional*. Podobnou konstrukci použijeme na dalším řádku pro zobrazení zprávy o vítězství.
- ❺ Sestavíme model hrací desky a na dalším řádku přidáme resetovací tlačítko.
- ❻ Případnou výjimku zpracování zachytíme za účelem archivace této události do logu a informování uživatele.

Úplné řešení najdeme v příložených příkladech. Jejich obsah může být zajímavý z pohledu algoritmu, ale z pohledu syntaxe jazyka už by neměl přinést významné překvapení. V případě nejasností doporučuji použít dokumentaci *JavaDoc* [5] k API knihovny *Java*. Pokud budete chtít sledovat chování metod a obsah proměnných, lze do zdrojových kódů vkládat logovací zprávy. Alternativou je krokování aplikace v pokročilém editoru (IDE).

7.6 Interaktivní aplikace s AJAX

V předchozích ukázkách webových stránek vedl stisk tlačítka ke stažení (a následnému vykreslení) celé nové HTML stránky. Takové řešení však zpomaluje

odezvu uživatelských reakcí, může vést k nepříjemnému problikávání obrazovky, a v některých případech může dokonce zvyšovat náklady na internetové připojení. Jaké to má řešení?

1. Můžeme odstranit *bílé znaky* mezi HTML elementy. V našich příkladech lze takové mezery vyloučit získáním kořenového HTML elementu alternativní metodou `HtmlElement.of()` – místo té původní metody `HtmlElement.niceOf()`. Pomocí konfigurační třídy `DefaultHtmlConfig` lze také potlačit zalamování řádků.
2. Na straně *webového kontejneru* (Tomcat) lze aktivovat kompresi odesílaných HTML stránek, která může redukovat objem stahovaných dat až na desetinu.
3. Účinným řešením je využití **konceptu AJAX** a právě tomuto tématu se budeme věnovat podrobněji dále.
4. Výše uvedené tipy lze vzájemně kombinovat.

7.6.1 Co je to AJAX

Pojem *AJAX* se objevil na veřejnosti v roce 2005, jeho principy se však vývojáři pokoušeli zapracovat do internetových prohlížečů už o několik let dříve. Standardizovaná verze pak byla pojmenována *ECMAScript*®. V zásadě jde o popis procesu (koncept), který vede ke změně vzhledu části webové stránky, kde *JavaScript* internetového prohlížeče získá (například ze serveru) nová data, aby je pak použil k úpravě vybraných elementů stávajícího HTML kódu. Název konceptu byl odvozen od anglického výrazu *Asynchronous JavaScript and XML*. Doplňme, že *JavaScript* je skriptovací jazyk běžně dostupný ve většině moderních internetových prohlížečů.



Příklady v této knize vyžadují *JavaScript Vanilla* verze *ES6*, jehož specifikace byla uvolněna v roce 2015. V současné době by ho měly podporovat všechny populární internetové prohlížeče.

V dalších *AJAX* ukázkách budeme nahrazovat výhradně **obsah** vybraného HTML elementu (jednoho či více), přičemž samotný cílový element zůstane beze změny. Pokud tedy plánujeme změnit barvu textu nějakého odstavce, je třeba nahradit jeho vnitřek novým obsahem doplněným o vnořený element s potřebným CSS stylem. Jasnější to bude z následující ukázky.

Zdrojový kód 78. Textový odstavec v HTML stránce

```
<p class="commonFont"> ❶  
  <span class="alertFont">Hello, World!</span> ❷  
</p>
```

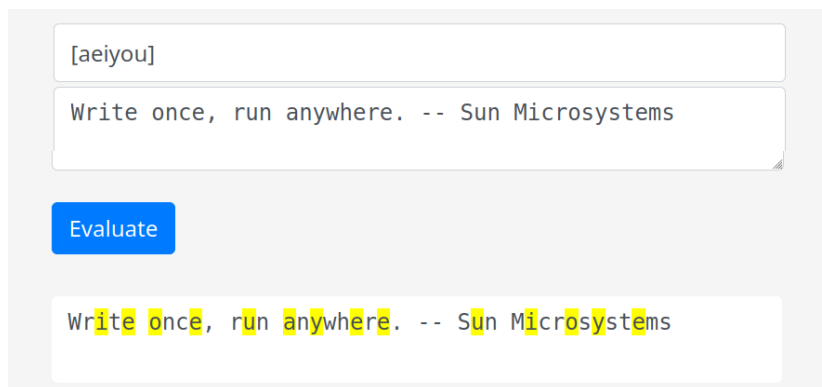
- ❶ Element odstavce, na který chceme směřovat změnu.

- ② Obsahem odstavce může být prostý text. Pokud však chceme změnit jeho barvu, je třeba obalit ho novým elementem s potřebným CSS stylem, protože styl odstavce tady už měnit nelze.

Praktické využití si ukážeme na následujících příkladech, jejichž řešení bylo ovlivněno snahou ušetřit čtenáře studia jazyka *JavaScript*. Nepochybně bychom však našli také řadu jiných postupů, které by přinesly stejný (či podobný) výsledek.

7.6.2 Testování regulárních výrazů

Naším dalším úkolem bude napsat servlet obsluhující tvorbu webového formuláře pro testování *regulárních výrazů*, který bude zobrazovat výsledek pomocí *AJAX*. Uživateli nabídneme formulář se dvěma textovými vstupy: první bude obsahovat testovaný *regulární výraz* a druhý bude obsahovat běžný text určený k prohledávání. Ve spodní části stránky zobrazíme rámeček pro zobrazení prohledávaného textu s graficky vyznačenou shodou.



Obrázek 38. Náhled formuláře pro testování regulárního výrazu

Začneme tvorbou nové webové stránky tak, jako bychom žádné použití *AJAX* neplánovali. Do formuláře stránky vložíme textové vstupy a doplníme tlačítko pro odeslání jejich obsahu na server. Základní funkce formuláře se v principu nebude lišit od dřívějších ukázek s textovými vstupy. Podívejme se na první verzi zdrojového kódu:

Zdrojový kód 79. Tělo metody `doGet()` servletu webové stránky

```
protected void doGet(HttpContext ctx) {  
    try (HtmlElement html = HtmlElement.niceOf("Regular expressions", ctx,  
        "css/regexp.css", "css/form3.css")) {  
        //new JavaScriptWriter().write(html.getHead()); ①  
        Message msg = highlight(ctx); ②  
        try (Element body = html.getBody()) {
```

```
body.addHeading(html.getTitle());
try (Element form = body.addForm().setMethod(Html.V_POST)
    .setAction("?")) {
    form.addInput(Constants.CONTROL_CSS) ③
        .setType(Html.V_TEXT)
        .setName(Constants.REGEXP)
        .setValue(ctx.getParameter(Constants.REGEXP))
        .setAttribute(Html.A_PLACEHOLDER, "Regular expression");
    form.addTextArea(Constants.CONTROL_CSS) ④
        .setAttribute(Html.A_PLACEHOLDER, "Plain Text")
        .setName(Constants.TEXT)
        .addText(ctx.getParameter(Constants.TEXT));
    form.addDiv().addButton("btn", "btn-primary").addText("Evaluate");
    form.addDiv(Constants.CONTROL_CSS, Constants.AJAX_AREA)
        .addRawText(msg); ⑤
}
WebTools.addFooter(html, this);
}
}
```

- ① Vložení *JavaScriptového* kódu do hlavičky HTML stránky ponechme dočasně zakomentované. Zápis do HTML hlavičky (element *header*) je nutné provést před prvním zápisem do těla stránky (element *body*).
- ② Metoda pro získání objektu s formátovanou výstupní zprávou. Výsledný objekt obsahuje také příznak **neplatného** regulárního výrazu, který získáme metodou *isError()*. Algoritmus sestavení celé zprávy není pro další výklad důležitý a nebudeme se jím zabývat.
- ③ Textová položka pro vkládání regulárního výrazu.
- ④ Textová položka pro vkládání prohledávaného textu.
- ⑤ Rámeček pro zobrazení výsledku.

Po spuštění aplikace už lze základní fungování stránky zkontrolovat. Dalším cílem bude rozšířit aplikaci tak, aby se po odeslání formuláře překreslil pouze text vyznačující shodu s regulárním výrazem. Všechny další úpravy budeme doplňovat do onoho rozpracovaného servletu.

Protože bez *JavaScriptu* se už dále neobejdeme, „odkomentujme“ příkaz pro jeho vkládání do hlavičky HTML stránky. Jaké služby budeme od *JavaScriptu* potřebovat? V prvé řadě musí převzít kontrolu nad odesíláním položek formuláře, aby (při potvrzení změn) posílal hodnoty formuláře na servlet a sám zpracoval odpověď. Formát odpovědi bude *JSON*, jehož popis jsme zmínili dříve v samostatné kapitole. Aby však servlet věděl, který formát odpovědi má připravit, připojí *JavaScript* do žádosti ještě jeden (pomocný) parametr, například s hodnotou *_ajax=true*. Uživatelům by se zřejmě hodila také průběžná aktualizace výsledku, aby nemuseli

každou změnu formuláře potvrzovat tlačítkem ručně. Odesílání formuláře však není dobré provádět po každé změně hned, ale spíše s krátkým prodlením, aby se komunikace se serverem zbytečně nezahltla. Nakonec potřebujeme, aby *JavaScript* zaměnil v HTML stránce text předchozí zprávy za nový.

Dobrá zpráva je, že *JavaScript* s takovou funkcí nemusíme psát sami, ale můžeme využít hotovou implementaci z frameworku *Ujorm* [23]. Výhodou jsou jednoduchá pravidla, stručný zdrojový kód a jeho nezávislost na dalších knihovnách (při běhu). Skript reaguje na změny vybraných textových vstupů a rozumí *JSON* odpovědi, ve které každý klíč obsahuje právě jeden CSS selektor cílového elementu HTML stránky a hodnota obsahuje vnitřní HTML kód určený k náhradě.

Zdrojový kód 80. Zpracování AJAX události

```
@Override
protected void doPost(HttpContext ctx) throws IOException { ❶
    if (DEFAULT_AJAX_REQUEST_PARAM.of(ctx, false)) { ❷
        doAjax(ctx, JsonBuilder.of(ctx.writer())).close(); ❸
    } else {
        doGet(ctx); ❹
    }
}
protected JsonBuilder doAjax(HttpContext ctx, JsonBuilder json) throws IOException {
    Message msg = highlight(ctx); ❺
    json.writeClass(Constants.AJAX_AREA, e -> e ❻
        .addElementIf(msg.isError(), Html.SPAN, "error") ❼
        .addRawText(msg)); ❽
    return json;
}
```

- ❶ Protože takovou implementaci budeme využívat i v následujících příkladech, bude výhodné ji přesunout do společné rodičovské třídy *MyHttpServlet*.
- ❷ Nejdříve je třeba rozpoznat *AJAX* požadavek, který očekáváme v metodě *doPost()*. Využijeme pro to metodu statické konstanty *DEFAULT_AJAX_REQUEST_PARAM.of()*, jež získá hodnotu parametru *_ajax*, kterou zmíněný *JavaScript* k dotazu automaticky přikládá. Druhým parametrem metody *of()* je hodnota výsledku pro případ, že HTTP parametr chybí.
- ❸ V závislosti na hodnotě parametru se další zpracování deleguje na metodu *doAjax()*, nebo na původní metodu *doGet()*. První z metod očekává dva parametry: původní *HTTP* požadavek (pro získání parametrů formuláře) a objekt typu *JsonBuilder* pro sestavení odpovědi ve formátu *JSON*.
- ❹ Pokud se **nejedná** o *AJAX* request, sestavení celé HTML stránky provede standardně metoda *doGet()*.
- ❺ Pro získání fragmentu HTML kódu (pro záměnu) zavoláme stejnou metodu, jako tomu bylo při sestavení celé HTML stránky.

- ⑥ Metoda `JsonBuilder.write()` zapíše do *JSON* jeden záznam typu **klíč:hodnota**, kde parametr klíče obsahuje CSS selektor cílového elementu. Pokud je selektorem CSS třída (viz obsah konstanty `AJAX_AREA`), použijeme metodu `writeClass()`. Druhým parametrem je objekt typu `ValueProvider` s parametrem `e` typu `Element`. Na tento objekt můžeme nahlížet tak, jako by se jednalo o HTML element rodiče, jehož **obsah** chceme nahrazovat (ve výsledné HTML stránce). Obsah sestavíme pomocí *lambda* výrazu.
- ⑦ Metoda `addElementIf()` slouží k podmíněnému vkládání vnořeného elementu, který potřebujeme pro grafické zvýraznění nevalidního regulárního výrazu. Pokud by vložení aktuálního elementu bylo potlačeno, jeho vnořené elementy budou nadále zachovány.
- ⑧ Metoda `addRawText()` umí vložit do modelu formátovaný text s HTML značkami. Právě takové chování totiž potřebujeme pro vkládání zprávy s formátovaným obsahem.

Pokud jsme použili správně CSS selektory, naše práce končí. O modifikaci původního obsahu HTML stránky se už postará kód vloženého *JavaScriptu*. Výhodou použitého řešení je, že bude fungovat i v prohlížečích bez podpory *JavaScriptu*, jen bude nutné změny formuláře potvrzovat opět tlačítkem.

7.6.3 Piškvorky s rychlejší odezvou

V původní hře Piškvorky každý tah hráče vyvolá reakci, během které server sestaví novou HTML stránku s velmi malým počtem přidaných informací. Přitom by stačilo vyměnit obsah zpravidla jen **tří** buněk tabulky (z počtu bezmála tří set). Konkrétně se jedná o:

1. jednu buňku obsahující nově vložený kámen uživatele,
2. buňku zobrazující odpověď počítače,
3. dále bude třeba odstranit grafickou značku **posledního** vloženého kamene, protože po dalším tahu bude tato informace zastaralá.

Pro implementaci *AJAX* použijeme stejný přístup jako v minulém příkladu. Tentokrát však nebudeme upravovat původní servlet, ale změny připravíme do servletu nové třídy, který bude jeho potomkem. Přitom si osvěžíme výhody dědičnosti tříd. Podívejme se na tři nejdůležitější překryté metody, ty ostatní jsou prakticky identické s předchozí ukázkou.

Zdrojový kód 81. Metody nového servletu

```
@Override
protected void writeHeaderElements(HtmlElement html) {
```

```
        new JavaScriptWriter().write(html.getHead()); ❶
    }

    @Override
    protected void createBoardModel(Element form, BoardModel board) {
        form.addInput().setType(Html.V_HIDDEN) ❷
            .setName(LASTPOINT_PARAM)
            .setValue(board.getLastMove().getPointIndex(board));
        super.createBoardModel(form, board); ❸
    }
```

- ❶ Pro podporu AJAX potřebujeme vložit do HTML stránky dříve zmíněný *JavaScript*. Za tím účelem překryjeme (viz anotaci `@Override`) metodu, která byla v **rodičovské** třídě prázdná.
- ❷ Implementace původní metody `createBoardModel()` zapisovala do skrytého vstupního pole datový model aktuální hry. Ten však postrádá pozici naposledy vloženého kamene. Řešení se nabízí několik, ale možná nejméně pracné bude ukládat jeho index do další skryté textové položky.
- ❸ Protože do formuláře potřebujeme ukládat i ten původní model, zavoláme i původní metodu rodiče pomocí klíčového slova `super`.

Největší objem nového kódu je zapsán v metodě `doAjax()`:

Zdrojový kód 82. Zpracování AJAX požadavku

```
protected JsonBuilder doAjax(HttpServletRequest request, JsonBuilder response)
    throws ServletException, IOException {
    BoardModel board = service.createBoardModel(request); ❶
    int lastPoint = service.parseInt(request.getParameter(LASTPOINT_PARAM), 0); ❷
    for (BoardPoint point : board.getChangedPoints(lastPoint)) { ❸
        String pointSelector = "tr:nth-child(%s) td:nth-child(%s)".formatted( ❹
            point.getY() + 1, ❺
            point.getX() + 1);
        response.write(pointSelector, e -> writeStone(e, point, board)); ❻
    }
    response.writeId(BOARD_MODEL_ID, e -> createBoardModel(e, board)); ❼
    response.writeId(MESSAGE_ID, e -> createMessagePanel(e, board)); ❽
    return response;
}
```

- ❶ Model hrací desky získáme stejně jako v původní implementaci.
- ❷ Získáme index posledního tahu.
- ❸ Pro získání modifikovaných polí hrací desky využijeme metodu, kterou jsem připravil ještě v původní implementaci. Do jejího argumentu pošleme index posledního vloženého kamene. Při zahájení nové hry metoda vrátí popis **všech** polí hrací desky.

- ④ Dále potřebujeme nahradit obsah buněk tabulky bez jakéhokoli identifikátoru. Vzpomínáte ještě na CSS selektor využívající pořadové číslo HTML elementu? Přesně takový selektor využijeme i zde. Alternativou může být doplnění identifikátorů k buňkám tabulky překrytím metody odpovědné za sestavení buňky.
- ⑤ Protože selektory pracují s pořadovým číslem, je třeba k indexu přidat jedničku.
- ⑥ Pro vykreslení obsahu buňky už máme připravenou metodu, stačí ji jen zavolat.
- ⑦ Do odpovědi musíme ještě přibalit aktuální model hrací desky.
- ⑧ Závěrem doplníme zprávu o případném ukončení hry a vítězi.

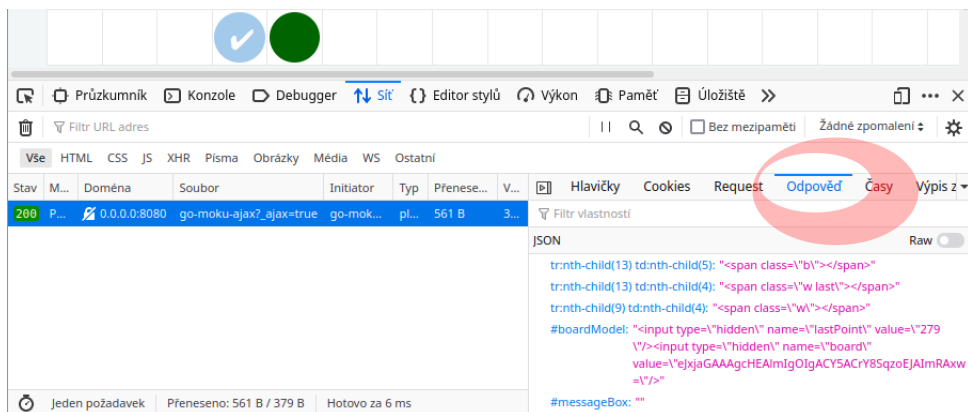
Stejně jako v předchozím případě platí, že záměnu obsahu HTML elementů zajistí dříve zmíněný *JavaScript*. Nyní se podívejme na ukázkou *JSON* odpovědi iniciované *AJAX* dotazem. Každý jednotlivý řádek zapsaný ve složených závorkách popisuje jeden atribut s hodnotou:

Zdrojový kód 83. Obsah JSON odpovědi

```
{
  "tr:nth-child(4) td:nth-child(10)": "<span class=\"b\"></span>", ①
  "tr:nth-child(6) td:nth-child(12)": "<span class=\"w last\"></span>",
  "tr:nth-child(5) td:nth-child(10)": "<span class=\"w\"></span>",
  "#boardModel": "<input type=\"hidden\" name=\"board\" value=\"eJxjYM.....\" ", ②
  "#messageBox": "" ③
}
```

- ① Každý řádek *JSON* dokumentu obsahuje dvojici hodnot: CSS selektor cílového HTML elementu a hodnotu s HTML kódem pro záměnu obsahu. Pokud by měl CSS selektor shodu s více elementy, nahradily by se všechny.
- ② Do HTML formuláře vložíme také *datový model* hrací desky a do dalšího neviditelného vstupu také index posledního tahu. Text HTML kódu je tady zkrácený.
- ③ Poslední selektor bude obsahovat zprávu pro uživatele, prázdný text předchozí zprávu smaže.

Odpovědi servletu si lze prohlédnout v internetovém prohlížeči. Postup zobrazení pro *Mozilla Firefox* (verzi 94.0) příkládám: klikneme postupně myší na: → Menu → Další nástroje → Nástroje pro webové vývojáře → záložka Síť → dále je třeba položit myší na hrací desku nový kámen → vlevo dole vybrat řádek posledního requestu → a zobrazit záložku Odpověď. Teď bychom měli spatřit obrazovku podobnou té z následujícího obrázku.



Obrázek 39. Komunikace v internetovém prohlížeči

Rychlejší odezva nové verze hry není jedinou změnou grafického rozhraní. Nyní můžeme pozorovat, že už nelze vrátit náš poslední tah tlačítkem webového prohlížeče pro návrat k předchozí stránce, protože celá hra běží v jediné stránce. Případné stornování posledního tahu bychom si museli naprogramovat sami.

7.7 Herní strategie od umělé inteligence

Zkratka *AI* pochází z anglického Artificial Intelligence, neboli umělá inteligence. Jedná se o obor informatiky, který se zabývá vývojem systémů schopných vykonávat úkoly, které obvykle vyžadují lidskou inteligenci. Patří sem například schopnost učení, řešení problémů, rozpoznávání vzorů nebo, jako v našem případě, rozhodování o optimální herní strategii.

7.7.1 Jak formulovat požadavek na herní strategii pro AI

Předchozí kapitola vás seznámila s implementací základního algoritmu pro hru Piškvorky. Nyní se podíváme, jak využít *AI* nástroje pro generování pokročilejší a efektivnější herní strategie. Cílem je získat kód pro metodu `getNextPoint(BoardModel board)` v rámci třídy `FineBoardService`. Tato metoda vrací optimální bod pro další tah ve hře. Než začnete formulovat dotaz, ujistěte se, že máte k dispozici definici třídy `BoardModel`, kterou *AI* potřebuje k pochopení způsobu reprezentace herního stavu. *AI* je nutné poskytnout dostatečný kontext a jasné požadavky na výstupní kód. Začněte stručným popisem hry – i když *AI* Piškvorky pravděpodobně zná, definujete tím herní rámec. Dále specifikujte, že chcete algoritmus pro generování optimálního tahu. Uveďte konkrétní metodu, pro kterou kód požadujete: `getNextPoint(final BoardModel board)`, a zmiňte, že má vracet objekt typu `Optional<BoardPoint>`. Ujistěte *AI*, že třída `BoardModel` a další související třídy jsou k dispozici, aby mohla generovat logicky správný kód. Žádejte,

aby se *AI* zaměřila na strategii, která se aktivně snaží vyhrát (dosáhnout pěti symbolů v řadě) a zároveň efektivně blokovat soupeře. Požadavek by mohl vypadat například takto:

Vytvoř kód v jazyce Java 25 pro strategii hry Piškvorky (známé také jako GoMoku). Hraje se na čtvercové síti, kde vítězí hráč, který jako první vytvoří souvislou řadu pěti svých symbolů (vodorovně, svisle nebo diagonálně). Implementaci proved' ve třídě *FineBoardService*, která bude rozšiřovat přiloženou třídu *BoardService*. Tvým úkolem je naprogramovat metodu *getNextPoint(BoardModel board)*, přičemž strukturu dat a potřebné metody odvodíš z kódu rodiče. Algoritmus se má zaměřit na vítěznou strategii a současně na blokování soupeře. Javadoc piš v angličtině.

7.7.2 Závěrečné kroky a autorská práva

Po obdržení kódu je třeba jej **začlenit** do vaší třídy *FineBoardService*. Ne vždy bude kód od *AI* perfektní, proto je třeba jej **zkontrolovat**, **otestovat** a případně **opravit**. Jako programátor **nesete plnou odpovědnost** za funkčnost a správnost výsledného kódu. Je důležité si uvědomit, že kód generovaný umělou inteligencí může podléhat **autorským právům** dané *AI* platformy nebo může být považován za veřejně dostupný. Vždy si **pečlivě prostudujte licenční podmínky** platformy, kterou pro generování kódu používáte. V akademickém prostředí je dobrým zvykem **uvést zdroj** a nástroj, který vám pomohl kód vytvořit.



Obsah této kapitoly byl generován umělou inteligencí **Google Gemini** [36] a do knihy byl zařazen po drobné autorské korektuře. Herní strategie ve třídě *FineBoardService* byla generována stejným modelem, přičemž součástí dotazu byl i zdrojový kód třídy *BoardModel*. Připomínám, že odeslání libovolného kódu do *AI* nástrojů znamená jeho poskytnutí třetí straně.

7.7.3 Automatizované testy

Když napíšeme nový algoritmus, je užitečné ověřit, že **funguje** podle našeho očekávání. Pokud máme k dispozici grafické rozhraní, nabízí se vyzkoušet algoritmus prostřednictvím tohoto rozhraní. V případě velkého množství tříd je však pracné kontrolovat všechny třídy opakovaně, přitom chybu lze zanést kdykoli později při nějaké zdánlivě banální úpravě. Z tohoto důvodu se osvědčilo psát programový kód, který správnou činnost vybraných algoritmů zkontroluje při každém sestavení projektu. Testovací kód tvoří opět *Java třídy* uložené ve struktuře projektu. Konvence pojmenování testovací třídy se odvozuje od testované třídy, pro odlišení se připojuje přípona **Test**. Testovací třídy bývají ve stejném *Java* balíčku, protože však nemá význam přikládat je do výsledné aplikace, jsou uloženy v oddělené adresářové struktuře. Při takzvaných **jednotkových testech** (anglicky

unit tests) se eliminuje závislost na službách jiných tříd a externích systémů – včetně síťových přenosů. Podívejme se na ukázkou jednoduchého testu využívajícího knihovnu *jUnit* [24].

Zdrojový kód 84. Ukázka testu metody *getPointIndex()*

```
@Test
public void testPointIndex() { ❶
    int x = 1;
    int y = 2;
    BoardPoint point = new BoardPoint(x, y); ❷
    BoardModel board = new BoardModel(3, 3, null);
    int index = point.getPointIndex(board); ❸
    Assertions.assertEquals(7, index); ❹
}
```

- ❶ Metoda určená k testování je veřejná a označuje se anotací `@Test` z knihovny *jUnit*.
- ❷ Sestavíme objekt pro testování. V tomto případě je to bod na hrací desce určený souřadnicovým systémem.
- ❸ Test kontroluje přepočítání souřadnic na index. Protože metoda pro přepočítání potřebuje znát velikost hrací desky, vytvoříme ji na předchozím řádku.
- ❹ Sestavíme tvrzení, že bod na souřadnici (x=1, y=2) bude mít na desce o velikosti 3 x 3 pole pořadové číslo (index) 7. Při neshodě metoda vyhodí výjimku. Třída pochází z knihovny *jUnit*.

Testy tedy kontrolují, zda je chování třídy či metody v souladu s jejím popisem (ideálně v *JavaDoc*). Spouštění automatizovaných testů je podporováno nástrojem *Maven* (příkazem `mvn test`). Při výchozím nastavení *Maven* se testy spouštějí automaticky při každém sestavení projektu. Pro další studium lze využít internetový zdroj [24].

7.8 Verzování zdrojového kódu

Pište kód tak, jako by ho po vás udržoval násilnický psychopat, který zná vaši adresu.

— John F. Woods, herní vývojář

Při psaní libovolného dokumentu bývá užitečné udělat krok zpět. Jednoduše zahodíme chybu a vrátíme se k předchozí verzi. Pokud na dokumentu pracuje celý tým, chceme znát také důvod změny, jméno autora a datum zápisu. Práci potřebujeme také bezpečně zálohovat a navzájem sdílet. Při vývoji programového kódu je to podobné.

S výše uvedenými nároky nám pomáhá verzovací systém. Mezi nejlepší patří **Git**, který je rychlý, spolehlivý a je k dispozici zdarma. Podporuje ho většina pokročilých editorů (IDE) na všech moderních operačních systémech. Lze ho efektivně používat i bez internetového připojení.

Pro sdílení týmové práce se však bez serveru neobejdeme. Server si můžeme nainstalovat sami, pohodlnější je však využít cloudové služby. Takové služby ukládají změny do vlastních Git repozitářů a nabízejí k tomu různé doplňkové funkce. Mezi nejznámější produkty patří *GitHub*, *GitLab* nebo *BitBucket*. Základní využití je zdarma, firmy však využívají spíše placené tarify s pokročilými funkcemi.

Jak Git funguje? Je to software, takže začneme jeho instalací na lokální počítač. Při prvním připojení k **projektu** si musíme vzdálené úložiště nejdříve **naklonovat**. Ve zvoleném adresáři si Git vytvoří aktuální pracovní kopii souborů a ve skrytém podadresáři `.git` založí databázi s celou historií změn. Do takové databáze můžeme začít ukládat svoji práci. Jakmile je hotová, upravené soubory přidáme do **změnové sady** a společně s komentářem uložíme do lokální databáze. Odesláním změny na server ho zpřístupníme ostatním.



Přestože je formát ukládání velmi efektivní, u rozsáhlých projektů může změnová databáze zabírat významné místo na disku.

Pokud bychom měli v kódu skrytou chybu, mohli bychom rozbít projekt i všem ostatním. Z toho důvodu budeme změny v Gitu ukládat do odděleného prostoru, kterému se říká **větev** (anglicky *branch*). Svoji větev si vytvoříme (zpravidla z posledního stabilního stavu) a dáme jí vhodný název. Ten by měl vyjadřovat její význam; například téma změny nebo číslo úkolu. Jakmile je práce hotová, novou větev nahrajeme na server a můžeme požádat kolegy o kontrolu změn. Takovému procesu kontroly se říká *code-review*. Pokud je vše v pořádku, změny ze své větve sloučíme do té původní a pracovní větev odstraníme.



Z vytvořené větve lze odklánět další větve, jejich počet je prakticky neomezený. Pokud si naklonujeme nový Git repozitář, výchozí větev se zpravidla nazývá **main**. Název výchozí větve je však pouze konvence a správce ji může změnit.

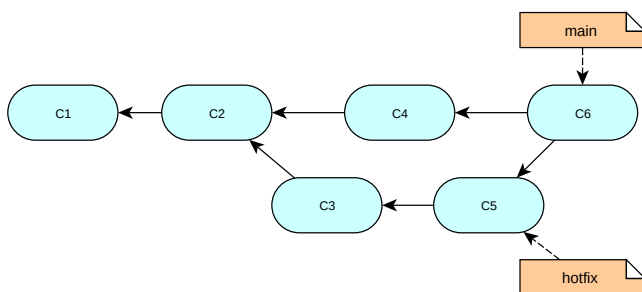
Základní pojmy v Gitu:

- **Commit**: zápis jedné ucelené dávky změn do historie projektu. Každý *commit* má své unikátní číslo zvané *hash*.
- **Hash**: unikátní identifikátor (otisk) *commitu*. Funguje podobně jako otisk prstu člověka. Uvádí se v hexadecimálním formátu, například `a1b2c3d...`.
- **Branch** (česky *větev*): oddělený prostor pro experimenty nebo vývoj nových

funkcí. Například **main** (hlavní větev) nebo **feature** (vývojová větev).

- **Push**: operace, která **protlačí** naše lokální commity na vzdálený server (např. *GitHub*).
- **Pull**: operace, která **stáhne** nejnovější commity ze serveru a sloučí je do naší lokální kopie.
- **Merge**: proces, který spojí dvě různé větve do jedné.
- **Squash**: proces, který spojí několik dílčích commitů do jednoho. Udrží historii přehlednou, ale riskuje ztrátu detailů a problémy při spojování větví.
- **Cherry-pick**: přenesení pouze **jeden** konkrétní commit z jiné větve.
- **Head**: ukazatel, který směřuje na poslední commit aktuálně vybrané větve.

Při sloučení dvou větví (operace *merge*) může nastat konflikt. Ten vzniká například tehdy, když stejný řádek kódu obsahuje v obou větvích různé změny. Pro řešení těchto konfliktů je výhodné využít grafické nástroje v editorech IDE. Pomocí takových editorů lze však provádět většinu rutinních operací Gitu.



Obrázek 40. Sloučení změn

Operace označené jmenovkami *C1* až *G6* znázorňují jednotlivé zápisy změn (anglicky *commit*) v chronologickém pořadí. Šipka s plnou čarou ukazuje na předchozí změnu (rodiče). Názvy obou větví jsou znázorněny v obdélníku s ohnutým rohem. Po *commitu C2* se vývoj rozdělil do dvou samostatných větví. Ve větvi **hotfix** probíhala urgentní oprava chyby, zatímco větev **main** pokračovala ve svém původním vývoji. Ke sloučení obou větví (příkazem *merge*) došlo v *commitu C6*.



S Gitem lze pracovat i pomocí příkazové řádky, což je výhodné například pro automatizaci pomocí skriptů. Podrobnější informace jsou například zde [\[39\]](#).

8 Úvod do databází a práce s daty

Data jsou nová ropa.

— Clive Humby, britský matematik

Některé aplikace potřebují trvalé (perzistentní) úložiště pro velký objem dat. Uživatelé takových aplikací očekávají rychlé vyhledávání dat a prostředky pro jejich ukládání, úpravu a mazání. Všechna uložená data musí být konzistentní, což také znamená, že při ukládání datové struktury chceme uložit vše, nebo nic (typicky po neočekávané události). Data je třeba také chránit před neoprávněným přístupem a řešit jejich zálohování; požadavků by se jistě našlo více.

Dnes jsou rozšířené databáze, které organizují data do tabulek s pevnými schématy a které spravují jejich vzájemné relace a integritu. Takový typ se nazývá *Systém pro správu relačních databází* se zkratkou **RDBMS** (z anglického Relational Database Management System). Jejich popularita je založena na vysoké spolehlivosti (*ACID*) a na využívání jednotného dotazovacího jazyka *SQL*. Zkratka *ACID* představuje čtyři základní pilíře spolehlivosti databáze:

- **Atomicita** – Transakce se provede celá, nebo vůbec; nedovolí částečné uložení dat.
- **Consistency** (konzistence) – Databáze přechází z jednoho platného stavu do druhého a dodržuje všechna integritní omezení.
- **Izolovanost** – Souběžně běžící transakce se navzájem neovlivňují, dokud nejsou dokončeny.
- **Durability** (trvanlivost) – Jakmile je transakce potvrzena, data zůstanou bezpečně uložena i v případě výpadku proudu či pádu systému.

Databáze běží často jako samostatná aplikace na vzdáleném (nebo lokálním) počítači a komunikace probíhá síťovým spojením. Proto se někdy databáze označuje pojmem *databázový stroj*. Databáze však může běžet i na lokálním počítači, dokonce jen v operační paměti *RAM*. Příkladem vhodných produktů mohou být open-source databáze *PostgreSQL* či *MySQL*, případně komerční produkty společností *Oracle* či *Microsoft*. Nabídka produktů na trhu je poměrně široká.

8.1 Základní pojmy relačních databází

Data se ukládají do *tabulek* (anglicky *table*). Každá *tabulka* má datovou strukturu, která je popsána sloupci (anglicky *column*), kde každý sloupec má svůj *datový typ*. Pro zamezení duplicit a zajištění integrity dat mohou mít sloupce omezení (anglicky

constraint). Pro rychlé vyhledávání může mít jeden či více sloupců definovaný *index*, což je pomocná datová struktura, kterou si databáze udržuje ve své režii. Unikátní *index* funguje zároveň jako *constraint* (omezení unikátnosti). Data se ukládají do *řádků* (anglicky *record* nebo *row*). Jeden sloupec databázové tabulky (nebo i více) může poskytovat jedinečný identifikátor řádku. Pokud má takový sloupec unikátní *index*, mluvíme o *primárním klíči*. Aby databáze mohla kontrolovat hodnotu cizího klíče (zda odpovídá *primárnímu klíči* v **cizí** tabulce), je třeba tento *vztah* deklarovat již při vytváření sloupce.

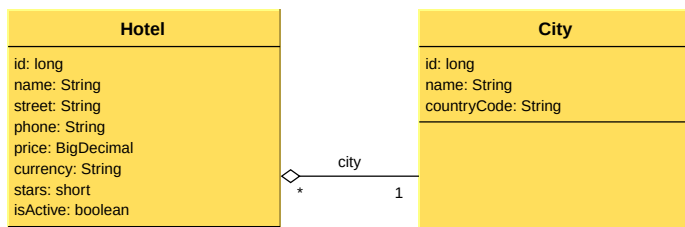


Je dobré vědět, že každý index zabírá místo na disku a jeho aktualizace mírně zpomaluje zápis.

Dvě *tabulky* mezi sebou mohou mít *vztah* (anglicky *relation*). Pokud má záznam v tabulce vazbu na více řádků cizí tabulky, mluvíme o vazbě typu **1:N**. Použití databáze si ukážeme na příkladu evidence **hotelů**. Zvažme uspořádání, kde jedno město může mít více hotelů (vazba **1:N**), ale jeden hotel může mít vazbu právě na jedno město (vazba **N:1**). V relačních databázích se vazba realizuje přidáním pomocného sloupce, který obsahuje hodnotu *primárního klíče* (zpravidla) jiné tabulky. Takovému sloupci se pak říká *cizí klíč*. Setkat se můžeme také s vazbou typu **M:N**. Například, jeden host může mít více rezervací v různých hotelech a jeden hotel může mít více hostů. V relační databázi se taková vazba řeší vložením pomocné vazební tabulky, která obsahuje cizí klíče na obě hlavní tabulky. Požadovaný vztah se tak rozloží na dvě vazby typu **N:1**.

8.2 Relace a entity

Zmínili jsme dvě databázové *tabulky* pro hotely a města s vazbou N:1. Protože s hotely chceme pracovat v prostředí Javy, připravíme si dvě entity, do kterých můžeme databázové záznamy kopírovat.



Obrázek 41. Entity hotelu

Popis položek diagramu:

- **Hotel** – objekt této třídy bude obsahovat data jednoho řádku tabulky **hotel**.

- `City` – objekt této třídy bude obsahovat data z jednoho řádku tabulky `city`.
- `id` – primární klíč mapovaný na sloupec `id`.
- `name` – název hotelu (či města).
- `city` – tento atribut bude v databázi realizován sloupcem `city_id`, který označíme jako *cizí klíč* do tabulky `city`.
- `isActive` – sloupec typu boolean umožňující logicky vyjmout řádek z vyhledávání bez jeho smazání.
- `countryCode` – kód země, který by v reálné aplikaci pravděpodobně odkazoval do samostatného číselníku zemí.

8.3 Jazyk SQL aneb jak mluvit s databází

Požadavky na dotazování a manipulaci dat se popisují jazykem *SQL* (anglicky *Structured Query Language*). Jazyk *SQL* je standardem, který byl naposledy aktualizován v roce 2023 pod označením *ISO/IEC 9075:2023*. V této kapitole si ukážeme jen základní možnosti jazyka, na další studium lze navázat například zde [\[38\]](#).



Dodavatelé databází implementují standard, ale často přidávají vlastní dialekty, které nejsou přenositelné.

Mezi základní operace pro práci s daty patří **vložení, čtení, aktualizace a mazání**. Soubor těchto čtyř operací se označuje zkratkou *CRUD* (z anglických sloves: *Create, Read, Update, Delete*). V jazyce *SQL* se tyto operace realizují příkazy **INSERT**, **SELECT**, **UPDATE** a **DELETE**. Abychom však mohli pracovat s daty, musíme mít nejdříve definovanou databázovou strukturu. Názvy databázových struktur zpravidla nerozlišují malá a velká písmena, a pokud se názvy skládají z více slov, oddělují se běžně podtržítkem. *SQL* rozlišuje velikost písmen pouze u textových hodnot v uvozovkách (v některých případech). Pro přehlednost budou klíčová slova jazyka *SQL* psána (v této knize) velkými písmeny a ostatní názvy (tabulky, sloupce, indexy) budou psány písmeny malými. Jednořádkové komentáře v kódu se uvádějí zpravidla dvěma znaky pomlček `--`.

8.3.1 Tvorba databázové tabulky

Začneme příkazem pro založení tabulky hotelů. Seznam sloupců je uzavřen v kulatých závorkách a oddělovačem je čárka. Mezery v *SQL* příkazech mohou obsahovat i zalomení řádků, což budeme pro přehlednost využívat. Předpokládejme, že tabulka měst s názvem `city` již existuje.

```
CREATE TABLE hotel ( ❶
    id BIGINT PRIMARY KEY AUTO_INCREMENT, ❷
    name VARCHAR(255) NOT NULL, ❸
    city_id BIGINT NOT NULL, ❹
    street VARCHAR(255) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    price DECIMAL(19, 4) NOT NULL, ❺
    currency CHAR(3) NOT NULL DEFAULT 'USD', ❻
    stars SMALLINT NOT NULL, ❼
    is_active BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT fk_hotel_city FOREIGN KEY (city_id) REFERENCES city(id), ❽
    CONSTRAINT chk_stars CHECK (stars BETWEEN 1 AND 5) ❾
);
CREATE UNIQUE INDEX idx_hotel_name ON hotel(name, city_id); ❿
```

- ❶ Za příkazem založení nové tabulky následuje její název.
- ❷ Primární klíč typu `BIGINT` koresponduje s typem `long` v Javě. Klíčové slovo na konci značí, že databáze bude hodnoty přiřazovat automaticky.
- ❸ Datový typ `VARCHAR` koresponduje s typem `String`. Číslo v závorce udává maximální délku textu. Sloupce s frází `NOT NULL` neumožňují zápis prázdné (`null`) hodnoty.
- ❹ Sloupec realizuje vazbu na záznam v tabulce `city`.
- ❺ Typ `DECIMAL(19, 4)` se mapuje na javový typ `BigDecimal`. První číslo uvádí celkový počet cifer (včetně desetinných míst), druhé počet desetinných míst.
- ❻ Typ `CHAR(3)` ukládá plnou délku textu. Využívá se u krátkých kódů. Za typem sloupce lze deklarovat výchozí hodnotu pomocí `DEFAULT`.
- ❼ Typ `SMALLINT` ukládáme do Java typu `short`.
- ❽ Sloupci `city_id` přiřadíme **omezení** na cizí klíč odkazující do tabulky `city`.
- ❾ Omezení `CHECK` zaručí, že číslo bude pouze v rozmezí 1 až 5.
- ❿ Index garantuje unikátní kombinaci názvu hotelu v rámci jednoho města.

SQL příkazy pro zakládání a úpravu struktury se označují zkratkou *DDL* (z anglického *Data Definition Language*).

8.3.2 Vložení řádku

Při vložení řádku do tabulky `hotel` předpokládejme, že v tabulce `city` již existuje město s identifikátorem 1.

Zdrojový kód 86. Příklad vložení jednoho hotelu.

```
INSERT INTO hotel  
(name, city_id, street, phone, price, currency, stars) ❶  
VALUES  
( 'Grand', 1, 'Tor', '123456789', 55, 'USD', 4) ❷
```

- ❶ První závorky obsahují seznam sloupců, do kterých vkládáme data. Vynechat lze sloupce s **výchozí** hodnotou, s povolenou prázdnou hodnotou, nebo *primární klíč* s generovanou hodnotou.
- ❷ Hodnoty musí být ve stejném pořadí jako sloupce; texty se v *SQL* ohraničují jednoduchými uvozovkami.

8.3.3 Čtení řádku

Pro vyhledávání řádků používáme příkaz **SELECT**. V této kapitole si popíšeme dotaz, který vyhledá prvních deset aktivních hotelů ve městech začínajících na zadaný text.

Zdrojový kód 87. Vyhledání záznamů.

```
SELECT hotel.id, hotel.name, hotel.street, hotel.stars, city.name ❶  
FROM hotel ❷  
JOIN city ON city.id = hotel.city_id ❸  
WHERE is_active = true ❹  
AND city.name LIKE 'Prague%' ❺  
ORDER BY hotel.name, city.name ❻  
LIMIT 10 ❼  
OFFSET 1; ❽
```

- ❶ Seznam sloupců, jejichž data chceme získat. Lze použít i hvězdičku ***** pro všechny sloupce, jejich pořadí pak určí databáze.
- ❷ Určení primární tabulky, ze které data bereme.
- ❸ Fráze **JOIN** připojí další tabulku na základě společného klíče. Pokud by se záznam připojené tabulky nenašel, nezobrazí se ani záznam primární tabulky. Existují však modifikátory (například **LEFT JOIN**), které umožňují vypsát záznamy i v případě, že k nim neexistuje odpovídající protějšek. Opakováním fráze lze připojit postupně tabulky další.
- ❹ Za **WHERE** uvádíme podmínky. Lze je spojovat operátory **AND** nebo **OR**. Vyšší prioritu má **AND**, změnu lze vynutit kulatými závorkami.
- ❺ Operátor **LIKE** slouží k porovnání textových vzorů. Zástupný symbol **%** reprezentuje libovolné množství znaků. Vyhledávání se zástupným symbolem na konci řetězce efektivně využívá index na sloupci **name**, což zaručuje velkou

rychlost i při velkém objemu dat. Pokud parametr žádný symbol **neobsahuje**, dotaz vyžaduje přesnou shodu, pro kterou je vhodnější použít operátor `=`. Chování operátoru `LIKE` (z hlediska rozlišování velikosti písmen) závisí na nastavení *collation* konkrétní databáze.

- ⑥ Fráze `ORDER BY` slouží pro řazení výsledků. Opačné pořadí lze vynutit přidavkem `DESC` (z anglického slova *descending*), který se uvádí za názvem sloupce.
- ⑦ Omezení počtu vrácených řádků.
- ⑧ Určení, od kterého řádku se má výpis zobrazit, je pro efektivní stránkování nezbytné.

Povinné fráze jsou `SELECT` a `FROM`, fráze `WHERE` je silně doporučená. Kromě základních příkazů nabízí *SQL* také **agregační funkce** pro výpočty nad více záznamy. Tyto funkce se často kombinují s klauzulí `GROUP BY`, která seskupuje řádky podle společného znaku – například můžeme zjistit počet hotelů pro každé město zvlášť.

- `SUM` – vypočítá celkový součet hodnot ve sloupci (např. tržby).
- `COUNT` – vrátí počet řádků (např. počet hotelů v databázi).
- `AVG` – vypočítá průměrnou hodnotu (např. průměrná cena za pokoj).

8.3.4 Změna záznamu

Další příklad ukazuje, jak změnit hodnocení hotelu nastavením počtu hvězdiček pomocí primárního klíče. Předpokládejme, že máme k dispozici primární klíč (unikátní databázový identifikátor) hotelu, který chceme upravit.

Zdrojový kód 88. Změna hodnoty sloupce.

```
UPDATE hotel
SET stars = 5 ①
WHERE id = 1; ②
```

- ① Za klíčovým slovem `SET` následuje přiřazení nové hodnoty. Pokud upravujeme více sloupců, zápisy se oddělují čárkou. Za rovnítkem lze uvést výraz; například zvýšení hodnocení o hvězdičku by šlo popsat výrazem: `hotel.stars + 1`. Syntaxi výrazů lze studovat například zde [\[38\]](#).
- ② Podmínka určuje, které řádky se mají změnit.

8.3.5 Mazání řádku

Záznam hotelu můžeme smazat následovně:

```
DELETE FROM hotel  
WHERE id = 1;
```

Při mazání řádku databáze kontroluje integritu. Smazání hotelu selže, pokud na něj odkazují jiné záznamy a není nastaveno *kaskádové mazání*, které odstraňuje i řádky s cizím klíčem. Alternativou je tzv. **logické mazání** nastavením příznaku `is_active=false`. Logické mazání předpokládá, že všechny *SQL* dotazy aplikace budou tento atribut respektovat.

8.3.6 Databázové transakce

Transakce zajišťuje, že se skupina příkazů provede jako celek, tedy *atomicky*. Pro řízení transakcí se používají *SQL* příkazy:

- **BEGIN** – zahájení transakce.
- **COMMIT** – trvalé potvrzení změn.
- **ROLLBACK** – zrušení všech změn provedených od začátku transakce.

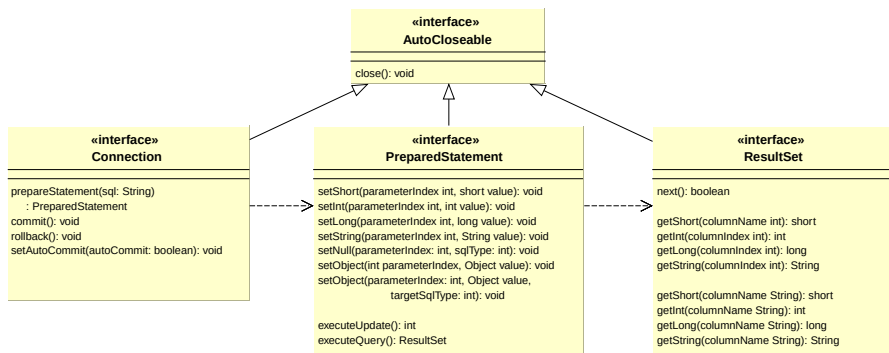
8.4 Programové rozhraní Javy

Pro komunikaci s databází v jazyce Java potřebujeme:

1. Znat programové rozhraní (API) knihovny, pomocí kterého budeme s databází komunikovat.
2. Mít implementaci (ovladač/driver) – knihovnu pro konkrétní databázový produkt.

8.4.1 Základní rozhraní JDBC

Standardní rozhraní v Javě se nazývá **JDBC** (*Java Database Connectivity*). Všechna jeho hlavní rozhraní podporují automatické zavírání zdrojů díky rozhraní `AutoCloseable`. Mezi nevýhody JDBC patří nutnost psát kód pro kopírování dat z JDBC objektů do aplikačních Java objektů (a zpět).



Obrázek 42. Diagram vybraných tříd JDBC

1. **Connection** – realizuje spojení s databází. Obsahuje parametry spojení a řídí transakce. Protože vytváření databázových spojení bývá náročné na výkon, v praxi se tyto objekty často recyklují v tak zvaném **connection poolu**.
2. **PreparedStatement** – slouží k bezpečnému provádění dotazů s parametry.
3. **ResultSet** – obsahuje výsledná data z dotazu **SELECT** a umožňuje jimi procházet.

Vkládání parametrů SQL dotazu přes rozhraní *JDBC* chrání aplikaci před útokem typu **SQL Injection**. To je název pro nebezpečnou techniku, kterou útočník podvrhne škodlivý kód do vstupních polí aplikace. Místo **názvu** hotelu zkusí třeba vložit: `' OR 1=1; DROP TABLE hotel; --'`. Pokud by aplikace parametry spojovala jako běžný text, databáze by tento příkaz provedla. Třída **PreparedStatement** riziko eliminuje oddělením struktury dotazu od dat. Její použití je proto nezbytné pro ochranu před zmíněným útokem.

8.4.2 Pokročilé rozhraní JPA

JPA je pokročilé programové rozhraní pro práci s relačními databázemi. Jedním z principů je deklarativní styl programování, kde popisujeme spíše co potřebujeme, než jak to provést. Výhodou je snadnější převod databázových dat na Java objekty (a zpět) a možnost automatického generování běžných *SQL* příkazů. *JPA* má v sobě zabudované mechanismy pro řízení databázových transakcí. Nabízí automatické zakládání databázových tabulek (řízené anotacemi Java entit) včetně rozdílového doplňování struktury. Populární implementaci rozhraní *JPA* nabízí framework *Hibernate*.

I když je *JPA* dnes považováno za průmyslový standard pro perzistenci dat v Java aplikacích, má své kritiky. Ti upozorňují na skrytou složitost, která komplikuje hledání příčiny nečekaných pádů či zpomalení odezvy. Porozumět správně **relacím** trvá déle než se naučit psát čisté *SQL* dotazy. Generované *SQL* dotazy nejsou vždy

efektivní a mohou zbytečně zatěžovat databázi. *JPA* může někdy generovat stovky *SQL* dotazů tam, kde by stačil jediný příkaz s frází **JOIN**. Pro menší projekty může být *JPA* příliš těžkopádná.

8.4.3 Jak usnadnit práci s JDBC

Domnívám se, že pro pochopení principů práce s databází je lepší se *JPA* zpočátku vyhnout. Práci s *JDBC* by však mohla usnadnit nějaká jednoduchá obálka (anglicky *wrapper*, technicky lépe *facade*) nad třídou `PreparedStatement`. Takové řešení by mohlo nabízet přívětivější API, přičemž úkoly by se nadále delegovaly na *JDBC*. Co se dá zlepšit?

- Vkládání *SQL* parametrů pomocí pojmenovaných značek ve stylu *JPA*. Parametry se tedy budou vkládat do míst označených alfanumerickým textem začínajícím dvojtečkou.
- Zpracování výstupních dat by usnadnil objekt typu `Stream<ResultSet>`.
- Kratšímu zápisu by pomohla podpora řetězového volání jednotlivých metod (do jediného příkazu).
- Přehlednost kódu může zlepšit absence kontrolovaných výjimek.
- Při ladění se někdy hodí kontrola *SQL* příkazu včetně přiřazených parametrů.

Vhodných nástrojů by se našlo více. My se dále zaměříme na třídu `SqlParamBuilder` z volně dostupného frameworku *Ujorm* [23]. Jedná se o stručnou implementaci zapsanou prakticky v jediné třídě.



Objekt `SqlParamBuilder` je třeba po použití uzavřít, čímž se uzavře i vnitřní *JDBC* objekt. Stačí ho uzavřít do bloku `try`. Uzavření nemá vliv na databázové spojení.

8.5 Hledání hotelů v jazyce Java

Příklad databázové aplikace v jazyce *Java* si ukážeme na *HTML* stránce pro vyhledávání hotelů. Záznamy jednotlivých hotelů budeme zobrazovat v řádcích standardní tabulky. Nad tabulku přidáme filtr podle **názvu**, **města** a čísla **stránky**. Pokud prohlížeč podporuje *JavaScript* nezbytný pro *AJAX*, změny filtru budeme potvrzovat automaticky. V tom případě tlačítko **Find** ztrácí význam a lze ho skrýt. Veškerá data hotelů jsou smyšlená a byla vygenerována *AI* modelem *Google Gemini* [36].

Hotel:
City:
Page:

Hotel	City	Street	Phone	Price	Currency	Stars
Grand Hotel U Tří Pštrošů	Prague	U Lužického semináře 15	+420211123457	180	USD	★★★★
Hotel Zlatý lev	Prague	Opletalova 16	+420211123456	125	USD	★★★
The Emblem Prague	Prague	Platněřská 11	+420211123458	310	USD	★★★★★

Obrázek 43. Formulář pro vyhledávání hotelů.

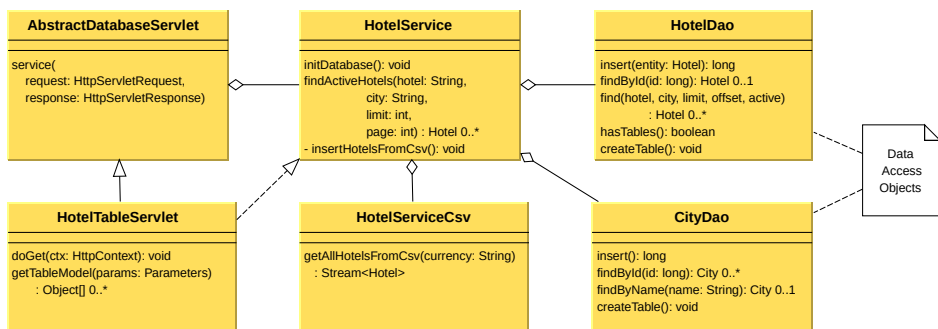
Dotazy *SQL* budeme směřovat na databázi s názvem *H2*. Jedná se o open-source projekt vytvořený v jazyce *Java*. Tuto databázi nemusíme instalovat. Navíc databáze podporuje běh v režimu **in-memory**, ve kterém budeme vytvářet tabulky při každém spuštění. To zjednoduší případné úpravy struktury při testování. Pro ignorování velikosti písmen v celé databázi přidáme na konec *URL* adresy databázového spojení text: `;IGNORECASE=TRUE`.

8.6 Diagram tříd

Při tvorbě servletu vyjdeme z příkladu pro zobrazení čísel v tabulce, který jsme popsali v kapitole [Tabulka náhodných čísel](#). Nový servlet pojmenujeme například `HotelTableServlet`. Hlavní změna spočívá v implementaci metody `getTableModel()`, která bude číst data přímo z databáze.

Kód servletu má řešit pouze grafickou podobu *HTML* stránky, proto logiku aplikace zapíšeme do servisní třídy s názvem `HotelService`. Implementace servisních tříd by však neměla obsahovat kód závislý na konkrétním externím zdroji. Z toho důvodu přesuneme *SQL* příkazy do tříd určených výhradně pro získávání dat. Takové třídy se označují zkratkou *DAO*, která vychází z anglického termínu *Data Access Object*.

Pro efektivní využití databázového spojení je vhodné ho otevírat na co možná **nejkratší** dobu. Spojení si otevírá zpravidla servisní metoda, která ho potřebuje. Stejná metoda také odpovídá za jeho korektní uzavření. Moderní frameworky automatizují správu spojení pomocí **anotací**. Protože v našem případě každý HTTP request vyvolá jen krátký databázový dotaz, pro zjednodušení přesuneme správu databázového spojení do abstraktního servletu.



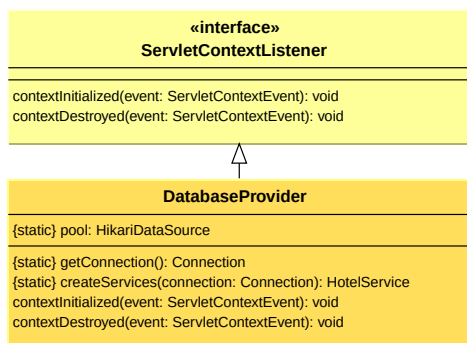
Obrázek 44. Diagram vybraných tříd

- **AbstractDatabaseServlet** – tato třída poskytuje databázové spojení. Využívá k tomu pomocnou třídu **DatabaseProvider**, kterou zmíníme dále.
- **HotelServlet** odpovídá za uživatelské rozhraní, tedy za vykreslení HTML stránky a zpracování parametrů. Seznam nalezených hotelů získáme voláním metody **findActiveHotels()** na třídě **HotelService**. Referenci na tuto službu nám připravila rodičovská třída servletu.
- **HotelService** poskytuje seznam hotelů podle zadanych parametrů. Provádění databázových příkazů však deleguje na **DAO** objekty. Metoda **initDatabase()** kontroluje, zda v databázi existují potřebné tabulky, a v případě potřeby zajistí nápravu voláním metody DAO objektu. Po vytvoření nových tabulek se rovnou nahrají testovací data, která poskytuje třída **HotelServiceCsv**. Tato třída načítá seznam hotelů ze zdrojového textového souboru s názvem **Hotels.csv**. Struktura CSV souboru připomíná tabulku, kde první řádek obsahuje komentář a druhý seznam názvů sloupců oddělený znakem *vislé čáry* (anglicky *pipe*). Další řádky obsahují samotná data oddělená stejným znakem jako v hlavičce. V praxi se používají různé oddělovače, velmi často například středník.
- **HotelDao** a **CityDao** – realizují procesy získávání dat, což jsou technické detaily, které by jinak komplikovaly srozumitelnost algoritmů v servisní vrstvě. Případná změna datového zdroje by se implementovala výhradně v těchto třídách, zatímco algoritmy v servisní vrstvě zůstanou nedotčené. Podrobnější popis vybraných metod najdete v další kapitole.

8.7 Životní cyklus databázového spojení

Na počátku **HTTP** dotazu otevřeme databázové spojení a na jeho konci ho zase uzavřeme. Obě události ošetříme v rodičovském servletu překrytím metody **service()**. Kód této metody bude mít tři části: nejprve získáme databázové **spojení**, poté zavoláme původní metodu rodiče (pomocí klíčového slova **super**) a nakonec

spojení uzavřeme. Aby bylo možné objekt spojení poskytnout dalším třídám, musíme ho někde uschovat. Nelze ho však ukládat do běžného atributu servletu, protože jeden objekt servletu může obsluhovat více dotazů současně. Každý dotaz přitom potřebuje mít svůj **vlastní**, jinak může docházet k chybám a neočekávanému chování. Objekt spojení však můžeme uložit do statického atributu typu `ThreadLocal`. Tento typ garantuje, že každý HTTP dotaz (vlákno) bude mít přístup výhradně ke svému vlastnímu spojení. Protože však servlet nebude pracovat s spojením přímo, uložíme do `ThreadLocal` atributu rovnou objekt služby `HotelService`. Připojíme tam i závislosti, které databázové spojení potřebují. Řešení pro správu spojení pak přesuneme do abstraktního rodiče `AbstractDatabaseServlet`. Oddělíme tím technickou správu databáze do samostatné třídy, kterou pak můžeme snadno využít v jiném servletu.



Obrázek 45. Třída poskytující databázové spojení

Pro lepší přehlednost přesuneme kód na získání spojení do třídy s názvem `DatabaseProvider`, kterou jsme zmínili dříve. Tato třída implementuje rozhraní z knihovny `Servlet API` a je označena anotací `@WebListener`. Metody tohoto rozhraní volá servlet kontejner automaticky při svém spuštění a ukončení. Třída tedy při startu připraví objekt databázového spojení, vytvoří databázovou strukturu a nahraje do ní testovací data. Přitom sestaví všechny nezbytné servisní objekty. Abychom se přiblížili reálnému produkčnímu prostředí, využijeme knihovnu `HikariCP`, která umí databázová spojení efektivně spravovat. Objekt z té knihovny najdeme ve statické proměnné `pool`.



Při práci s databázovým spojením je důležité, aby se spojení po použití uzavřelo za každých okolností. Tedy i v případě, že během zpracování dotazu dojde k chybě. K tomu účelu slouží konstrukce `try-with-resources`, která na konci svého bloku automaticky uzavírá všechny zdroje uvedené v hlavičce. Bez tohoto ošetření by mohlo dojít k postupnému vyčerpání všech dostupných spojení, což by vedlo k zablokování či pádu celé aplikace.

8.8 Čtení databáze v Java objektech

Podívejme se na Java kód pro vyhledávání hotelů ve třídě `HotelDao`.

Zdrojový kód 90. Metoda třídy `HotelDao`.

```
public List<Hotel> find(String hotel, String city,
    int limit, int offset, boolean active) {
    String sql = "" ❶
        SELECT hotel.id
            , hotel.name
            , hotel.street
            , hotel.phone
            , hotel.price
            , hotel.currency
            , hotel.stars
            , hotel.is_active
            , city.name as cityName
        FROM hotel
        JOIN city ON city.id = city_id
        WHERE is_active = :active ❷
            %s %s ❸
        ORDER BY name, cityName
        LIMIT :limit ❹
        OFFSET :offset;
    ""formatted( ❸
        hotel.isEmpty() ? "" : "AND hotel.name LIKE :hotel",
        city.isEmpty() ? "" : "AND city.name LIKE :city");
    try (SqlParameterBuilder builder = new SqlParameterBuilder(dbConnection)) { ❺
        return builder.sql(sql) ❻
            .bind("active", active) ❼
            .bind(!hotel.isEmpty(), "hotel", escape(hotel) + "%") ❽
            .bind(!city.isEmpty(), "city", escape(city) + "%")
            .bind("limit", limit)
            .bind("offset", offset)
            .streamMap(resultSet -> map(resultSet, "cityName")) ❾
            .toList(); ❿
    }
}
```

- ❶ Do proměnné `sql` si připravíme potřebný SQL příkaz. S výhodou využijeme víceřádkový literál typu `String`.
- ❷ Umístění hodnoty parametru (SQL dotazu) označíme alfanumerickým výrazem začínajícím dvojtečkou.
- ❸ V době psaní kódu ještě nevíme, které hodnoty filtru uživatel použije. Část textového SQL příkazu proto bude nutné sestavit dynamicky podle okolností. Pokud bude textová hodnota prázdná, podmínku filtru budeme ignorovat. Finální SQL šablony sestavíme metodou `formatted()`, která bude vkládat dílčí podmínky na značky s výrazem `%s`. Pokud je hodnotou prázdný text, do SQL příkazu vložíme také prázdný text.

- ④ Vložení parametrických hodnot pro fráze `LIMIT` a `OFFSET` se od ostatních neliší.
- ⑤ Na tomto řádku získáme instanci třídy `SqlParameterBuilder`, která funguje jako fasáda kolem `JDBC` třídy `PreparedStatement`. Protože objekt je vhodné po použití uzavřít, proměnnou `builder` deklarujeme v hlavičce bloku `try`. Protože kód pro vytváření builderu budeme využívat častěji, přesuneme ho do samostatné metody `builder()` abstraktního předka.
- ⑥ Následuje přiřazení parametrů. Každá metoda vrací svůj vlastní objekt, a tak lze metody vzájemně řetězit. První metodou `sql()` vložíme do objektu připravený `SQL` příkaz.
- ⑦ Metoda `bind("active", active)` doručí bezpečně **hodnotu parametru** na místo označené výrazem `:active`. Stejný formát značek používá také `JPA` a další databázové knihovny. Pokud `SQL` příkaz nemá parametry, vynecháme ho.
- ⑧ Varianta předchozího přiřazení hodnoty s podmínkou použití.
- ⑨ Metoda `map()` umí přepsat databázové hodnoty z objektu `ResultSet` do cílové entity. Implementaci si ukážeme dále. Přesunutím této logiky do samostatné třídy zajistíme lepší oddělení odpovědností a usnadníme si psaní testů.
- ⑩ Objekt typu `Stream<ResultSet>` poskytuje data jen po dobu, kdy je `builder` otevřený. Po uzavření pokus o čtení vyhodí výjimku. Protože se `builder` uzavře při opuštění bloku `try`, seznam hotelů převedeme včas do objektu `List`, který už závislý není. Alternativou by bylo otevřít a uzavřít `SqlParameterBuilder` už v servisní vrstvě. Pro pochopení principu se však zdá být stávající řešení vhodnější.

Algoritmus pro kopírování dat z jedné objektové struktury do druhé se někdy nazývá **mapování**. V následující metodě si ukážeme mapování z `JDBC` objektu `ResultSet` do naší entity `Hotel`.

Zdrojový kód 91. Přenos dat z `ResultSet`.

```
private Hotel map(ResultSet rs, String cityColumn)
    throws java.sql.SQLException { ①
    Hotel result = new Hotel();
    result.setId(rs.getLong("id")); ②
    result.setName(rs.getString("name"));
    result.setStreet(rs.getString("street"));
    result.setPhone(rs.getString("phone"));
    result.setPrice(rs.getBigDecimal("price"));
    result.setCurrency(rs.getString("currency"));
    result.setStars(rs.getShort("stars"));
    if (cityColumn != null) { ③
        result.setCity(new City());
        result.getCity().setName(rs.getString(cityColumn));
    }
    return result;
}
```

- 1 Protože zde pracujeme přímo s objektem knihovny *JDBC*, je třeba propagovat (či zachytávat) kontrolovanou výjimku.
- 2 Návratový objekt si připravíme v proměnné `result`. Do objektu pak postupně přepíšeme hodnoty z objektu `ResultSet`, který obsahuje metody pro čtení různých datových typů. Na sloupec se můžeme odkázat jeho databázovým názvem (což je přehlednější), nebo pořadovým číslem v *SQL* dotazu (což vede k rychlejšímu běhu). Obecně se doporučuje preferovat čitelnost kódu a výkon optimalizovat až v případě prokázaných problémů.
- 3 Metoda se používá pro dva mírně odlišné příkazy `SELECT`. V jednom je k dispozici navíc **název města**. Abychom nemuseli psát a udržovat dvě různé implementace mapování, tak přiřazení města podmíníme druhým pomocným parametrem s názvem toho volitelného sloupce. Pokud název databázového sloupce není definován, zápis se neprovede.

Specifikace *JDBC* poskytuje také API pro získání dat o struktuře databáze. Říkáme tomu *metadata*. Právě takové informace využívá i metoda `hasTables()`. Zjednodušená implementace kontroluje pouze přítomnost databázové tabulky `hotel`. Pokud tabulka chybí, budeme předpokládat, že chybí i druhá tabulka `city`. Práce s metadaty však patří mezi pokročilejší techniky, a tak případné zájemce jen odkážu na dokumentaci *JavaDoc*. Implementaci najdeme v příkladech.

8.9 Vkládání Java objektů do databáze

Připomeňme, že vkládání dat do databáze provádíme příkazem `INSERT` a kód najdeme ve třídách typu `DAO`. Podívejme se na komentovanou ukázkou.

Zdrojový kód 92. Java kód pro vložení hotelu.

```
public long insert(Hotel entity) {  
    String sql = ""  
        INSERT INTO hotel 1  
        ( name, city_id, street, phone, price, stars ) VALUES  
        ( :name, :cityId, :street, :phone, :price, :stars )  
        "";  
    try (SqlParamBuilder builder = builder()) { 2  
        builder.sql(sql)  
            .bind("name", entity.getName()) 3  
            .bind("cityId", entity.getCity().getId())  
            .bind("street", entity.getStreet())  
            .bind("phone", entity.getPhone())  
            .bind("price", entity.getPrice())  
            .bind("stars", entity.getStars())  
            .executeInsert(); 4  
        return builder.generatedKeys(resultSet -> resultSet.getLong(1))  
            .findFirst().get(); 5  
    }  
}
```

- ❶ Příkaz se od nativního *SQL* liší jen značkami parametrů.
- ❷ Využití metody pro získání builderu zpřehlední kód a usnadní případnou změnu sestavení na ostatních místech.
- ❸ Formu přiřazení hodnot už známe z příkazu *SELECT*.
- ❹ Metoda *execute()* by nám data také korektně vložila. Pokud však potřebujeme získat hodnotu databázového identifikátoru, je třeba zavolat metodu *executeInsert()*.
- ❺ Metoda *generatedKeys()* pak vrátí seznam přiřazených identifikátorů ve formátu *Stream<ResultSet>*. Při vkládání jednoho hotelu můžeme očekávat jeden identifikátor. Podle dokumentace *JDBC* ho najdeme na pozici 1. V případě prázdného Streamu by metoda *get()* vyhodila výjimku. Pro případ obecného použití tento kód přesuneme do samostatné metody abstraktního předka. Takovou metodu využije i třída *CityDao*.

Ostatní metody obou *DAO* objektů jsou psány ve stejném duchu, kód najdete v příložených příkladech.

8.10 Generická abstraktní třída

Aplikace pro vyhledávání hotelů obsahuje pro každou databázovou tabulku jednu *DAO* třídu. Reálná aplikace může pracovat s desítkami takových tabulek, a proto je vhodné přesunout obecné služby do společného předka. Dalším důvodem je sjednocení API podobných metod tak, aby měly stejný název i logiku. Příkladem může být vyhledávání databázového záznamu podle jeho primárního klíče nebo vkládání nového hotelu do databáze. V takovém případě však budeme potřebovat v abstraktní třídě pracovat s datovým typem, který zatím ještě neznáme. To je dobrá příležitost připomenout si *generické datové typy*, které tento problém elegantně řeší. Implementaci si vysvětlíme na zjednodušené ukázce abstraktní *DAO* třídy.

Zdrojový kód 93. Generická abstraktní DAO.

```
/** Common DAO object
 * @param <E> Entity type */
public abstract class AbstractDao<E> { ❶
    public abstract Optional<E> findById(long id); ❷
    public abstract long insert(E entity); ❸
}
```

- ❶ Generický typ třídy je reprezentován textem uvedeným za názvem třídy ve špičatých závorkách. Třída může mít i více generických typů, jejichž názvy se pak oddělují čárkou. Použitý znak *E* je odvozený od slova *Entita*. Význam generického typu je vhodné zdokumentovat v *JavaDoc*.

- ❷ Příklad použití generického typu jako návratové hodnoty metody.
- ❸ Příklad použití generického typu jako parametru metody.

Deklarace potomka generické třídy se liší zejména doplněním konkrétního datového typu.

Zdrojový kód 94. Použití generického rodiče.

```
public class HotelDao extends AbstractDao<Hotel> { ... } ❶
```

- ❶ Potomek třídy se odkazuje na abstraktního rodiče včetně konkrétního typu, který je uveden ve špičatých závorkách. Tři tečky zde zastupují tělo třídy, které pro přehlednost neuvádíme.

9 Jak psát dobrý kód

Každý hlupák umí napsat kód, kterému porozumí počítač. Dobří programátoři píší kód, kterému rozumějí lidé.

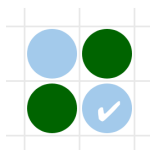
— Martin Fowler, autor moderních metod tvorby softwaru

Toto téma by vydalo na samostatnou knihu, přesto je vhodné zmínit, že dobrá aplikace vždy začíná kvalitním návrhem. Při návrhu tříd je užitečné mít na paměti, že náklady na samotný vývoj softwarové aplikace mohou tvořit jen malou část celkových nákladů na projekt. Větší část prostředků může spotřebovat údržba a implementace dodatečných požadavků, které probíhají ještě mnoho let po uvedení do produkčního provozu. Proto by měla mít přehlednost a srozumitelnost zdrojového kódu vysokou prioritu. Podívejme se na některá obecná doporučení:

1. Pišme komentáře ke třídám a metodám. Připomeňme, že *JavaDoc* by měl popisovat především to, **co** má *metoda* dělat. Konkrétní způsob implementace se považuje za implementační detail, který se v komentářích běžně neuvádí. Pokud nějaká **skupina** příkazů vyžaduje vlastní vysvětlující komentář, stojí za úvahu takový kód přesunout do **samostatné** metody a komentář přeměnit na *JavaDoc* této metody.
2. Preferujeme kratší metody s jasně vymezenou zodpovědností. Metody jedné třídy by měly pokrývat pouze jeden typ činnosti na své úrovni abstrakce. To znamená, že pro zpracování detailů je vhodnější volat metody jiných tříd.
3. Duplicitní zdrojový kód je nutné minimalizovat pomocí dědičnosti objektů, kompozice, programových knihoven nebo kombinací těchto přístupů. Literály se stejným významem (sémantikou) sdílíme pomocí konstant.
4. Pišme včas automatizované testy. Zejména *jednotkové testy* (anglicky *unit tests*) snižují riziko hromadění mnoha funkcí v metodě a mívají pozitivní vliv na dobrý návrh programového rozhraní.
5. Vyvarujme se předčasné optimalizace výkonu. K optimalizaci kódu je vhodné přistoupit až po ověření, že stávající kód je skutečně odpovědný za nepřiměřené zpomalení algoritmu. Předem doporučuji zvážit všechny výhody, rizika a náklady. Původní algoritmus by měl být pokrytý *jednotkovými testy*, které garantují zachování původního chování. Každá optimalizace, která přinese zhoršení čitelnosti zdrojového kódu, může být ve výsledku kontraproduktivní.

Zájemcům o toto téma doporučuji hledat na internetu pojem *návrhové vzory* [12]. Vývojářům grafických aplikací pak doporučuji nepřehlédnout architektonický návrhový vzor zvaný *Model-View-Controller* [13].

Pokud jste dočetli knihu až sem, přeji vám mnoho úspěchů při uplatňování získaných znalostí. Dovolte mi připomenout, že jakékoli postřehy, dojmy a technické poznámky k obsahu této knihy jsou vítány. Kontakt najdete na domovské stránce knihy [1]. Děkuji za ně.



Reference a zdroje pro další studium

Některé z uvedených zdrojů jsou v angličtině.

- [1] – Domovská stránka této knihy:
<https://jbook.ponec.net/cs3>
- [2] – Odkaz ke stažení příkladů ke knize:
<https://pponec.github.io/jbook/cs3/download-3.0.0.html>
- [3] – Java Language and Virtual Machine Specifications – Oracle © Corporation:
<https://docs.oracle.com/javase/specs/>
- [4] – Domovská stránka OpenJDK:
<https://openjdk.java.net/>
- [5] – Dokumentace programového rozhraní Java 25 ve formátu JavaDoc:
<https://docs.oracle.com/en/java/javase/25/docs/api/>
- [6] – TIOBE community index for measure of popularity of programming languages:
<https://www.tiobe.com/tiobe-index/>
- [7] – Apache Maven in 5 Minutes – The Apache Software Foundation:
<https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html>
Příklady byly odladěny na Maven, verze 3.
- [8] – Netbeans IDE (domovská stránka):
<https://netbeans.apache.org/>
- [8] – IntelliJ IDEA (domovská stránka):
<https://www.jetbrains.com/idea/>
- [9] – Knihovna volně šiřitelných obrázků:
<https://pixabay.com/>.
- [10] – Česká Wikipedie:
<https://cs.wikipedia.org/>
- [11] – List of Unicode characters:
https://en.wikipedia.org/wiki/List_of_Unicode_characters
- [12] – Návrhový vzor:
https://cs.wikipedia.org/wiki/N%C3%A1vrhov%C3%BD_vzor
- [13] – Architektonický návrhový vzor Model-View-Controller:
<https://cs.wikipedia.org/wiki/Model-view-controller>
- [14] – The Java 25 Stream API Tutorial:
<https://www.baeldung.com/java-8-streams>

- [15] – Question and answer site Stack Overflow:
<https://stackoverflow.com/>
- [16] – Webová stránka na prohledávání repozitáře Maven:
<https://search.maven.org/>
- [17] – Domovská stránka Linuxu Xubuntu:
<https://xubuntu.org/>
- [19] – Introduction to Java Primitives:
<https://www.baeldung.com/java-primitives>
- [20] – HTML Tutorial of w3schools.com:
<https://www.w3schools.com/html/>
- [21] – CSS Tutorial of w3schools.com:
<https://www.w3schools.com/css/>
- [22] – Java operators:
https://www.w3schools.com/java/java_operators.asp
- [23] – Domovská stránka Java knihovny Ujorm:
<https://ujorm.org/web>
- [24] – JUnit Tutorial for Beginners: Learn in 3 Days:
<https://www.guru99.com/junit-tutorial.html>
- [25] – Apache Software Foundation (ASF):
<https://www.apache.org/foundation/>
- [25] – Bootstrap, an open source toolkit for developing with HTML, CSS, and JS:
<https://getbootstrap.com/>
- [26] – Java Language Keywords:
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/_keywords.html
- [27] – Regulární výrazy v JavaDoc:
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/regex/Pattern.html>
- [28] – Virtualization application Docker®:
<https://www.docker.com/>
- [29] – Security-oriented, lightweight Linux distribution Alpine Linux:
<https://alpinelinux.org/>
- [30] – Wikipedia JSON (JavaScript Object Notation):
https://cs.wikipedia.org/wiki/JavaScript_Object_Notation
- [31] – Wikipedia CSS Selector:
<https://en.wikipedia.org/wiki/CSS#Selector>
- [32] – Wikipedia WEB colors:

https://en.wikipedia.org/wiki/Web_colors

- [33] – Guide to jlink:
<https://www.baeldung.com/jlink>
- [34] – Java Interfaces:
<https://www.baeldung.com/java-interfaces>
- [35] – Lambda Expressions and Functional Interfaces:
<https://www.baeldung.com/java-8-lambda-expressions-tips>
- [36] – AI Google Gemini:
{attr-attr-google-gemini-url}
- [38] – SQL Tutorial of W3schools:
<https://www.w3schools.com/sql/>
- [39] – Git book:
<https://git-scm.com/book/cs/v2>

Rejstřík slov

@

`@Override`, 83
`@param`, 46, 96
`@return`, 46, 96
`@WebServlet`, 82, 86

A

abstraktní metoda, 48
ACID, 139
AI, 134-135, 147
AJAX, 127
algoritmus, 15
algoritmus, 17
alokovat, 18
anotace, 82
`ApiElement`, 85
atomicky, 145
atributy, 28, 64
autoboxing, 33, 55
autoboxing, 97

B

balíček, 39, 45, 82, 110
Base64, 113
bit, 15
bitový součet, 118
bitový součin, 118
body, 65
bytecode, 14
bílé znaky, 55, 64, 67, 108, 127

C

CamelCase, 28, 63
checked exceptions, 44
`Class`, 114
class, 83
class diagram, 27
`Consumer`, 115
CSS, 108
CSS selektor, 67

D

data, 15, 17
datovod, 58
datový model, 25
datový typ, 19
datový typ objektu, 26
datově konzistentní, 39
dekrementace, 92
diagram tříd, 27
`Dockerfile`, 78
dvoukovou číselnou soustavu, 18
dědičnost tříd, 39

E

`Element`, 84-85
element, 63
elementy, 63
Emulátor terminálu, 75
emulátor znakového terminálu, 75
enumeration, 119
`Error`, 44
`Exception`, 44
extends, 83

F

file, 70
`final`, 41
font, 19
`for`, 91, 94

G

Garbage collector, 12
generické, 154
Generický datový typ, 49
GET, 79

H

hardware, 73
head, 65
HelloWorldServlet, 82

Hibernate, 146

hrana, 25

href, 70

HTML, 47, 65

HTML5, 65

HtmlElement, 85-86

http, 70

https, 70

HttpServlet, 79

HttpServletRequest, 81

HttpServletResponse, 81

I

IDE, 97

IllegalArgumentException, 44

IllegalStateException, 44

implementační detail, 156

implements, 83

indexu, 112

inkrementace, 92

instance třídy, 26

int, 96

integrovaná vývojová prostředí, 79

interface, 48, 83

IOException, 44

iterace, 17

J

Jakarta, 110

Java Development Kit, 11, 14

Java Runtime Environment, 12, 14

JavaDoc, 10, 46-47, 90, 96, 103, 126

Jetty, 79

jmenný prostor, 64

K

klient-server, 62

kompilátor, 14

kompilátor, 14

kompozice, 28

konstruktor, 29

kontejner, 78

kontrolované výjimky, 44

kořen, 25

kroky algoritmu, 16

L

lambda, 58

lambda, 131

lambda výraz, 115

link, 66

list, 25

literál, 20, 23

literál, 22, 86, 89

Logger, 114

logická hodnota, 20

M

Map, 57

Maven Wrapper, 74, 76

metoda, 17

metody, 17

model, 25

modifikátor, 38

modul, 74

multiplatformní, 12

N

nekontrolované výjimky, 44

null, 34, 37-38, 47, 53, 96, 106, 109, 115,
120, 123

NullPointerException, 44

návratový typ, 40

návrhové vzory, 157

název třídy, 28

názvy stylů, 68

O

Object, 41

objekt, 25, 85

objektové typy, 38

OpenJDK, 12

optimalizace, 17

OutOfMemoryError, 44

overriding, 41

P

package, 39
paměť, 18
paralelní zpracování, 17
pevný disk, 11
pole, 20, 38, 52, 71
polymorfismus, 41
pom.xml, 88
požadavek, 79
primitivní, 71
primitivní typ, 20, 33, 38, 113, 116
PrintException, 44
PrintWriter, 79, 81, 84
private, 38-39
programovací jazyk, 19
programového rozhraní, 59
programový objekt, 26
projekt, 14
protected, 39-41, 83
proud, 58
public, 38-41, 83
přetypování, 22
přetížená metoda, 54, 112
přetížený konstruktor, 104
Příkazový řádek, 75

R

Random, 96
reference, 31
reference na metodu, 51
regulární výraz, 101, 107-108, 128
regulární výraz, 105
request, 81
response, 81
rozhraní, 48, 81, 83

S

selektor, 67
servlet, 79, 93, 99
Servlet API, 110
složka, 14
soubor, 11
specifikace jazyka, 13

správně strukturovaný, 64
SSL, 70
Stack Overflow, 10
stack trace, 43-44, 114
StackOverflowError, 44
statická metoda, 33
statický atribut, 33
stereotyp, 119
Stream, 58
String, 33, 41
stromová struktura, 25
super, 104
switch, 123

T

table, 90, 93
ternární operátor, 106
this, 104
Throwable, 43
throws, 44, 83
tovární metoda, 33
try, 87
typová konverze, 22
třída, 26, 45, 71
třídy, 20

U

Ubuntu, 73, 78
Ujorm, 84
UML, 27
unchecked exceptions, 44
Unicode, 21
Uniform Resource Locator, 62
UnsupportedOperationException, 44
uzel, 25, 63

V

viditelnost, 50
VirtualBox, 77
void, 83
vstupní parametry, 40
výjimky, 37

W

webový kontejner, 79

webový server, 79

X

Xfce, 73

XHTML, 65

XML, 64, 88

XSD, 64

Z

zapouzdření, 38

zdroje, 45

znak, 20

znaková sada, 86, 100, 113, 115

znakový terminál, 75



Pavel Ponec (nar. 1965) vystudoval v Brně automatizaci na fakultě VUT a od té doby pracuje jako SW vývojář a analytik, od roku 2000 pracuje převážně v jazyce Java s webovými technologiemi. V roce 2008 získal certifikát Certified Programmer for the Java 2 Platform, Standard Edition 5.0 od společnosti Sun Microsystems. Je autorem několika open-source projektů.

učebnice jazyka

JAVA

a tvorba webových aplikací
pro samouky

Průvodce začátečníka programovacím jazykem Java od jeho základů až po interaktivní webové aplikace. V první části knihy čtenář získá znalost pojmů objektového programování, přehled o datových typech a užitečných metodách standardní knihovny jazyka, setká se i s pojmy potřebnými pro stavbu webových stránek. Druhá část knihy přináší výklad použitých programových konstrukcí na pozadí řešených aplikací. Java patří dlouhodobě mezi nejžádanější programovací jazyky na trhu práce v oboru informačních technologií.

```
5 |  
6 | □  
7 |  
8 |  
9 |  
0 |  
  
/** A constructor with default converter  
 * @param width A width of the board  
 * @param height a height of the board  
 * @param content An optional content taken from the method {@link #exportBo  
 */  
  
private final BitSet fields;  
/** An initial value is out of border */  
private final BoardPoint lastMove = new BoardPoint  
/** An instance of a Base64 converter */  
private final Base64Converter converter;  
/** Optional error message */  
private String errorMessage;  
/** A game over status */  
private boolean gameOver;
```

